

Passive Sonar Classification

Yesterday 7:04 PM

hey chatgpt i was going to build my project "passive sonar based target ship classification it is an binary classification wether target ship is present or not present "

That's a solid project, and it's actually a good fit for both research and placements because it combines **signal processing**, **machine learning/deep learning**, and **real-world defense applications**.

Project Title

Passive Sonar-Based Binary Target Ship Classification Using Deep Learning

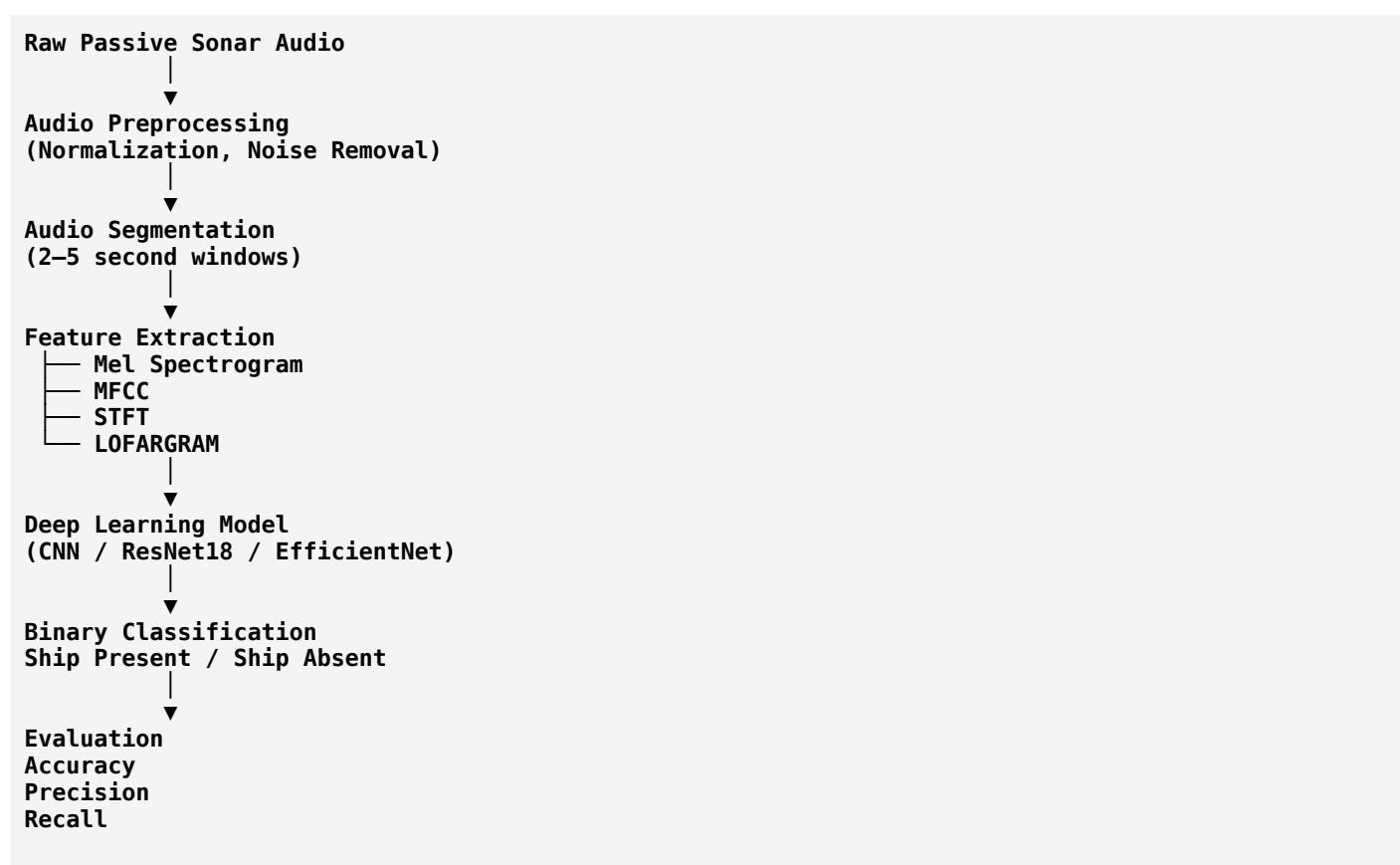
Objective:

Develop an AI model that analyzes passive sonar recordings and classifies them into:

- **Class 0:** Target Ship Not Present
- **Class 1:** Target Ship Present

Unlike active sonar, passive sonar only listens to underwater sounds without transmitting any signal, making it ideal for stealth surveillance.

Overall Workflow



Step 1: Dataset

You need two classes.

Positive Class

Audio containing target ships.

Possible datasets:

- DeepShip
- ShipsEar
- Passive sonar recordings from research datasets

Negative Class

Audio without ships.

Examples:

- Ocean ambient noise
- Rain
- Waves
- Wind
- Marine life
- Underwater silence

These can come from ambient ocean noise datasets or by extracting non-ship segments from longer recordings.

Step 2: Audio Preprocessing

Typical preprocessing:

```
Load WAV
↓
Convert to mono
↓
16 kHz sampling rate
↓
Normalize amplitude
↓
Band-pass filtering (optional)
↓
Segment into 3-second clips
```

Each clip becomes one training sample.

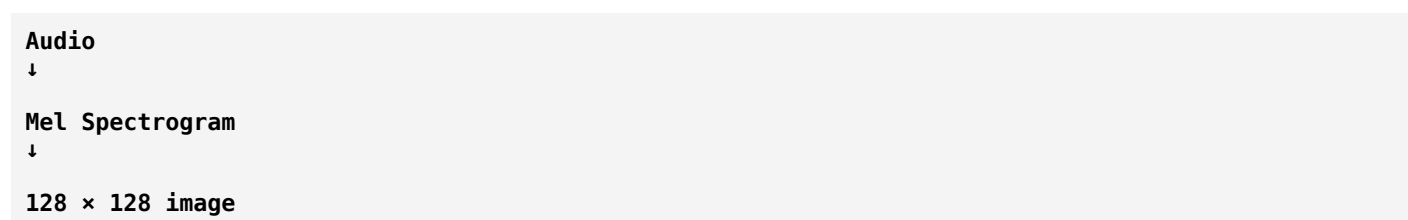
Step 3: Feature Extraction

Instead of feeding raw audio directly, convert it into image-like representations.

Mel Spectrogram

Most commonly used.

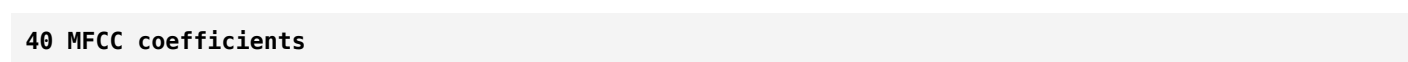
Input:



MFCC

Captures the spectral characteristics of sound.

Usually:



STFT

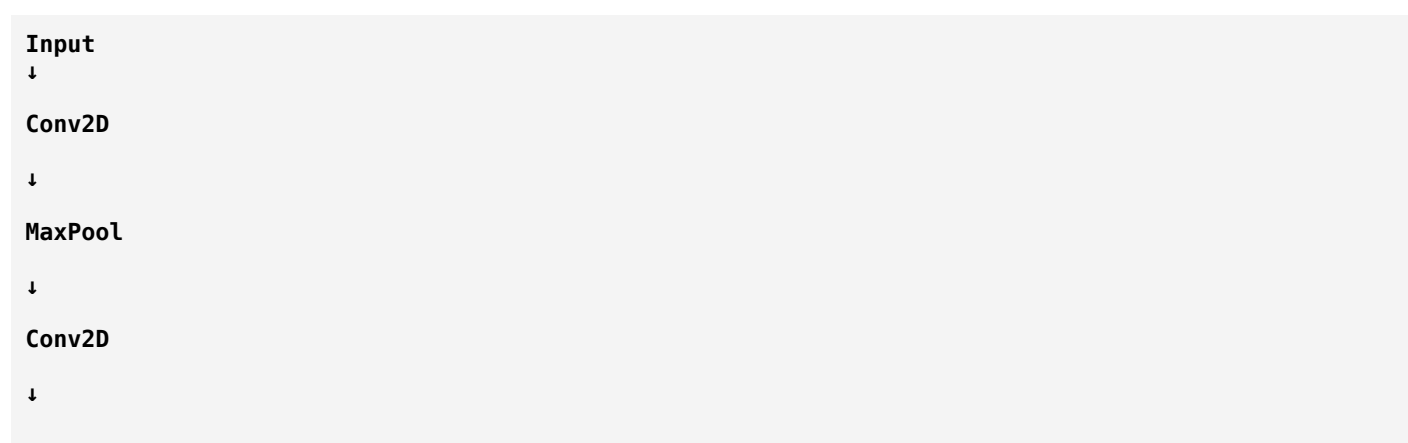
Shows how frequencies change over time.

LOFARGRAM

Very popular in naval sonar applications because it highlights machinery tonal components.

Step 4: Deep Learning Model

Option 1: Simple CNN



Flatten

↓

Dense

↓

Sigmoid

Good for a baseline model.

Option 2: ResNet18

Advantages:

- Faster training
- Good performance
- Suitable for limited datasets

Option 3: EfficientNet-B0

Advantages:

- High accuracy
- Lower parameter count
- Good balance between speed and performance

Option 4: MobileNetV3

Useful if deployment on embedded hardware or edge devices is a goal.

Binary Classification Output

Final layer:

```
Python
Dense(1, activation='sigmoid')
```

Prediction:

0 → No Ship

1 → Ship Present

Loss function:

Binary Cross Entropy

Training

Example settings:

```
Epochs = 30
Batch size = 32
Learning rate = 0.0001
Optimizer = Adam
```

Evaluation Metrics

Report:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC
- Confusion Matrix

Example:

	Predicted	
	Ship	No Ship
Actual Ship	96	4
Actual No	7	93

Suggested Folder Structure

```
PassiveSonar/
├── Dataset/
│   ├── Ship/
│   └── NoShip/
├── preprocessing/
├── feature_extraction/
├── models/
├── training/
├── evaluation/
├── results/
└── app/
```

Technology Stack

- **Programming Language:** Python
 - **Deep Learning:** TensorFlow/Keras or PyTorch
 - **Audio Processing:** Librosa
 - **Numerical Computing:** NumPy
 - **Visualization:** Matplotlib
 - **Machine Learning Utilities:** Scikit-learn
 - **Notebook Environment:** Jupyter Notebook or Google Colab
-

Research Novelty Ideas

To make your project more research-oriented, consider one or more of these enhancements:

- Compare multiple feature representations (Mel Spectrogram vs MFCC vs LOFARGRAM).
 - Benchmark different CNN architectures (CNN, ResNet18, EfficientNet-B0, MobileNetV3).
 - Apply data augmentation techniques such as time shifting, pitch shifting, and additive ocean noise.
 - Use attention mechanisms (e.g., CBAM or SE blocks) to improve feature learning.
 - Evaluate robustness across different signal-to-noise ratio (SNR) levels.
 - Use explainability methods like Grad-CAM to visualize which spectrogram regions influence predictions.
-

Expected Deliverables

1. Curated binary sonar dataset (Ship / No Ship).
2. Preprocessing pipeline for audio normalization and segmentation.
3. Feature extraction scripts for Mel Spectrogram, MFCC, and optionally LOFARGRAM.
4. Trained deep learning model with saved weights.
5. Performance evaluation with standard metrics and confusion matrix.
6. Inference script that accepts a new sonar recording and predicts **"Ship Present"** or **"Ship Not Present"**.
7. Research report documenting methodology, experiments, and results.

This design is appropriate for a final-year engineering project and can be extended into a publishable research prototype by incorporating robust evaluation and comparing multiple deep learning architectures.

ok as of now lets focus on intial stage related to digita audio processing steps

That's the right approach. In a passive sonar project, **Digital Audio Processing (DAP)** is the foundation. If the preprocessing is weak, even the best deep learning model will perform poorly.

Here's a structured roadmap for the **initial Digital Audio Processing stage**.

Passive Sonar Signal Processing Pipeline



Let's understand each step.

Step 1: Audio Acquisition

Input

Usually sonar recordings are stored as

- WAV
- FLAC
- MP3 (less common)
- Hydrophone recordings

Example

```
ship_001.wav
```

A WAV file consists of sampled amplitude values over time.

Example

```
Time (s)
0.00
0.01
```

```
0.02
0.03
...

Amplitude

0.24
0.35
0.48
0.27
...
```

Step 2: Load Audio

Read the waveform into memory.

Output

```
Amplitude array

[0.14,
0.18,
0.22,
0.30,
...
]
```

and

```
Sampling Rate

16000 Hz
```

Step 3: Resampling

Different datasets may have different sampling rates.

Example

Dataset A

```
44100 Hz
```

Dataset B

```
22050 Hz
```

Dataset C

```
16000 Hz
```

A deep learning model expects consistent input, so convert everything to one sampling rate.

Common choices

```
16000 Hz
```

or

22050 Hz

For passive sonar, **16 kHz** is often sufficient unless you're studying very high-frequency signatures.

Step 4: Convert Stereo to Mono

Some recordings have two channels.

Left

Right

Most sonar models use one channel.

Convert

Stereo

↓

Mono

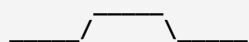
Usually by averaging

Mono = (Left + Right)/2

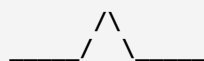
Step 5: Remove DC Offset

Sometimes the waveform is shifted above or below zero.

Bad waveform



Good waveform



Remove the mean value.

signal = signal - mean(signal)

Why?

- Better filtering
- Better FFT
- Better normalization

Step 6: Normalization

Different recordings have different loudness.

Example

Recording A

Amplitude
-0.2 to 0.2

Recording B

Amplitude
-0.9 to 0.9

Normalize both to the same scale.

Common normalization

-1
to
+1

This ensures the model focuses on patterns rather than recording volume.

Step 7: Noise Reduction (Optional)

Ocean recordings contain many background sounds:

- Waves
- Rain
- Wind
- Marine mammals
- Other ships
- Sensor noise

A simple approach is to apply a denoising filter while being careful not to remove useful ship signatures.

Step 8: Band-pass Filtering

Ships produce most of their acoustic energy in certain frequency ranges.

Instead of keeping every frequency,

0 Hz ----- 8000 Hz

retain only the frequencies of interest.

Example

50 Hz

↓

3000 Hz

Benefits

- Removes very low-frequency drift
- Removes high-frequency electronic noise
- Improves signal quality

The exact band depends on your dataset and target ships.

Step 9: Segmentation

A 5-minute recording is too long for a neural network.

Split into fixed-size windows.

Example

5 minute recording

↓

3 second clips

↓

100 clips

Each clip becomes one training sample.

Step 10: Windowing

Before spectral analysis, apply a window function to each segment.

Common choices

- Hamming
- Hann
- Blackman

Example

Raw segment



↓

Hamming Window



Windowing reduces **spectral leakage**, producing cleaner frequency analysis.

Step 11: Feature Extraction (Next Stage)

After preprocessing, convert each segment into representations suitable for machine learning.

Popular options:

- FFT (Fast Fourier Transform)
- STFT (Short-Time Fourier Transform)
- Mel Spectrogram
- MFCC
- LOFARGRAM (widely used in passive sonar)

Complete Initial Pipeline



Concepts you should understand before moving to feature extraction

Make sure you're comfortable with these fundamentals:

1. **Sampling** and sampling rate (Nyquist theorem).
2. **Quantization** and bit depth (e.g., 16-bit PCM).

3. **Digital audio representation** (how a waveform becomes an array of samples).
4. **Time domain vs. frequency domain.**
5. **Fourier Transform** and why it reveals frequency content.
6. **Filtering** (low-pass, high-pass, band-pass).
7. **Windowing** and spectral leakage.
8. **Segmentation** (frame size and overlap).

Once these concepts are clear, you'll be ready to move into **feature extraction**, where you'll learn why **FFT** → **STFT** → **Spectrogram** → **Mel Spectrogram** → **MFCC** → **LOFARGRAM** are used and how each contributes to passive sonar target detection.

ok in first what is the fourier tranform and what its role here

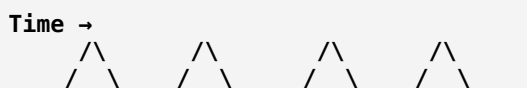
Excellent. Before learning **FFT (Fast Fourier Transform)**, you must first understand **Fourier Transform (FT)**. It is the single most important concept in digital signal processing, especially for sonar.

Let's build the intuition instead of just looking at formulas.

Why Do We Need Fourier Transform?

Imagine your hydrophone records this sound:

Ship Engine Sound



This graph is called the **time-domain signal**.

It tells us:

- How the sound changes over time.
- The amplitude (loudness) at each instant.

But here's the problem:

Can you tell which frequencies are present in this sound just by looking at the waveform?

No.

The waveform does not directly reveal:

- Whether the sound contains a 100 Hz component.
- Whether there's a 500 Hz propeller tone.
- Whether there are harmonics at 1000 Hz.

Think of Music

Imagine someone plays three notes together on a piano:

- C
- E
- G

Your ears hear a **single combined sound**.

But your brain can distinguish the individual notes.

Similarly, a sonar recording is a mixture of many frequencies:

```
Ship Sound
=
Engine Noise
+
Propeller Noise
+
Gearbox Noise
+
Water Noise
+
Wave Noise
+
Marine Life
```

The hydrophone records only the combined signal.

What Does Fourier Transform Do?

Fourier Transform answers this question:

"Which frequencies make up this signal, and how strong is each one?"

It converts:

Time Domain

into

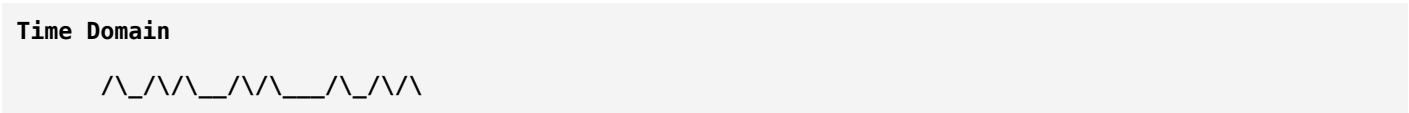
Frequency Domain

Example

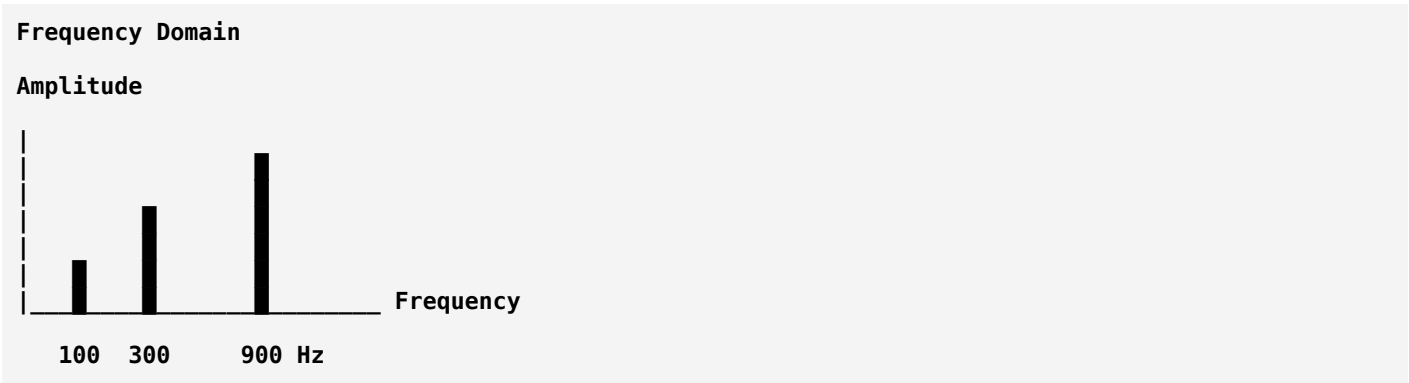
Suppose the recorded waveform is made of:

- 100 Hz engine vibration
- 300 Hz propeller vibration
- 900 Hz machinery vibration

The hydrophone records only one mixed waveform:



After applying the Fourier Transform:



Now you can clearly see the dominant frequencies.

Time Domain vs Frequency Domain

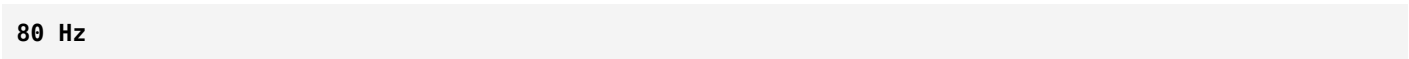
Time Domain	Frequency Domain
Shows amplitude over time	Shows amplitude at each frequency
Easy to hear	Easy to analyze
Complex waveform	Simple frequency peaks
Raw hydrophone output	Output of Fourier Transform

Why Is This Important for Passive Sonar?

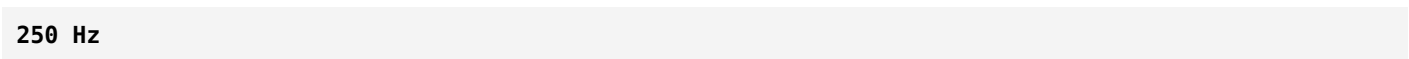
Ships produce characteristic frequencies.

For example:

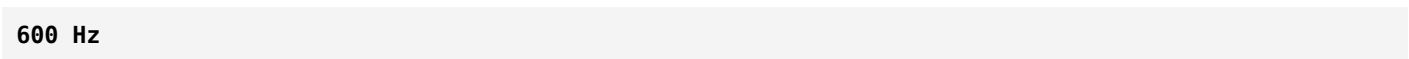
Engine



Propeller



Gearbox



Harmonics

1200 Hz
1800 Hz

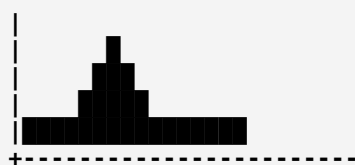
If you only look at the waveform, these are hidden.

Fourier Transform reveals them.

Example: Ship vs Ocean Noise

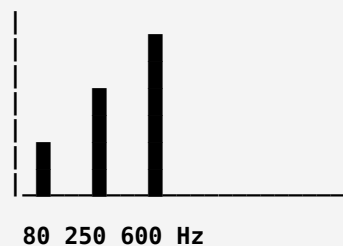
Ocean Noise

Frequency Spectrum



This is spread across many frequencies without strong tonal peaks.

Ship Present



Notice the distinct peaks caused by the ship's machinery.

This difference is what your AI model can learn.

The Mathematical Idea

Any complex signal can be represented as a sum of simple sine and cosine waves.

For example:

Signal

=
100 Hz sine wave
+
300 Hz sine wave
+
900 Hz sine wave

The Fourier Transform decomposes the signal into these individual frequency components.

An Analogy

Imagine a fruit smoothie.

The blender mixes:

- Apples
- Bananas
- Mangoes
- Milk

Once blended, it looks like one drink.

The Fourier Transform is like a magical machine that tells you:

- 2 apples
- 1 banana
- 3 mangoes
- 250 ml milk

Even though you only gave it the smoothie.

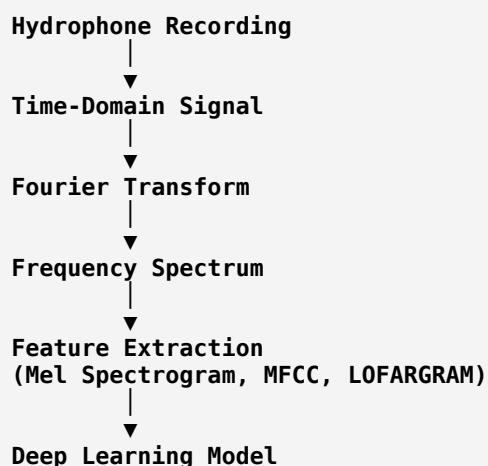
Similarly:

Recorded Audio

becomes

100 Hz → Strong
250 Hz → Medium
600 Hz → Weak
1000 Hz → Strong

Where Fourier Transform Fits in Your Pipeline



Notice that **Mel Spectrograms, MFCCs, and LOFARGRAMs all rely on frequency analysis**, which begins with the Fourier Transform.

Why Not Feed the Raw Waveform to AI?

You could, but for many passive sonar tasks it's less efficient.

A raw waveform mainly tells the model **when** the signal changes.

A frequency representation tells the model **what frequencies** are present, which often correspond directly to physical sources like engines, propellers, and pumps. These frequency patterns are much more informative for distinguishing **"ship present"** from **"ship absent."**

Key Takeaways

- **Time domain** shows how the signal changes over time.
- **Frequency domain** shows which frequencies are present and how strong they are.
- The **Fourier Transform** converts a signal from the time domain to the frequency domain.
- In passive sonar, different ship components generate characteristic frequency signatures.
- These signatures become the basis for advanced features such as **spectrograms, Mel spectrograms, MFCCs, and LOFARGRAMs**.

Next Step

Once you're comfortable with the Fourier Transform, the next concept is **Fast Fourier Transform (FFT)**. FFT is **not a different transform**—it's simply a much faster algorithm for computing the Fourier Transform on digital signals, making it practical for processing long sonar recordings efficiently.

ok tell about fft

Great! Now that you understand **Fourier Transform (FT)**, learning **Fast Fourier Transform (FFT)** becomes much easier.

The most important thing to remember is:

FFT is not a new transform. It is simply a fast algorithm to compute the Fourier Transform on digital data.

Why Was FFT Invented?

Imagine you have a 5-second sonar recording sampled at 16 kHz.

That means:

$$5 \times 16000 = 80,000 \text{ samples}$$

If we compute the Fourier Transform directly (using the Discrete Fourier Transform, DFT), the computer must perform an enormous number of calculations.

The DFT has a time complexity of:

$$O(N^2)$$

where **N** is the number of samples.

For 80,000 samples:

$$80,000 \times 80,000 \\ = 6.4 \text{ billion operations}$$

That's far too slow for practical use.

FFT solves this by reducing the complexity to:

$$O(N \log_2 N)$$

For 80,000 samples, that's roughly:

$$80,000 \times \log_2(80,000) \\ \approx 80,000 \times 16 \\ \approx 1.3 \text{ million operations}$$

Instead of **6.4 billion**, you need only about **1.3 million** operations—a massive speedup.

DFT vs FFT

DFT	FFT
Mathematical transform	Fast algorithm to compute the DFT
Slow	Very fast
$O(N^2)$	$O(N \log N)$
Rarely used directly	Used in almost every DSP application

So when people say:

"Apply FFT"

they actually mean:

"Compute the Fourier Transform using the Fast Fourier Transform algorithm."

How FFT Works (Intuition)

Suppose you have 8 audio samples:

[2, 5, 1, 3, 4, 7, 6, 8]

Instead of analyzing all 8 samples together, FFT uses a **divide-and-conquer** strategy.

It splits the data into:

Original Signal



Even Samples

2 1 4 6

Odd Samples

5 3 7 8

Then it keeps dividing:

Even

2 4

16

and

Odd

5 7

38

until only very small pieces remain.

It computes the transforms of these small pieces and then combines them efficiently.

This reuse of intermediate calculations is what makes FFT so fast.

If you've studied **Merge Sort**, the idea is similar:

- Divide the problem.
- Solve smaller parts.
- Combine the results.

FFT in Passive Sonar

Suppose your hydrophone records this signal:

Time Domain

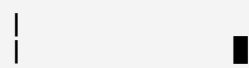
$$\wedge _ \wedge \wedge _ \wedge _ \wedge \wedge _$$

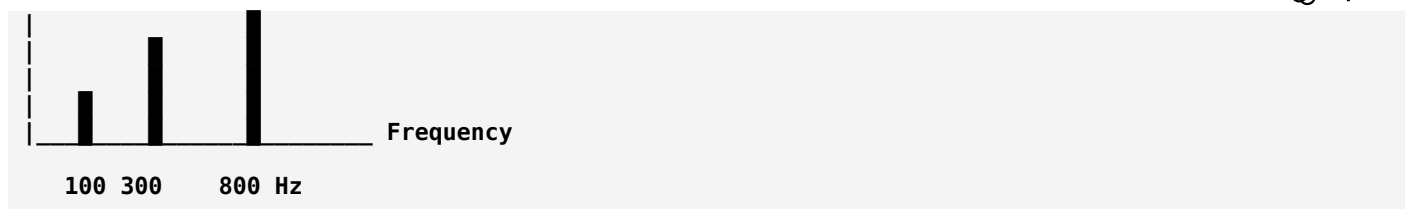
This waveform contains many sounds mixed together.

Applying FFT gives:

Frequency Spectrum

Amplitude





Now you know:

- 100 Hz is present.
- 300 Hz is present.
- 800 Hz is present.

These frequencies often correspond to physical components such as engines, propellers, or pumps.

What Does FFT Output?

Suppose you compute an FFT on a signal.

Instead of another waveform, you get values like:

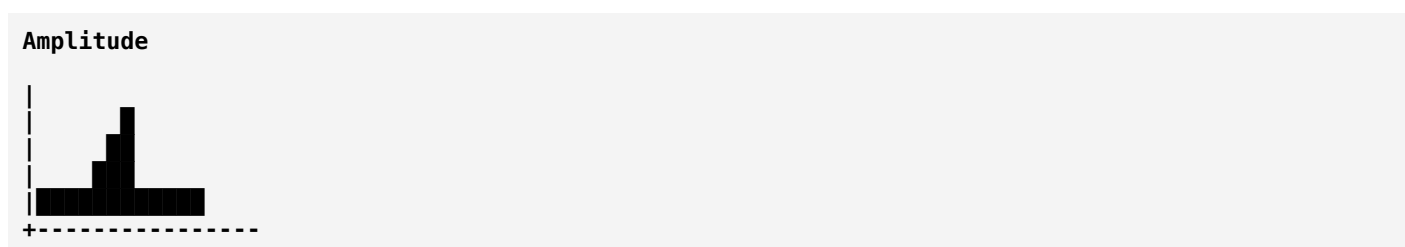
Frequency (Hz)	Magnitude
0	2
50	10
100	80
150	5
200	2
250	65
300	4

The **frequency** tells you *which frequency is present*, and the **magnitude** tells you *how strong that frequency is*.

Plotting these magnitudes gives the frequency spectrum.

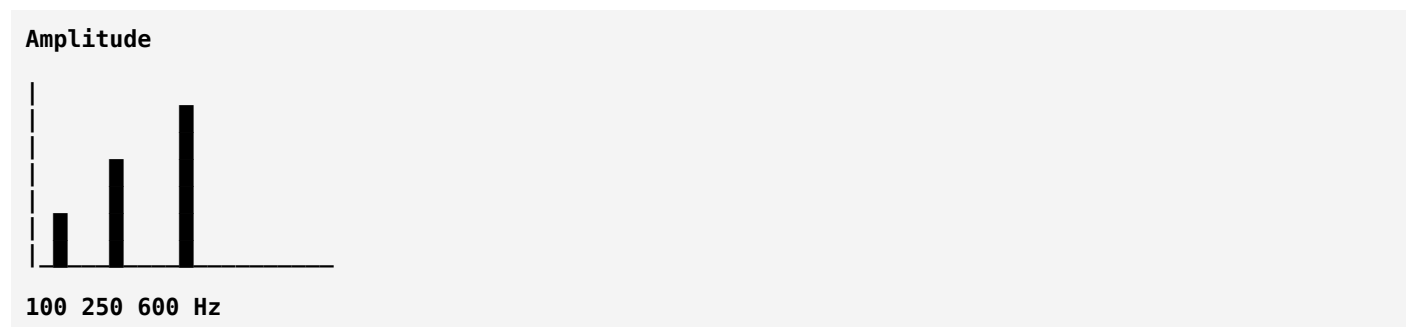
Example: Ship Detection

No Ship



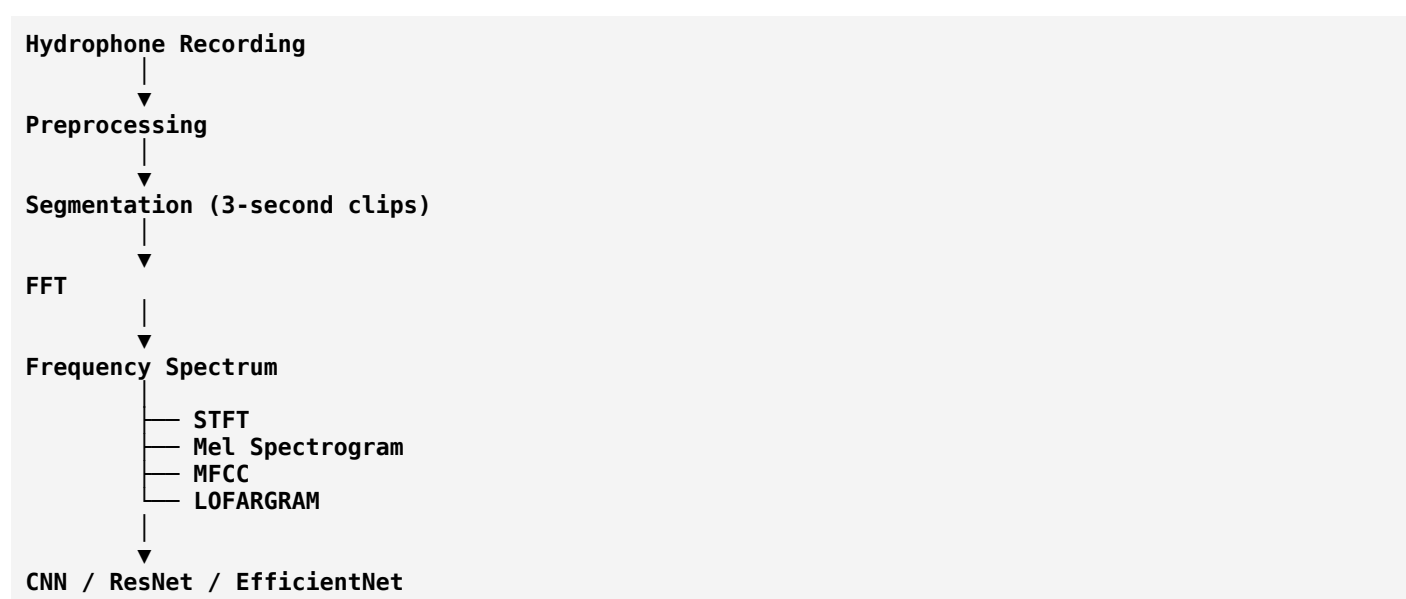
This is mostly random background noise.

Ship Present



Notice the sharp peaks at certain frequencies. These are often called **tonal components**, produced by rotating machinery and engines.

Where FFT Is Used in Your Project



Notice something important:

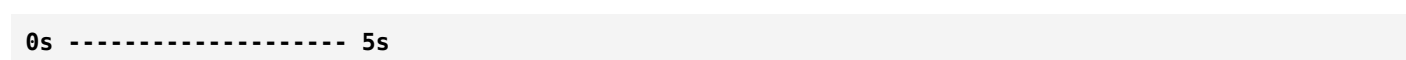
FFT is rarely the final feature used for deep learning.

Instead, FFT is the **building block** for more advanced representations.

Why Isn't FFT Alone Enough?

FFT analyzes the **entire signal at once**.

Imagine a 5-second recording:



If a ship passes only between **2 and 3 seconds**, the FFT will tell you that those frequencies exist, but **not when they occurred**.

This is a limitation because FFT has excellent **frequency resolution** but no **time resolution**.

To solve this, we use the **Short-Time Fourier Transform (STFT)**, which computes FFT over many short overlapping windows, allowing us to see **how the frequency content changes over time**.

Summary

- **Fourier Transform** converts a signal from the time domain to the frequency domain.
- **DFT** is the mathematical method used for digital signals.
- **FFT** is a fast algorithm for computing the DFT.
- FFT reduces computation from **$O(N^2)$** to **$O(N \log N)$** .
- In passive sonar, FFT reveals the frequency signatures produced by ship machinery.
- FFT is the foundation for **STFT, spectrograms, Mel spectrograms, MFCCs, and LOFARGRAMs**.

Next Concept

The next concept is **Short-Time Fourier Transform (STFT)**. It addresses the main limitation of FFT by answering not only "**Which frequencies are present?**" but also "**At what time do those frequencies occur?**" This makes it one of the most important tools in audio and passive sonar analysis.

ok tell me about stft

Excellent. **STFT (Short-Time Fourier Transform)** is one of the most important concepts in audio signal processing. If you understand STFT well, you'll naturally understand **spectrograms, Mel spectrograms, MFCCs, and LOFARGRAMs** because they are all built on top of it.

Let's build the intuition first.

Why Do We Need STFT?

We learned that FFT tells us:

"Which frequencies are present in the signal?"

But it cannot tell us:

"When did those frequencies occur?"

This is a major problem for sonar.

Example

Suppose you have a **6-second** passive sonar recording.

Time

0s-----6s

During these 6 seconds:

- 0-2 s → Only ocean noise
- 2-4 s → Ship passes by
- 4-6 s → Ocean noise again

Timeline:

0 2 4 6
Noise | Ship | Noise

If you apply **one FFT** to the entire 6-second signal:

Entire Signal
↓
FFT

Output:

100 Hz
250 Hz
600 Hz

The FFT tells you these frequencies exist somewhere in the recording.

But it **does not tell you**:

- Did they appear at 1 second?
- At 3 seconds?
- At 5 seconds?

It loses all timing information.

The Solution: STFT

Instead of analyzing the whole signal at once, STFT analyzes **small pieces (frames)** of the signal.

Think of watching a movie.

A movie is not one giant image.

It is made of many frames:

Movie
Frame1
Frame2
Frame3
Frame4
...

Similarly, STFT divides audio into many short frames.

Step 1: Split the Audio

Suppose we have:

6 seconds

Divide it into:

0–1 s

1–2 s

2–3 s

3–4 s

4–5 s

5–6 s

Each frame is analyzed separately.

Step 2: Apply FFT to Each Frame

Frame 1

FFT

↓

Spectrum 1

Frame 2

FFT

↓

Spectrum 2

Frame 3

FFT

↓

Spectrum 3

Continue until the end.

STFT Is Simply

STFT
=
Many FFTs

Each FFT is performed on a short time window.

That's why it's called:

Short-Time Fourier Transform

Visual Representation

Original waveform:

Time →
/\ /\ //\ /\ /_

Sliding window:

```
[#####]
  [#####]
    [#####]
      [#####]
```

Each window:

FFT

The results are stacked together.

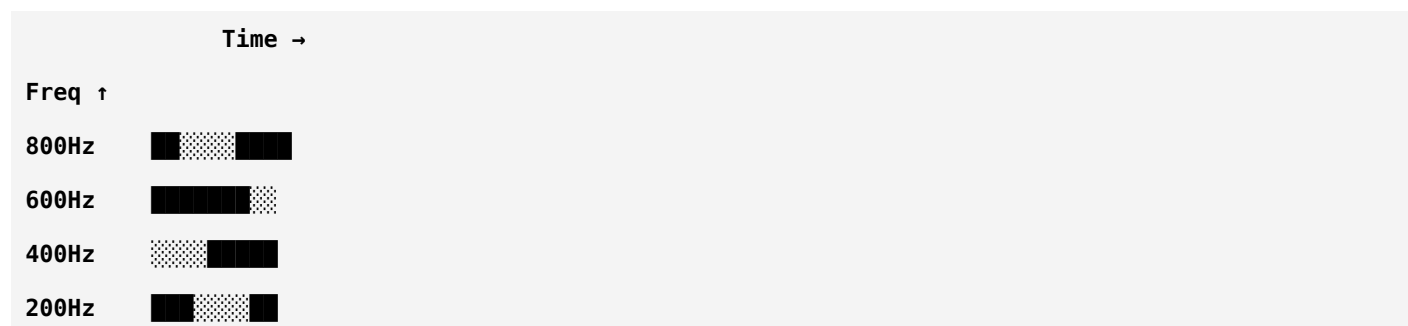
Final STFT Output

Instead of one spectrum,
we obtain **many spectra**.

Time →
Spectrum1
Spectrum2
Spectrum3
Spectrum4
Spectrum5

These are combined into a 2D representation.

STFT Matrix



Rows

↓

Frequency

Columns

↓

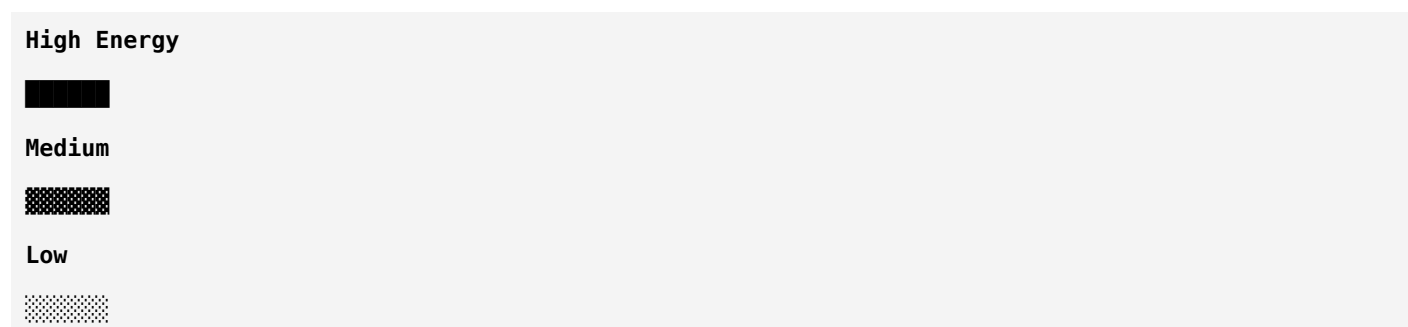
Time

Each cell represents:

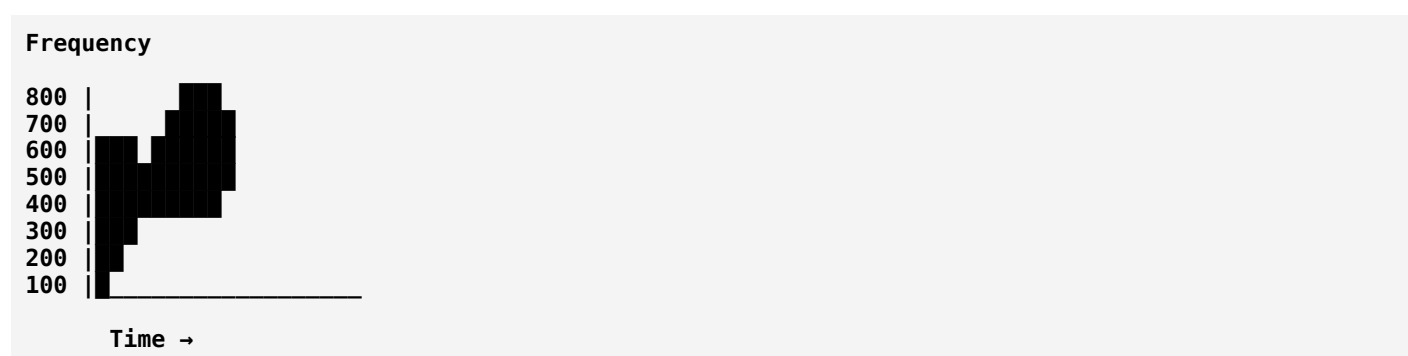
How much energy a particular frequency has at a particular time.

Spectrogram

If we color the STFT matrix,
we obtain a **spectrogram**.



Example:



This image is exactly what CNNs like to process.

Ship Detection Example

Suppose

0–2 sec

Ocean

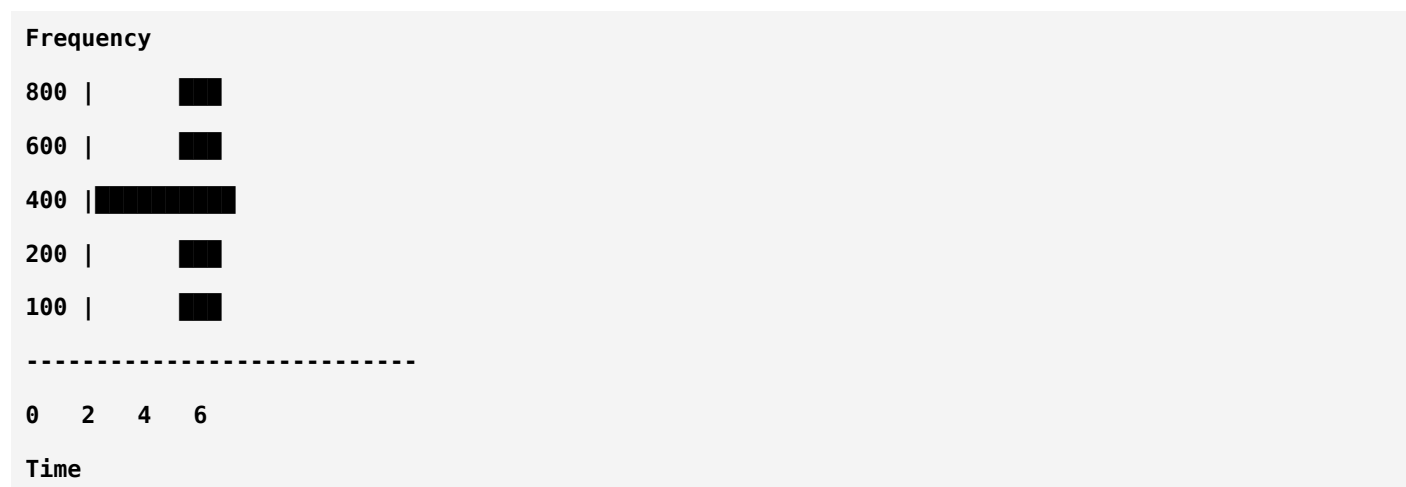
2–4 sec

Ship

4–6 sec

Ocean

The spectrogram might look like



Notice

Between

2–4 sec

new frequency bands appear.

That tells us

A ship is present only during that interval.

FFT cannot tell us this.

STFT can.

Window Size

This is an important concept.

Suppose your window is

50 ms

You get:

- Excellent time resolution
- Poor frequency resolution

Suppose the window is

500 ms

You get:

- Excellent frequency resolution
- Poor time resolution

This is called the **time-frequency trade-off**.

Example

Small window



You know exactly **when** something happened, but frequency estimates are less precise.

Large window



You know the frequencies more accurately, but not the exact timing.

Window Overlap

STFT usually uses overlapping windows.

Example

Window 1

#####

Window 2

#####

Window 3

#####

Common overlap

50%

or

75%

Why?

Because it prevents sudden changes between adjacent windows and produces a smoother, more continuous spectrogram.

STFT in Your Passive Sonar Project

Hydrophone Recording

↓

Preprocessing

↓

Segmentation

↓

Windowing (Hamming)

↓

STFT

↓

Spectrogram

↓

Mel Spectrogram

↓

CNN

Notice something important:

Mel Spectrogram = STFT + Mel Filter Bank

So if you understand STFT, you've already understood the core computation behind Mel spectrograms.

FFT vs STFT

FFT	STFT
One FFT for the entire signal	Many FFTs on short windows
Shows frequencies present	Shows frequencies over time
Output is a 1D frequency spectrum	Output is a 2D time-frequency matrix
No timing information	Preserves timing information
Good for stationary signals	Better for changing signals like speech and sonar

Why STFT Is Important for Ship Classification

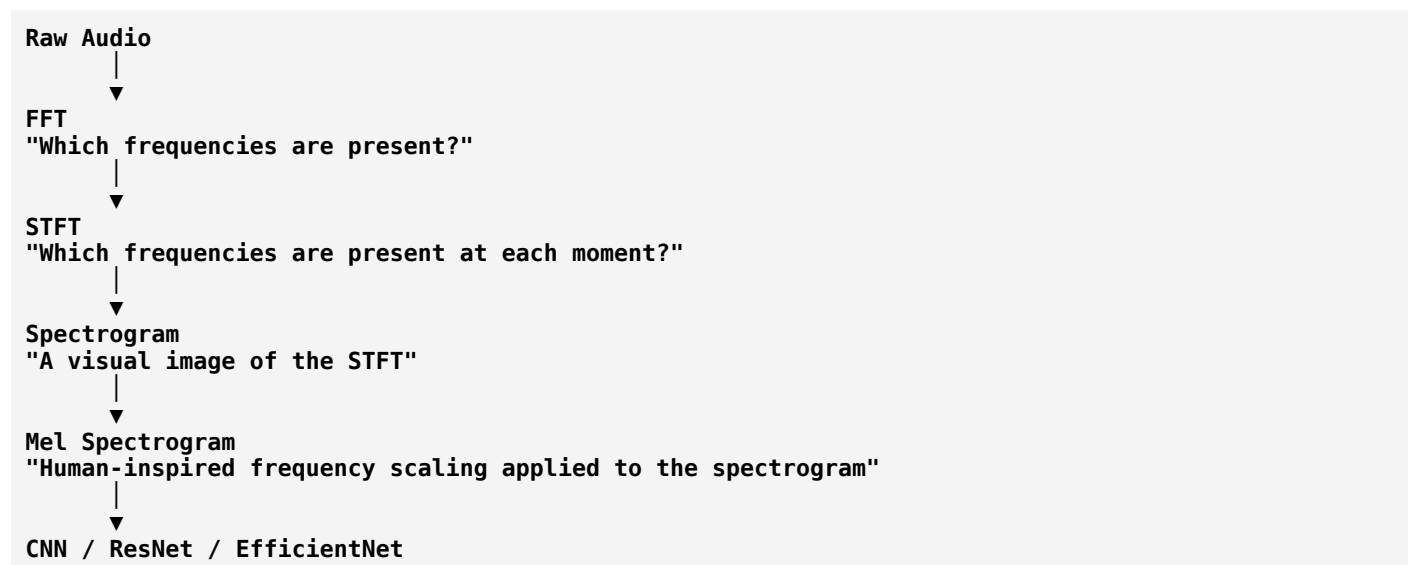
A ship's acoustic signature changes over time because of:

- Propeller speed variations.
- Engine load changes.
- Doppler shifts as the ship moves.
- Background ocean noise fluctuations.

STFT captures **both frequency and timing**, allowing your model to distinguish these dynamic patterns instead of seeing only an average frequency spectrum.

Summary

Think of the progression like this:



For your project, the next natural topic is **Spectrograms**. You'll see that a spectrogram isn't a new transform—it's simply a **visual representation of the STFT**, and understanding that makes the rest of the feature extraction pipeline much easier.

ok tell about spectrograms

Perfect. Now that you understand **FFT** and **STFT**, **Spectrograms** become very easy.

In fact, many people think STFT and Spectrogram are different, but they are closely related.

STFT is the computation.

Spectrogram is the visualization of the STFT.

Let's understand this step by step.

What is a Spectrogram?

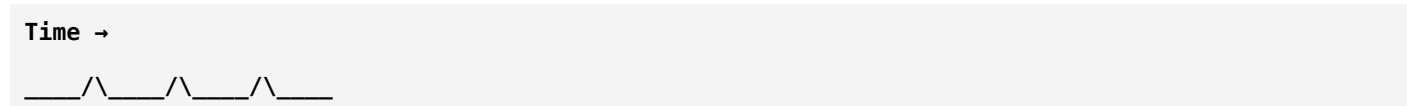
A **spectrogram** is a **2D image** that shows:

- **Time** (X-axis)
- **Frequency** (Y-axis)
- **Energy/Intensity** (Color or Brightness)

Instead of looking at the waveform, we look at **how the frequency content changes over time**.

Why Do We Need a Spectrogram?

Suppose your hydrophone records this signal:



This waveform doesn't tell us:

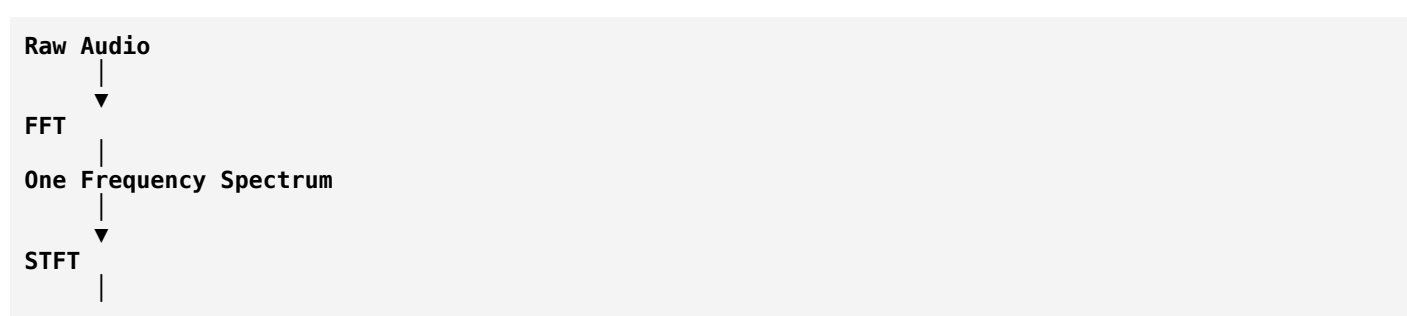
- Which frequencies are present?
- When they appear?
- Which frequencies are strongest?

After applying STFT, we get frequency information for every short window.

Instead of reading thousands of numbers, we convert them into an image.

That image is the **spectrogram**.

Relationship Between FFT, STFT, and Spectrogram



Many Frequency Spectra
↓
Spectrogram
(Image of all those spectra)

Think of it like this:

- **FFT** → One photograph
- **STFT** → Many photographs over time
- **Spectrogram** → A movie strip made from those photographs

How is a Spectrogram Created?

Suppose we have a 6-second audio.

Step 1: Split into windows

```
0-----1-----2-----3-----4-----5-----6
[Window1]
    [Window2]
        [Window3]
            [Window4]
                ...
```

Step 2: FFT on every window

Window 1

```
100 Hz → 20
200 Hz → 50
300 Hz → 10
```

Window 2

```
100 Hz → 18
200 Hz → 60
300 Hz → 15
```

Window 3

```
100 Hz → 70
200 Hz → 90
300 Hz → 80
```

Each window produces one frequency spectrum.

Step 3: Arrange them as columns

	Time →
Freq	
300Hz	10 15 80 ...
200Hz	50 60 90 ...
100Hz	20 18 70 ...

This matrix is the STFT magnitude.

Step 4: Convert numbers into colors

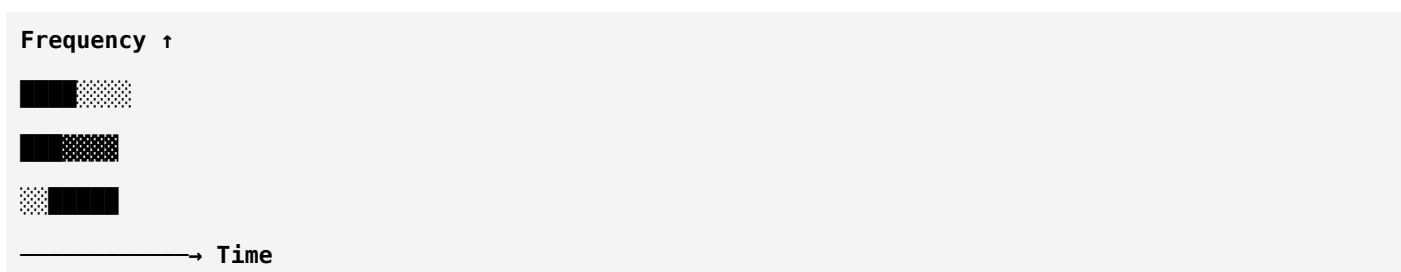
Suppose

0 = Black
50 = Gray
100 = White

Then

10 → Dark
50 → Gray
90 → Bright

The result becomes



This image is called a **spectrogram**.

Understanding the Axes

	Time →
	0s 1s 2s 3s 4s
800Hz	
600Hz	
400Hz	
200Hz	

100Hz

X-axis

Represents

Time

Y-axis

Represents

Frequency

Color

Represents

Signal Energy

or

Magnitude

Bright color

↓

Strong frequency

Dark color

↓

Weak frequency

Real Passive Sonar Example

Suppose

0–2 sec

Ocean Noise

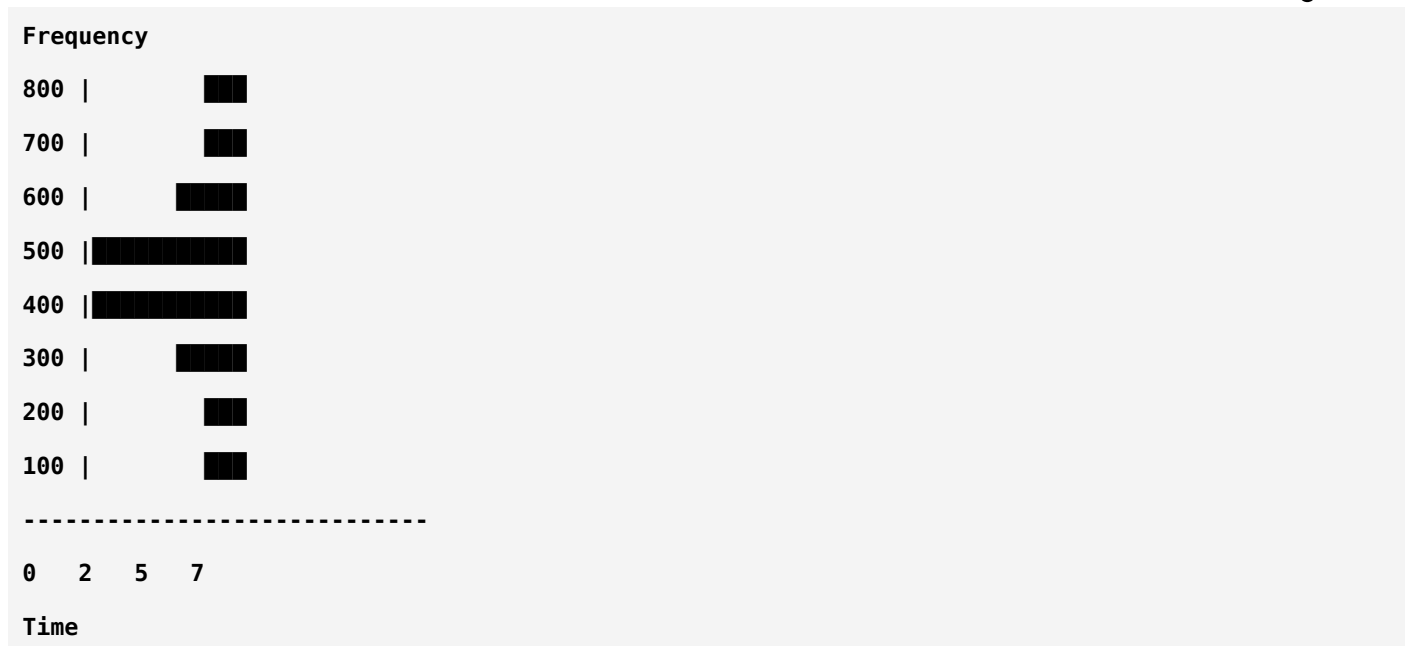
2–5 sec

Ship Passing

5–7 sec

Ocean Noise

The spectrogram might look like



Notice

Only during

2–5 sec

new frequencies appear.

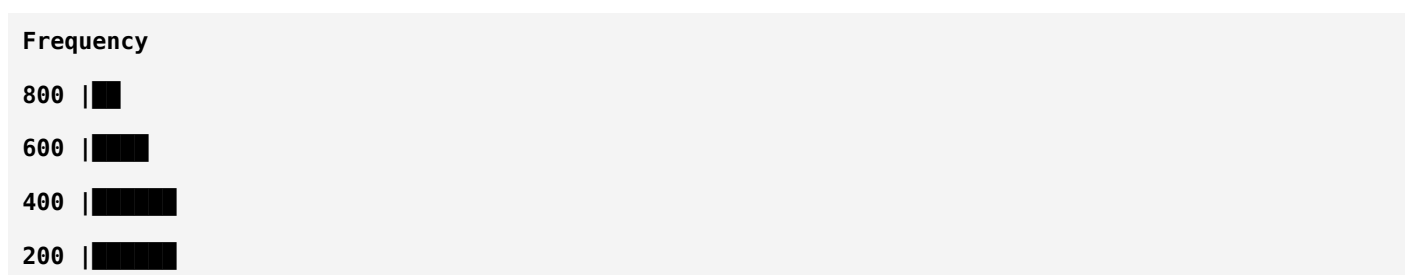
This indicates a ship is present.

Ship Signature

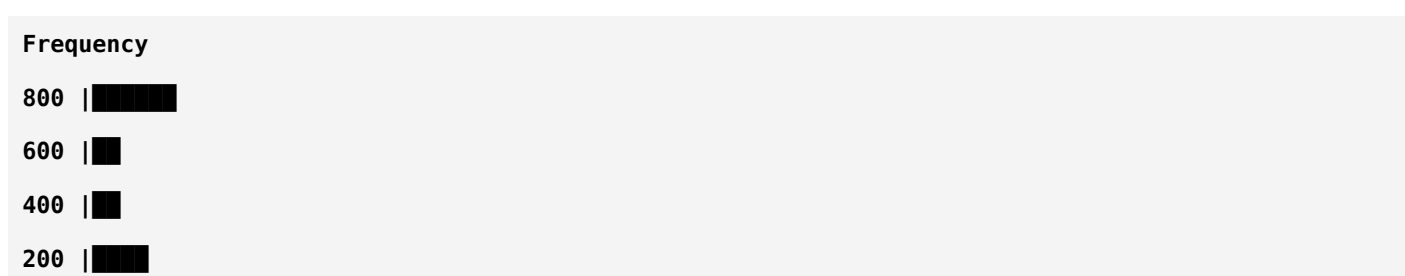
Different ships create different frequency patterns.

For example

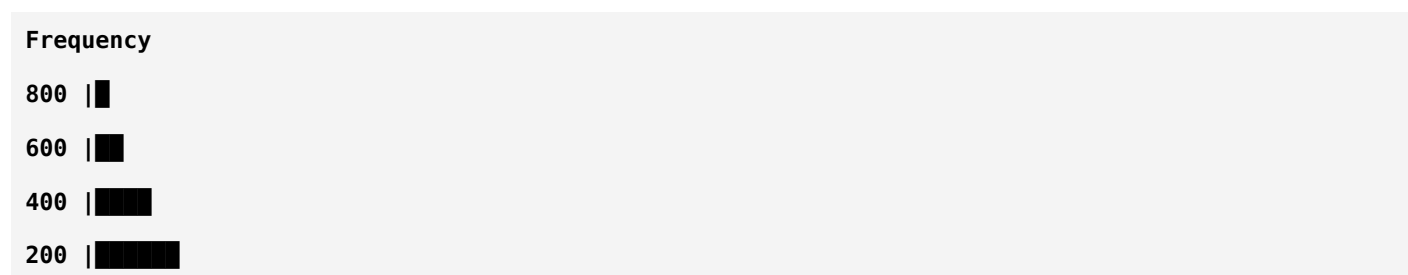
Cargo Ship



Fishing Boat



Submarine



A CNN can learn these visual patterns just like it learns patterns in photographs.

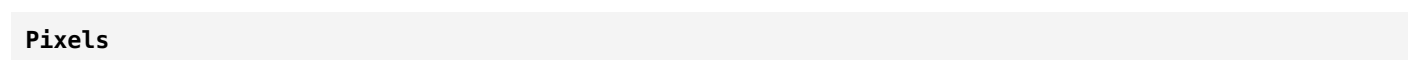
Why CNN Works So Well

A CNN does not know anything about sound.

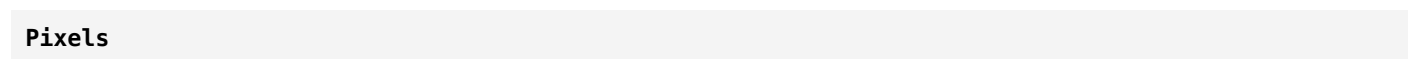
It only sees images.

Example

Cat Image



Spectrogram



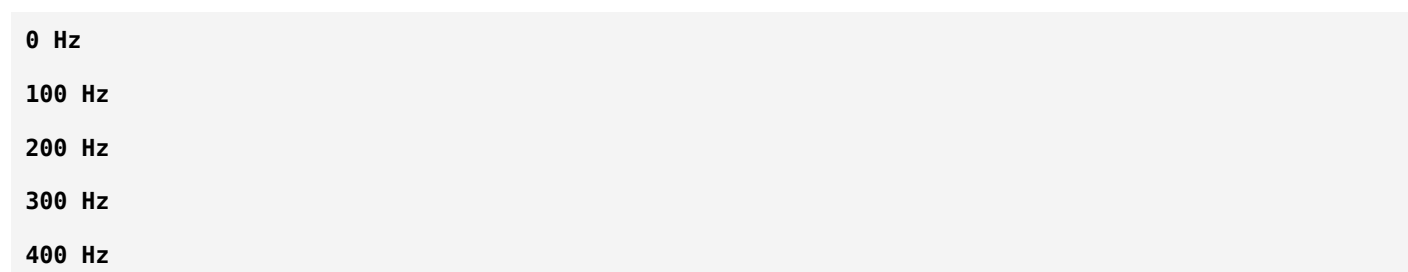
For the CNN, both are simply matrices of pixel values.

This is why spectrogram-based classification is so successful in audio tasks.

Spectrogram Types

1. Linear Spectrogram

Uses the original frequency scale.



Every frequency interval is equally spaced.

2. Log Spectrogram

Instead of raw magnitude

Magnitude

we display

Log(Magnitude)

Why?

Suppose

Frequency
100 Hz → 5
200 Hz → 1000

Without a logarithm

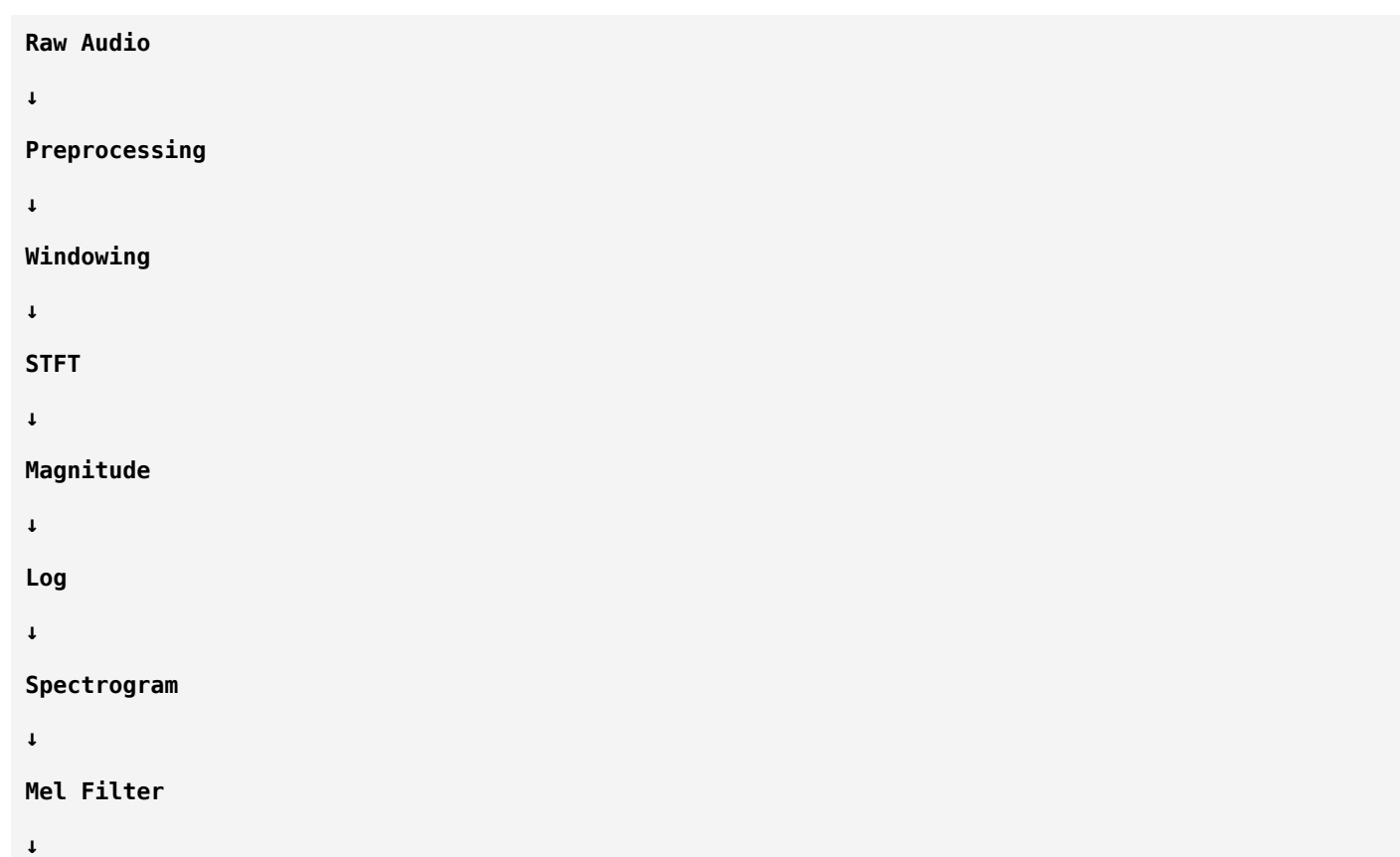
5
1000

The smaller signal is almost invisible.

Using a logarithmic scale makes both weak and strong components visible.

Most real-world audio systems use **log-magnitude spectrograms** because they reveal subtle details.

Where Spectrogram Fits



Mel Spectrogram

↓

CNN

Notice

Spectrogram is simply

$\text{Magnitude}(\text{STFT})$

Sometimes

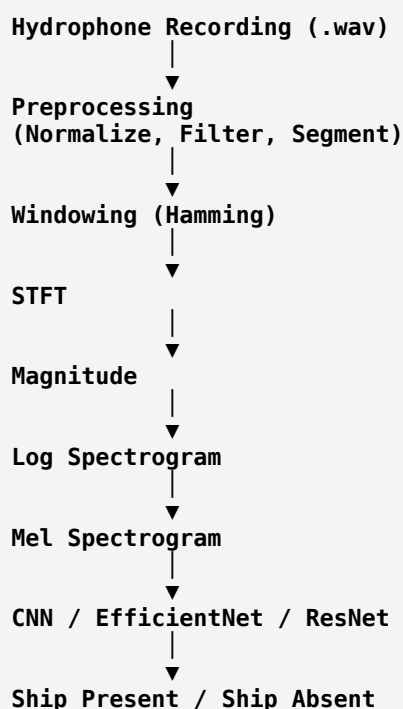
$\text{Log}(\text{Magnitude}(\text{STFT}))$

FFT vs STFT vs Spectrogram

FFT	STFT	Spectrogram
One transform on the whole signal	Many FFTs over short windows	Image of the STFT magnitude
Output: 1D frequency spectrum	Output: 2D time-frequency matrix	Output: 2D image
No timing information	Preserves timing	Easy for humans and CNNs to interpret

Spectrogram in Your Passive Sonar Project

Your processing pipeline will look like this:



Key Takeaways

- A **spectrogram is not a new transform**; it is a **visual representation of the STFT**.
- It answers three questions at once:
 - **When** did something happen? (Time)
 - **What frequency** was present? (Frequency)
 - **How strong** was it? (Color intensity)
- Spectrograms convert audio into an image, making them ideal inputs for deep learning models such as CNNs.

What's Next?

The next topic is **Mel Spectrogram**, which is the feature representation most commonly used in audio AI. It takes a regular spectrogram and transforms the frequency axis into the **Mel scale**, creating a representation that is often more compact and effective for machine learning. While the Mel scale was inspired by human hearing, it can also be a useful feature representation for sonar because it emphasizes broader spectral patterns and reduces dimensionality. We'll also discuss when a **standard spectrogram** or **LOFARGRAM** may be more appropriate for passive sonar applications.

ok next tell about Mel spectrogram

Excellent! Now we've reached one of the most important features in audio AI.

So far you've learned:

```
Raw Audio
  ↓
FFT
  ↓
STFT
  ↓
Spectrogram
```

Now we'll learn:

```
Spectrogram
  ↓
Mel Filter Bank
  ↓
Mel Spectrogram
```

Notice something important:

A Mel Spectrogram is NOT a completely new transform.

It is simply a **Spectrogram** whose frequency axis has been transformed using the **Mel scale**.

Why Do We Need Mel Spectrogram?

Suppose our STFT produces frequencies from

```
0 Hz ----- 8000 Hz
```

Imagine there are **256 frequency bins**.

```
0
31
62
93
124
...
8000 Hz
```

Every frequency interval is equally spaced.

This is called a **Linear Frequency Scale**.

Problem

Suppose these frequencies are present:

```
100 Hz
110 Hz
120 Hz
130 Hz
```

and

```
6000 Hz
6010 Hz
6020 Hz
6030 Hz
```

Do we really need to distinguish every 10 Hz difference at high frequencies?

Often, **no**.

Many nearby frequencies contain very similar information.

Instead of keeping every frequency separately, we can group them.

This reduces:

- Noise
- Data size
- Model complexity

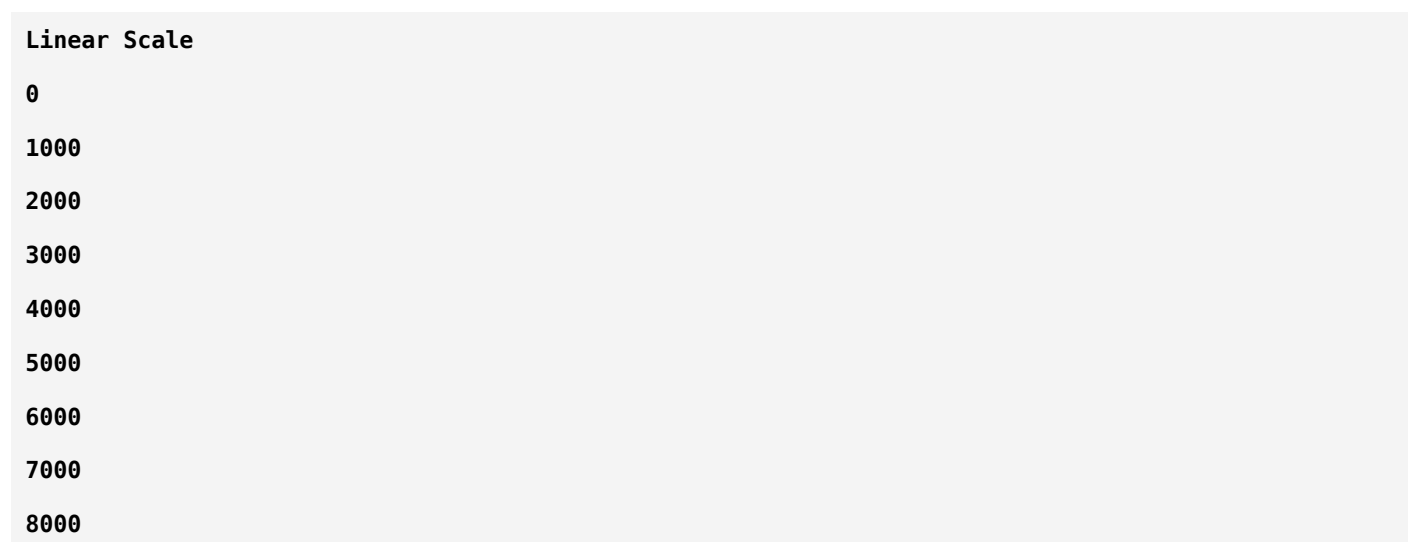
What is the Mel Scale?

The **Mel scale** is a frequency scale that is:

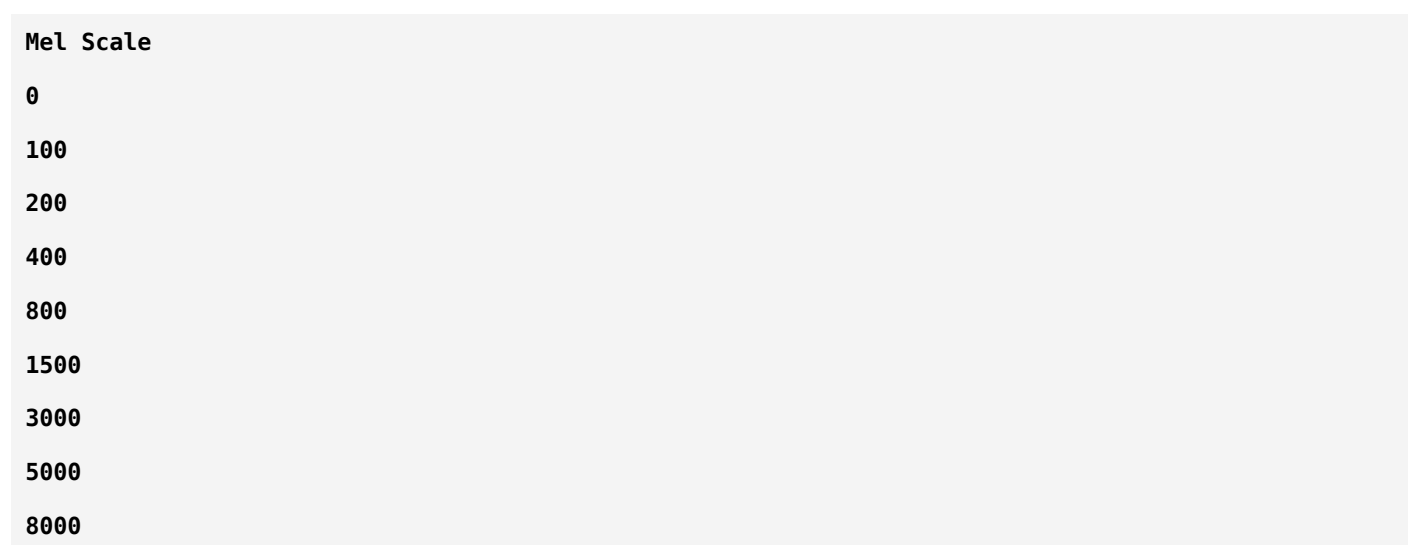
- More detailed at **low frequencies**

- More compressed at **high frequencies**

Instead of



we get something like



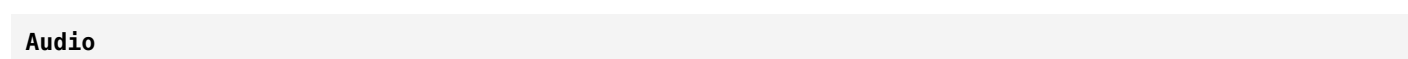
Notice:

- Low frequencies are represented with finer resolution.
- High frequencies are grouped into wider bands.

How is a Mel Spectrogram Created?

Step 1

Load audio



↓

Step 2

Compute STFT

Time × Frequency Matrix



Step 3

Apply Mel Filter Bank



Step 4

Take logarithm



Step 5

Mel Spectrogram

What is a Mel Filter Bank?

This is the most important idea.

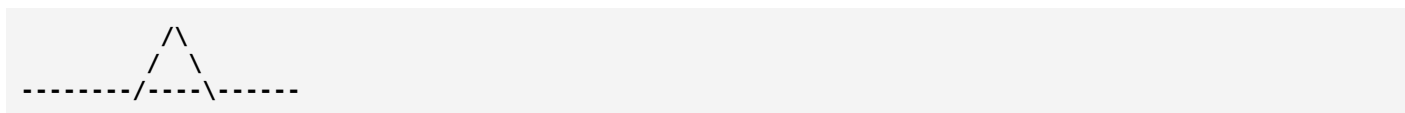
Suppose the STFT gives:

0
100
200
300
400
500
600
700
800 Hz

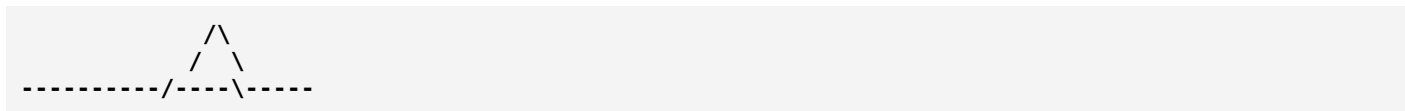
Instead of keeping every frequency separately,
we place overlapping triangular filters over the spectrum.
Example:



Another filter



Another



Each triangle combines nearby frequencies into one value.

Instead of:

256 frequency bins

we might keep only:

64 Mel bands

or

128 Mel bands

Visual Example

Suppose STFT gives:

Frequency	Energy
100	10
200	25
300	30
400	20
500	15
600	40
700	35
800	10

Instead of keeping all eight values,

Mel filters combine them:

Mel Band 1

100–250 Hz

↓

Average Energy

18

Mel Band 2

250–450 Hz

↓

Average

25

Mel Band 3

450–700 Hz

↓

Average

30

Now we have fewer, smoother values.

Mel Spectrogram

Normal Spectrogram

Frequency

800

700

600

500

400

300

200

100

Mel Spectrogram

Mel Band

64

63

62

...

2

1

Instead of frequency values,
the Y-axis now represents **Mel bands**.

Why Use Log Scale?

After applying Mel filters,
we take

Log (Energy)

Suppose

Energy

5

5000

Without logarithm
the smaller value becomes almost invisible.

Using logarithms

Log (5)

Log (5000)

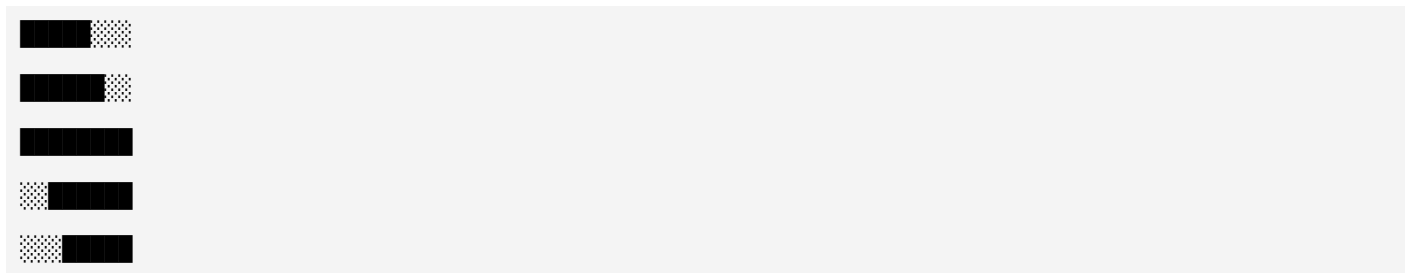
compresses the dynamic range, making both weak and strong components easier to analyze.

Complete Pipeline

```
Raw Audio
  ↓
Preprocessing
  ↓
Windowing
  ↓
STFT
  ↓
Magnitude
  ↓
Mel Filter Bank
  ↓
Log
  ↓
Mel Spectrogram
```

Why CNN Likes Mel Spectrograms

Suppose this is your Mel Spectrogram.



To a CNN,

this is just an image.

The CNN learns patterns like:

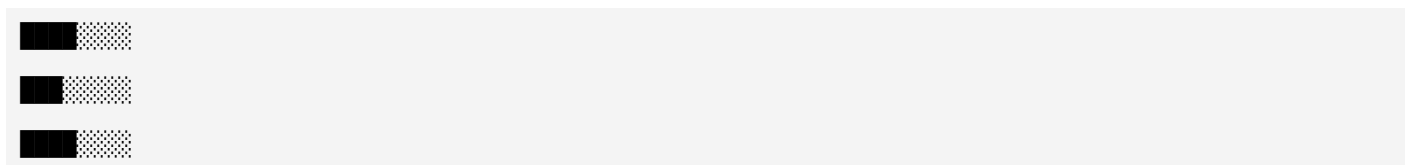
- Horizontal lines
- Vertical lines
- Bright regions
- Shapes

Different ships produce different Mel Spectrogram patterns.

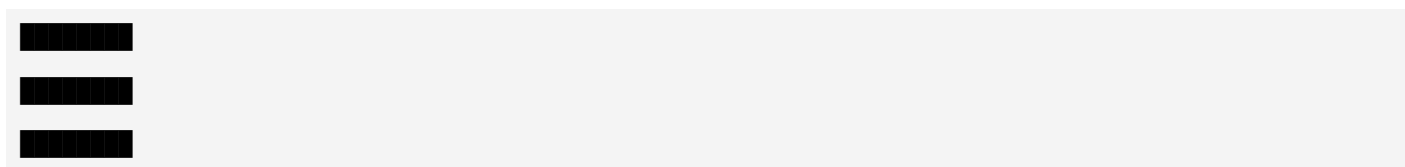
Mel Spectrogram in Ship Detection

Suppose

Ocean Noise



Ship Present



The CNN learns these visual differences.

Is Mel Spectrogram Good for Passive Sonar?

This is a very important question.

For **speech recognition**, Mel spectrograms are excellent because the Mel scale is designed to approximate how humans perceive pitch.

For **passive sonar**, the answer is more nuanced:

Advantages

- Reduces the number of frequency bins (lower dimensionality).
- Smooths noisy spectra.
- Often works well with CNN-based classifiers.
- Computationally efficient.

Disadvantages

A ship's machinery produces **precise tonal frequencies** (for example, engine harmonics and propeller blade rates). Compressing the frequency axis with Mel filters can reduce the resolution of these narrowband features, potentially losing information that is valuable for sonar analysis.

Because of this, many passive sonar systems use:

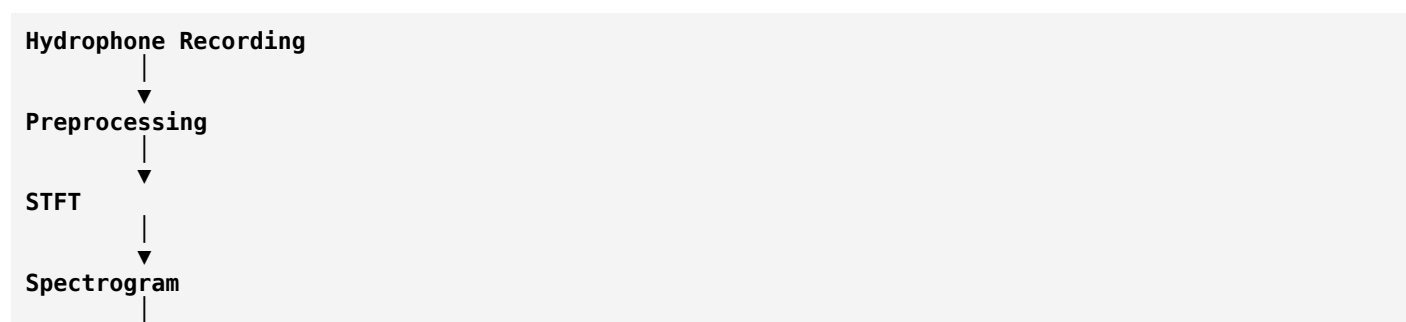
- **STFT Spectrograms**
- **LOFARGRAMs**
- **Constant-Q Transform (in some cases)**

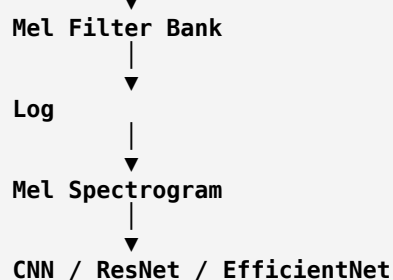
instead of, or alongside, Mel spectrograms.

Comparison

Feature	FFT	STFT	Spectrogram	Mel Spectrogram
Output	Frequency Spectrum	Time-Frequency Matrix	Image	Compressed Image
Time Information	✗	✓	✓	✓
Frequency Scale	Linear	Linear	Linear	Mel (nonlinear)
Good for CNN	✗	✗	✓	✓
Widely Used	DSP	DSP	Audio AI	Speech & Audio AI

Where Does Mel Spectrogram Fit?





For Your Passive Sonar Project

Since your goal is **binary classification (Ship Present vs Ship Absent)**, I recommend experimenting with multiple feature representations rather than assuming one is best:

1. **Log Spectrogram** (strong baseline).
2. **Mel Spectrogram** (compact and CNN-friendly).
3. **LOFARGRAM** (traditionally favored in passive sonar because it preserves narrowband tonal signatures).

You can train the same CNN or EfficientNet model on each representation and compare accuracy, precision, recall, and F1-score. This comparison itself can become a valuable part of your project or research paper.

Next Step

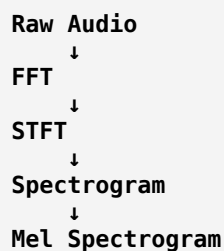
The next concept is **MFCC (Mel-Frequency Cepstral Coefficients)**.

MFCCs are **not images** like spectrograms. Instead, they compress a Mel spectrogram into a small set of numerical coefficients that summarize the spectral characteristics of each audio frame. They're widely used in speech recognition and can also be evaluated as features for passive sonar classification.

next MFcc

Excellent! You're now reaching the most advanced traditional audio feature. Once you understand **MFCC**, you'll have covered the core digital audio processing pipeline used in many audio AI systems.

So far you've learned:



Now we move to:



DCT (Discrete Cosine Transform)



MFCC

Notice something important:

MFCC is NOT another spectrogram.

It is a **compressed numerical representation** of a Mel Spectrogram.

Why Do We Need MFCC?

Suppose your Mel Spectrogram has

128 Mel Bands

For every time frame, you have

128 numbers

If your recording has 500 time frames,
your data size becomes

**500 × 128
=
64,000 values**

Many of these values are redundant because neighboring Mel bands are highly correlated.
Instead of storing all 128 values, MFCC compresses them.

The Main Idea

Think of a photograph.

Original image

4000 × 3000 pixels

Compress it into JPEG.

The image still looks almost the same,
but occupies much less memory.

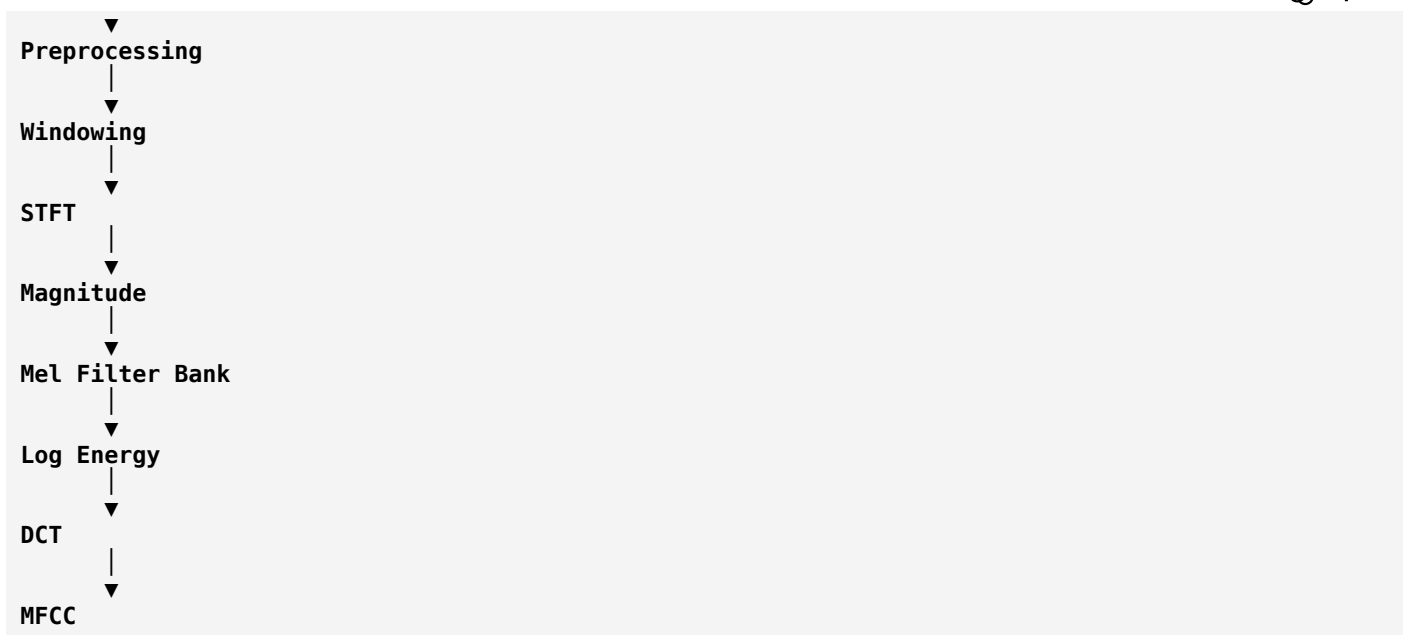
MFCC does something similar.

It compresses the Mel Spectrogram while preserving its most important information.

Complete MFCC Pipeline

Raw Audio





Everything is the same as the Mel Spectrogram pipeline until the final step.
The only new operation is the **Discrete Cosine Transform (DCT)**.

What is DCT?

Suppose the Mel Spectrogram gives

Frame 1

[10, 12, 15, 16, 17, 19, 20, 22]

These numbers are very similar.

Instead of keeping all of them,

DCT transforms them into

[55, 8, 2, 1, 0, 0, 0, 0]

Notice

Most important information is concentrated in the first few numbers.

The remaining coefficients are very small.

Therefore,

instead of storing

128 coefficients

we keep only

13 coefficients

This is the key idea behind MFCC.

Why DCT?

The DCT "decorrelates" the Mel-band energies.

Imagine you have marks in similar subjects:

Subject	Marks
Physics	90
Chemistry	91
Electronics	92

These values are closely related.

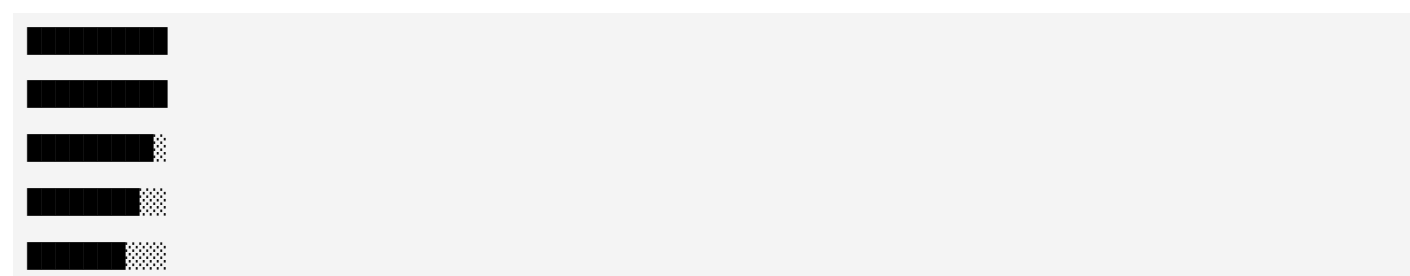
Instead of storing all three separately, you might summarize them as:

- Average ≈ 91
- Small differences

Similarly, DCT summarizes correlated Mel-band energies into a compact set of coefficients.

Visual Example

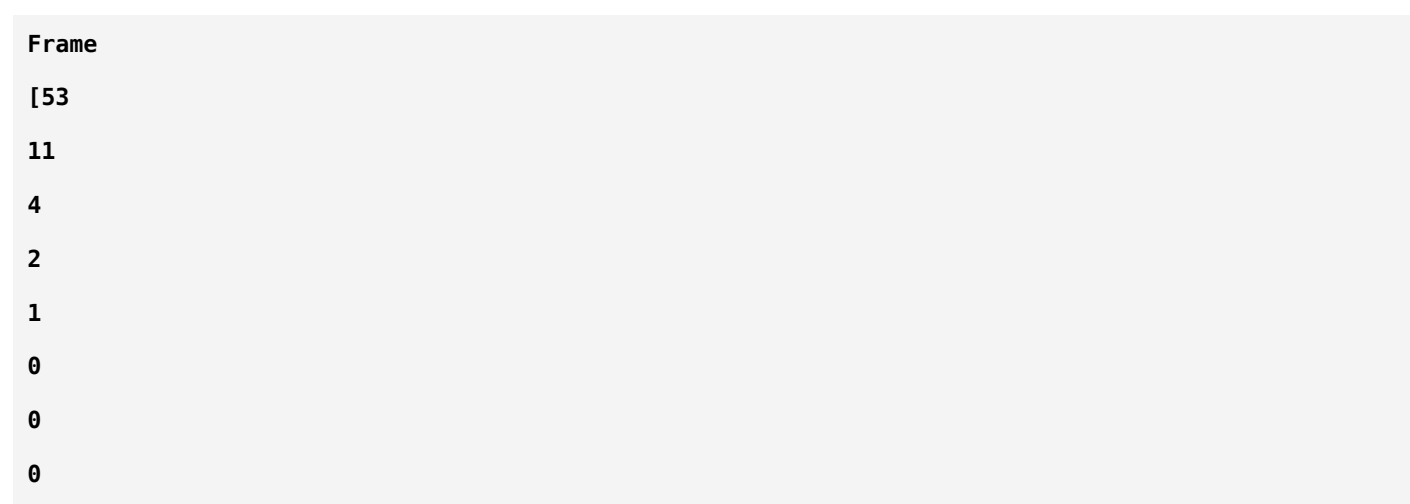
Mel Spectrogram

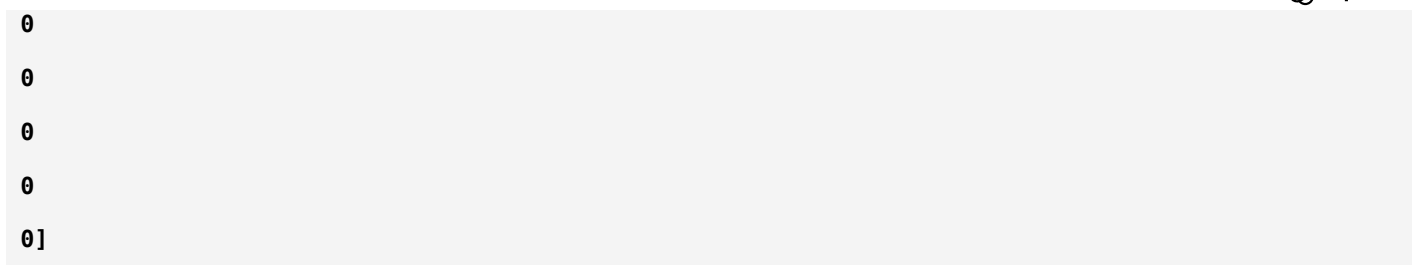


Each frame contains

128 values

After DCT





Only the first coefficients carry most of the useful information.

What Does MFCC Look Like?

Unlike a spectrogram,

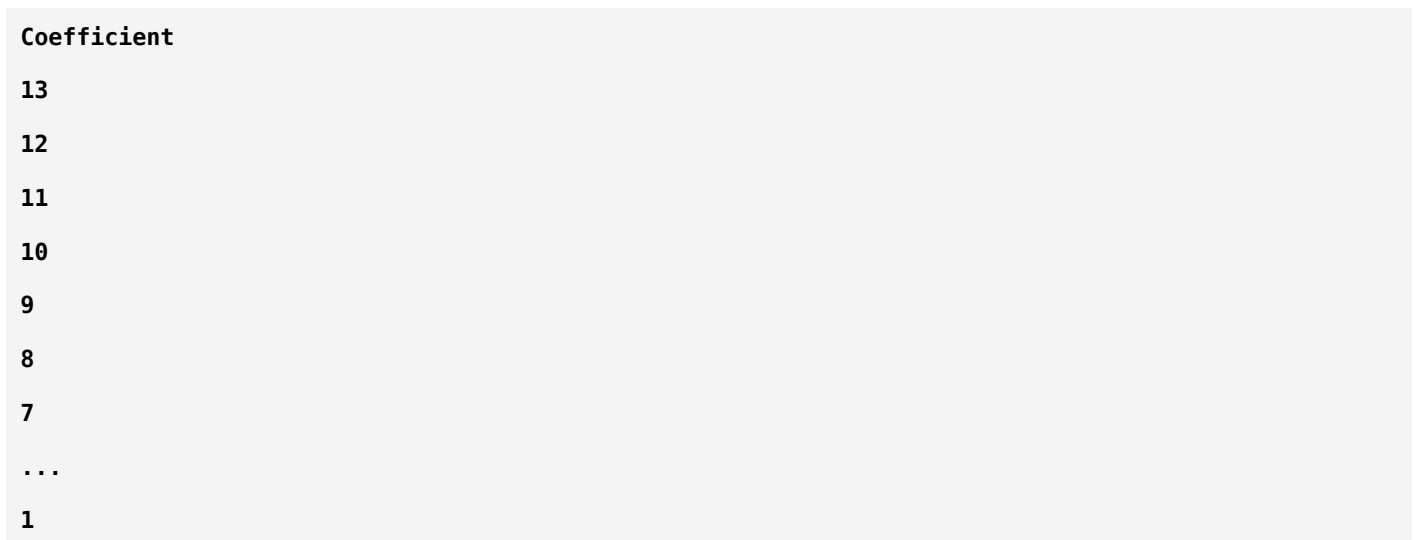
MFCC is **not displayed as frequencies**.

Instead

Y-axis



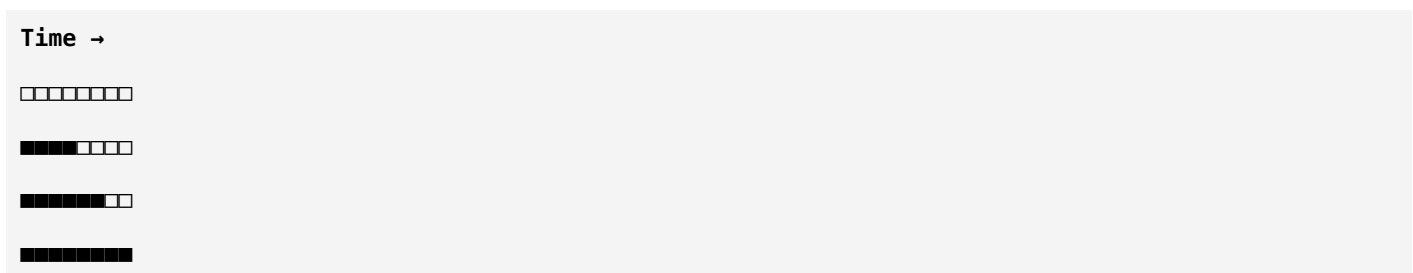
Coefficient Number



X-axis



Time



Each row is an MFCC coefficient rather than a frequency band.

Typical Number of MFCCs

Most applications keep

13

or

20

coefficients.

For example

Instead of

128 Mel Bands

↓

Store

13 MFCCs

This greatly reduces storage and computation.

Why MFCC Became Popular

In the early days of speech recognition,

computers had very limited memory.

Instead of processing huge spectrograms,

researchers compressed them into MFCCs.

Advantages:

- Smaller feature vectors.
- Faster machine learning.
- Reduced noise.
- Lower memory usage.

MFCCs became the standard feature for speech recognition for many years.

MFCC Example

Suppose your Mel Spectrogram contains

Frame 1

10

15
18
22
25
28
30
32

After DCT

MFCC
54
9
3
1
0
0
0
0

Instead of eight numbers,
only the first few coefficients are significant.

MFCC in Ship Detection

Imagine
Ocean Noise

MFCC
12
2
1
0
0
0

Ship Present

MFCC
45

15
8
4
2
1

Machine learning algorithms can learn these numerical patterns.

Why CNNs Prefer Spectrograms

Modern deep learning models (CNNs, ResNet, EfficientNet) are very good at learning directly from images.

Because of this,

many current audio applications use

MeL Spectrogram

instead of

MFCC

A CNN can discover useful features on its own without needing the compressed representation.

Is MFCC Good for Passive Sonar?

This is an important consideration.

Advantages

- Compact feature vectors.
- Fast to compute.
- Lower storage requirements.
- Good for classical machine learning models such as SVM, Random Forest, and KNN.

Limitations

MFCC was originally developed for **speech**, where the overall spectral envelope is more important than precise tonal frequencies.

In passive sonar, narrowband tonal components from engines, propellers, and gearboxes are often critical. Because MFCC compresses the spectrum, some of these fine details can be lost.

For this reason, many sonar researchers prefer:

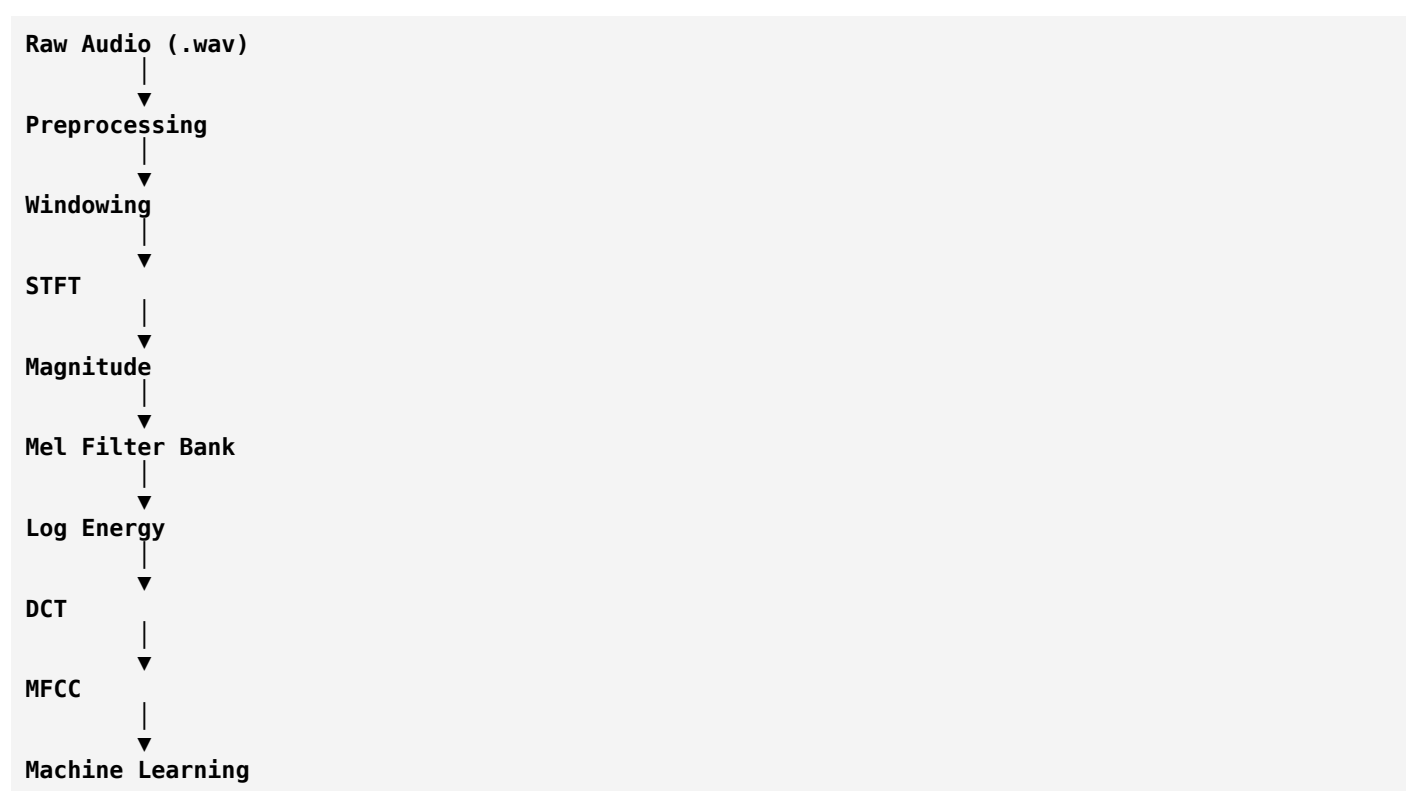
- Log Spectrograms.
- LOFARGRAMs.

- High-resolution STFT representations.

Comparison

Feature	Output	Good For	Information Retained
FFT	1D Spectrum	Frequency analysis	High
STFT	Time-Frequency Matrix	Non-stationary signals	High
Spectrogram	Image	CNNs	High
Mel Spectrogram	Compressed frequency image	Deep learning	Moderate to High
MFCC	Numerical coefficients	Classical ML	Moderate

Complete Audio Processing Pipeline



Which Features Should You Use?

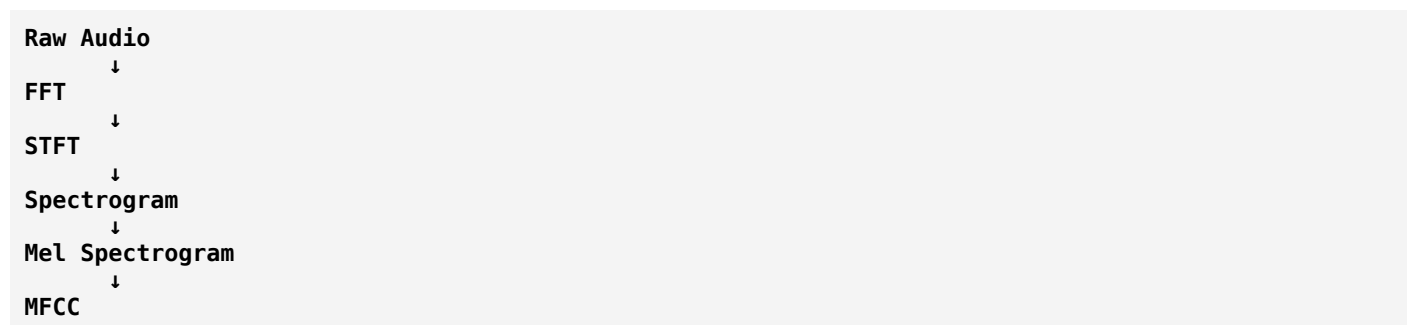
For your **Passive Sonar-Based Ship Presence Detection** project, here's a practical recommendation:

Feature	Recommendation	Reason
FFT	✗ Not as direct model input	Lacks time information
STFT	✓ Foundation	Needed to create later features
Log Spectrogram	★★★★★	Excellent baseline for CNNs

Feature	Recommendation	Reason
Mel Spectrogram	★★★★☆	Good balance of size and performance
MFCC	★★★★☆☆	Useful as a baseline with traditional ML models
LOFARGRAM	★★★★★★	Specifically designed for passive sonar and preserves tonal signatures

Where You Are Now

You now understand the complete feature extraction chain:



The **next topic should be LOFARGRAM**, because unlike MFCC—which was designed for speech—LOFARGRAM was developed specifically for analyzing underwater acoustic signals. It's one of the most important feature representations in passive sonar research and is highly relevant to your project.

LOFARGRAM

Excellent. **LOFARGRAM** is probably the **most important feature representation** in passive sonar. Unlike MFCC (developed for speech), LOFARGRAM was specifically developed for **underwater acoustic analysis**.

If your project is:

Passive Sonar-Based Target Ship Classification (Ship Present / Ship Absent)

then understanding LOFARGRAM is essential.

What is LOFAR?

LOFAR stands for:

LOw Frequency Analysis and **Re**cording.

Originally, LOFAR systems were developed by naval forces to detect and classify submarines and ships using passive sonar.

Why Was LOFAR Developed?

Imagine a hydrophone records:

**Ocean Noise
+
Ship Engine
+
Propeller
+
Pump
+
Gearbox
+
Marine Animals**

All these sounds are mixed together.

A normal waveform looks like this:

Amplitude

Can you identify the ship?

X No.

Apply FFT:

Frequency

100 Hz

200 Hz

300 Hz

500 Hz

Better.

You now know which frequencies exist.

But...

When did they occur?

FFT cannot answer.

Apply STFT:

Time

↓

Frequency

Now you know:

- Which frequency
- At which time

Good.

But sonar engineers observed something interesting.

Ships produce **very stable narrowband tones**.

Example:

Engine

100 Hz

Propeller

240 Hz

Gearbox

620 Hz

These frequencies remain visible for a long time.

LOFARGRAM was designed to emphasize these stable tonal components.

Is LOFARGRAM Different from STFT?

This is the biggest misconception.

Many students think:

STFT

↓

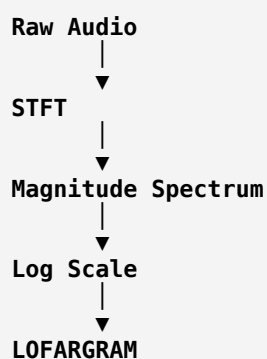
LOFARGRAM

are completely different.

Actually,

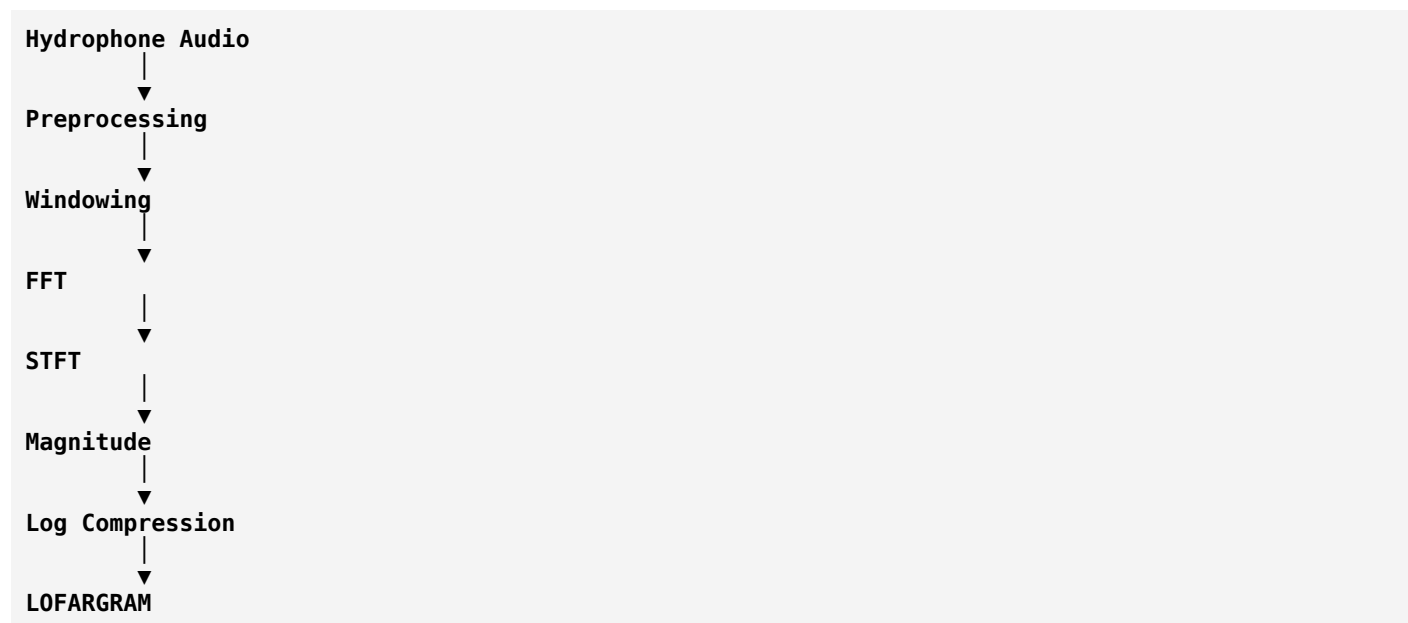
LOFARGRAM is **built using STFT**.

Think of it like this:



It is essentially a **log-scaled spectrogram** optimized for low-frequency passive sonar analysis.

LOFARGRAM Pipeline



Notice

Everything is identical to the spectrogram pipeline.

The difference lies in how the spectrum is displayed and interpreted.

Why "Low Frequency"?

Most ship machinery generates energy in lower frequencies.

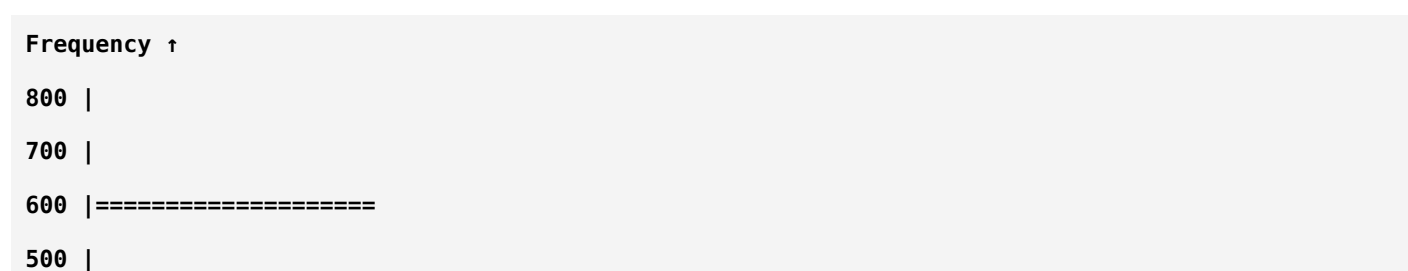
Typical examples:

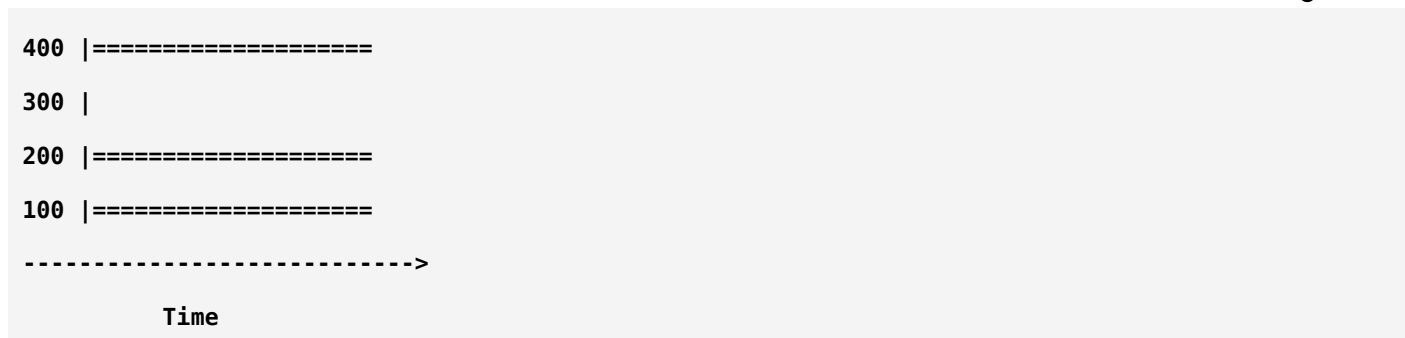
Source	Frequency Range
Engine	20–200 Hz
Propeller	50–500 Hz
Gearbox	100–1000 Hz
Pumps	200–1500 Hz

Because these low-frequency tones travel long distances underwater, LOFAR systems focus on them.

What Does a LOFARGRAM Look Like?

Imagine this image.





The horizontal bright lines represent stable frequencies.

Each line corresponds to a machine inside the ship.

Why Horizontal Lines?

Suppose the engine rotates at a constant speed.

It continuously produces

120 Hz

The frequency remains the same.

So in the LOFARGRAM,

120 Hz

appears as

A horizontal line.

Similarly,

Propeller

240 Hz

↓

Gearbox

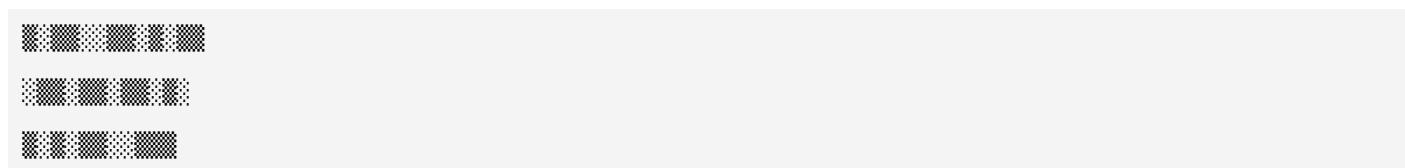
630 Hz

↓

Ocean Noise

Ocean noise is random.

Its LOFARGRAM looks like



There are no clear horizontal lines.

Ship Present



Notice

Stable horizontal bands.

These are called **tonal signatures**.

Why Are Tonal Signatures Important?

Every ship has

- Different engine
- Different propeller
- Different gearbox
- Different shaft speed

Therefore

every ship produces

a unique combination of tonal frequencies.

Think of it like a fingerprint.

Example

Ship A

100 Hz
250 Hz
620 Hz

Ship B

90 Hz
220 Hz
580 Hz

The CNN learns these patterns.

LOFARGRAM vs Spectrogram

A normal spectrogram displays all frequencies equally.

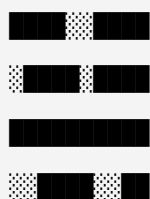
LOFARGRAM emphasizes:

- Low frequencies
- Narrowband tones
- Long-duration stable components
- Log-scaled intensity

making it more suitable for passive sonar.

Example Comparison

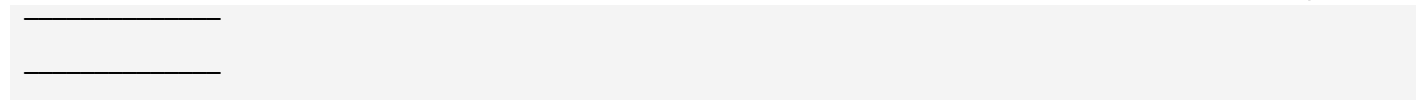
Spectrogram



Lots of texture.

LOFARGRAM





Very clean horizontal lines.

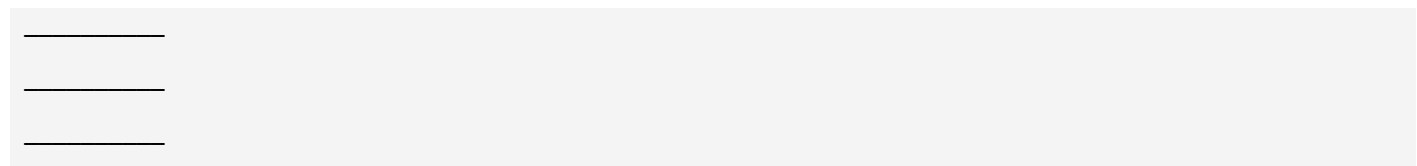
Those lines are exactly what sonar operators search for.

Why CNN Works So Well

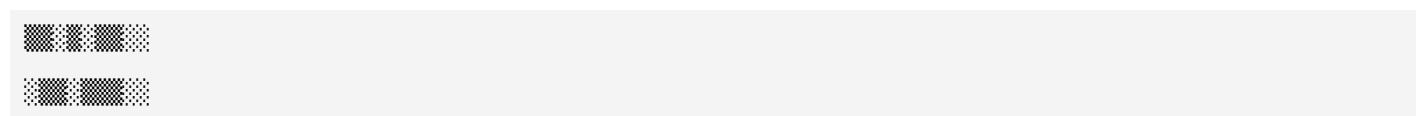
The CNN treats the LOFARGRAM like an image.

Example

Ship



No Ship



The CNN learns

- Horizontal patterns
- Bright lines
- Spacing between harmonics
- Energy distribution

without explicitly knowing they represent machinery.

Why LOFARGRAM is Better than MFCC for Sonar

MFCC	LOFARGRAM
Designed for speech	Designed for underwater acoustics
Compresses frequency information	Preserves narrowband tonal information
Loses some fine spectral details	Retains detailed tonal signatures
Good for speech recognition	Excellent for ship detection and classification

Where LOFARGRAM Fits

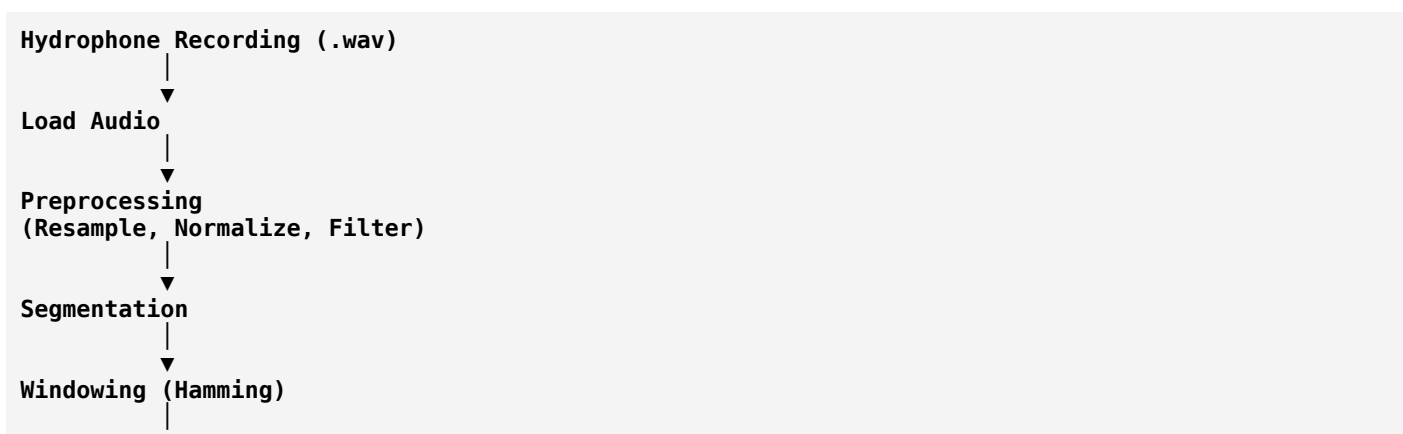


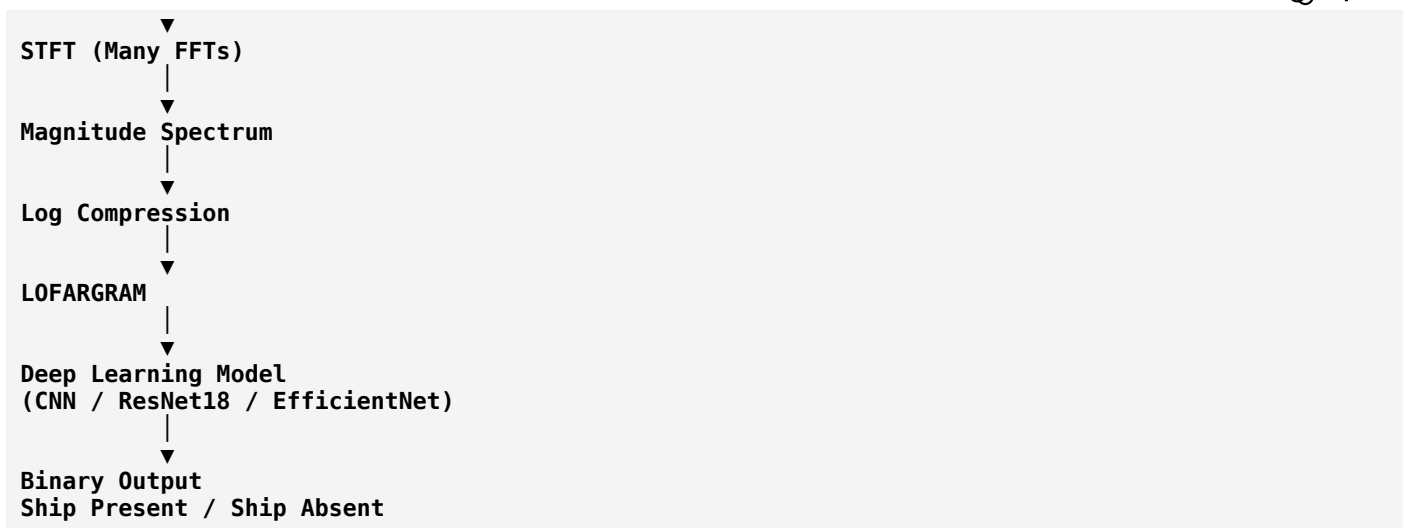
Which Feature Should You Choose?

For your project, here's how I'd prioritize them:

Feature	Suitability for Passive Sonar	Notes
FFT	★★☆☆☆	Good for understanding frequency content, but no time information.
STFT	★★★★☆	Essential intermediate step for all time-frequency analysis.
Spectrogram	★★★★☆	Strong baseline for CNN models.
Mel Spectrogram	★★★☆☆	Useful, but may smooth away important narrowband tones.
MFCC	★★☆☆☆	Better suited to speech than sonar.
LOFARGRAM	★★★★★★	Best traditional representation for highlighting ship tonal signatures.

Complete Passive Sonar Pipeline





One Important Clarification

In research papers, you may see slightly different definitions of a "LOFARGRAM." Some implementations include additional processing such as frequency averaging, background normalization, or tonal enhancement before visualization. However, the core idea remains the same:

- Start with **STFT**.
- Compute the **magnitude (or power) spectrum**.
- Apply **logarithmic scaling**.
- Emphasize stable **low-frequency tonal components** that characterize ships.

Understanding this foundation will make it much easier to read and implement different LOFARGRAM variants described in the passive sonar literature.

At this point, you've covered the complete digital audio feature extraction chain needed for your project. The next stage is deciding how to generate these features from your dataset in Python (using libraries such as `librosa` and `scipy`) and preparing them as inputs for your deep learning model.

Today 6:19 AM

No as of now I will not go into any implementation now I want to learn basics of machine learning that I have to learn to understand and complete my project mainly focuses on key terms and everything so that I can easily understand

That's the best approach. **Don't jump into coding yet.** First build a strong conceptual foundation. Once you understand the terminology, reading research papers and implementing models becomes much easier.

Since your project is:

Passive Sonar-Based Binary Ship Classification (Ship Present / Ship Absent)

you don't need to learn *all* of machine learning. You need a focused roadmap.

Machine Learning Roadmap for Your Project

1. Machine Learning Basics
 - ↓
2. Data & Features
 - ↓
3. Model Basics
 - ↓
4. Training Process
 - ↓
5. Optimization
 - ↓
6. Evaluation Metrics
 - ↓
7. Deep Learning Basics
 - ↓
8. CNN Fundamentals
 - ↓
9. Transfer Learning
 - ↓
10. Binary Classification

Phase 1: Machine Learning Basics

Understand these concepts first:

- What is Artificial Intelligence (AI)?
- What is Machine Learning (ML)?
- What is Deep Learning (DL)?
- AI vs ML vs DL
- Types of Machine Learning:
 - Supervised Learning
 - Unsupervised Learning
 - Reinforcement Learning
- What is a Dataset?
- What is a Feature?
- What is a Label?
- Training Set
- Validation Set
- Test Set

Phase 2: Data Fundamentals

These are essential for your sonar project:

- Sample
- Feature
- Label

- Feature Vector
 - Dataset
 - Class
 - Binary Classification
 - Multi-class Classification
 - Class Imbalance
 - Data Augmentation
 - Normalization
 - Standardization
-

Phase 3: Model Fundamentals

Learn what a model actually is:

- What is a Model?
 - Parameters
 - Hyperparameters
 - Input
 - Output
 - Prediction
 - Inference
 - Decision Boundary
 - Feature Space
-

Phase 4: Training Concepts

This is one of the most important topics.

Learn:

- Training
- Epoch
- Batch
- Batch Size
- Iteration
- Forward Propagation
- Loss Function
- Cost Function

- Gradient
- Gradient Descent
- Backpropagation
- Optimizer
- Learning Rate

These concepts explain **how a model learns**.

Phase 5: Neural Network Basics

Understand:

- Artificial Neuron
 - Perceptron
 - Activation Function
 - ReLU
 - Sigmoid
 - Tanh
 - Hidden Layer
 - Input Layer
 - Output Layer
 - Weight
 - Bias
-

Phase 6: Deep Learning

Learn:

- Neural Network
- Deep Neural Network (DNN)
- CNN
- RNN
- LSTM
- Attention (basic idea)

For your project, **CNN is the most important**.

Phase 7: CNN Concepts

These are directly related to spectrogram and LOFARGRAM images:

- Image Representation
 - Pixel
 - Channel
 - Convolution
 - Kernel / Filter
 - Feature Map
 - Padding
 - Stride
 - Pooling
 - Max Pooling
 - Average Pooling
 - Flatten
 - Fully Connected Layer
 - Softmax
 - Sigmoid
-

Phase 8: Transfer Learning

Since you may use models like **ResNet18** or **EfficientNet**, understand:

- Pre-trained Model
 - Fine-tuning
 - Feature Extraction
 - Frozen Layers
 - Trainable Layers
-

Phase 9: Evaluation Metrics

For binary classification, learn:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1 Score
- Specificity

- Sensitivity
- ROC Curve
- AUC
- Precision-Recall Curve

These tell you **how well your model performs**.

Phase 10: Overfitting & Generalization

Very important concepts:

- Underfitting
 - Overfitting
 - Generalization
 - Bias
 - Variance
 - Regularization
 - Dropout
 - Early Stopping
 - Cross Validation
-

Phase 11: Optimization

Understand:

- SGD
 - Momentum
 - RMSProp
 - Adam
 - AdamW
 - Learning Rate Scheduler
-

Phase 12: Loss Functions

For your project:

- Binary Cross Entropy ★★★★★
 - Categorical Cross Entropy
 - Mean Squared Error (know why it's not commonly used for binary classification)
-

Phase 13: Practical Concepts

Know these terms because you'll see them everywhere:

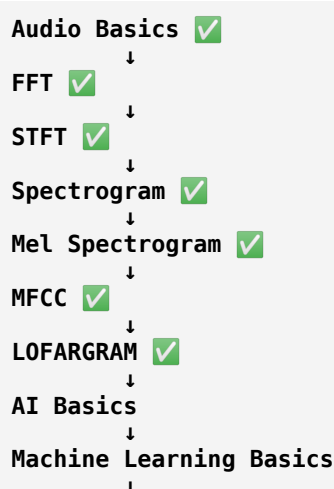
- Tensor
- Tensor Shape
- NumPy Array
- GPU
- CUDA
- TensorFlow
- PyTorch
- Model Checkpoint
- Inference
- Deployment

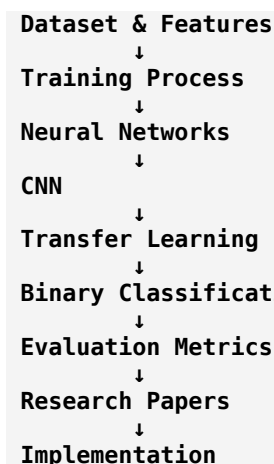
Phase 14: Research Concepts

When reading papers, you'll encounter:

- Baseline Model
- Proposed Method
- Ablation Study
- State-of-the-Art (SOTA)
- Benchmark Dataset
- Precision-Recall Trade-off
- Statistical Significance

Learning Order (Recommended)





How We'll Learn

Instead of learning random theory, we'll connect **every concept to your sonar project**.

For example:

- **Feature** → A spectrogram or LOFARGRAM extracted from a sonar clip.
- **Label** → "Ship Present" or "Ship Absent."
- **Epoch** → One complete pass through all sonar training samples.
- **CNN** → Learns patterns such as horizontal tonal lines in LOFARGRAMs.
- **Loss Function** → Measures how wrong the model's predictions are.
- **Precision** → Of all clips predicted as "Ship Present," how many actually contain a ship?

This way, every concept has a direct connection to your project rather than being an abstract definition.

My recommendation for the next topic

Start with "**What is Machine Learning?**" Then continue in this order:

1. AI vs Machine Learning vs Deep Learning
2. Supervised Learning (most important for your project)
3. Dataset, Sample, Feature, and Label
4. Binary Classification

Once these four topics are clear, the rest of machine learning becomes much easier because you'll already understand the overall learning process.

Ok the path you given exactly I need ok continue with it start phase 1

Perfect! We'll learn this like a university course, but **everything will be related to your Passive Sonar Ship Detection project**. We won't memorize definitions—we'll understand why each concept matters.

Phase 1: Machine Learning Basics

Topic 1: What is Artificial Intelligence (AI)?

Definition

Artificial Intelligence (AI) is the broad field of creating machines that can perform tasks that normally require human intelligence.

These tasks include:

- Recognizing images
- Understanding speech
- Making decisions
- Playing games
- Driving vehicles
- Detecting ships from sonar signals

Think of AI as the **big umbrella**.

Example in Your Project

Imagine a naval officer listens to a sonar recording.

They may say:

"I hear a ship."

That decision is based on years of experience.

If we build a system that can make the same decision automatically:

```

Sonar Audio
  ↓
AI System
  ↓
Ship Present
    
```

That system is an **Artificial Intelligence** system.

Topic 2: What is Machine Learning (ML)?

Machine Learning is a **subset of AI**.

Instead of writing explicit rules, we let the computer **learn patterns from data**.

Traditional Programming

Rules + Data



Program



Answer

Example:

If temperature > 38°C, print "Fever."

You write the rule.

Machine Learning

Data + Correct Answers



Learning Algorithm



Model

The computer discovers the rules by itself.

Sonar Example

Suppose you collect:

Sonar Recording	Label
Audio 1	Ship Present
Audio 2	No Ship
Audio 3	Ship Present
Audio 4	No Ship

Instead of telling the computer:

"Look for 120 Hz and 250 Hz."

You simply provide many examples.

The model learns the patterns automatically.

Why is it Called "Learning"?

Imagine teaching a child to recognize dogs.

You don't explain every rule.

You show many examples.

Dog 
 Dog 
 Dog 

Cat 🐱
Dog 🐶

Eventually, the child learns the concept of "dog."

Machine learning works similarly.

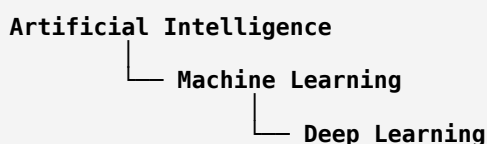
Instead of children, we train a model.

Instead of dogs, we use sonar recordings.

Topic 3: What is Deep Learning (DL)?

Deep Learning is a **subset of Machine Learning**.

It uses **Artificial Neural Networks** with many layers to automatically learn complex patterns.

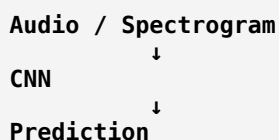


Traditional ML vs Deep Learning

Traditional ML:



Deep Learning:



The neural network learns many useful features automatically.

In Your Project

Input:

LOFARGRAM Image

↓

CNN

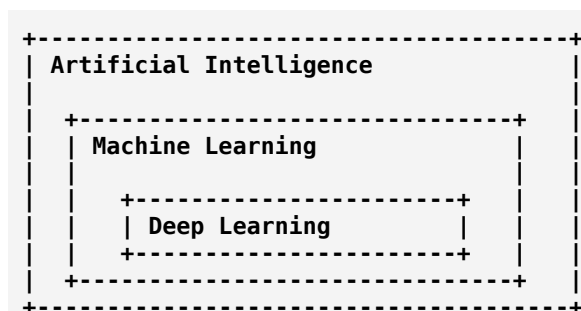
↓

Output:

Ship PresentThis is **Deep Learning**.

Topic 4: AI vs ML vs DL

Imagine three circles:



Artificial Intelligence

Goal: Build intelligent systems.

Examples:

- Chatbots
- Self-driving cars
- Sonar target detection

Machine Learning

Method: Learn from data.

Examples:

- Spam detection
- Disease prediction
- Ship detection

Deep Learning

Method: Learn using deep neural networks.

Examples:

- Image recognition
- Speech recognition

- Sonar image classification

Topic 5: Types of Machine Learning

There are three major types.

1. Supervised Learning ★★★★★

The model learns from **inputs and correct answers (labels)**.

Example:

Input	Label
Sonar Clip A	Ship Present
Sonar Clip B	No Ship
Sonar Clip C	Ship Present

The model learns the relationship between the input and the label.

Your project belongs to this category.

2. Unsupervised Learning

Here, you only provide the data.

No labels.

Example:

1000 sonar recordings

The algorithm groups similar recordings together.

It might discover:

- Group A: Similar engine sounds
- Group B: Similar background noise
- Group C: Marine life

No one tells it what these groups mean.

3. Reinforcement Learning

The model learns by interacting with an environment and receiving rewards or penalties.

Example:

An underwater robot:

Move Left

Reward: +10

Move Right

Penalty: -5

Over time, it learns the best actions.

This is **not used** in your project.

Which Type Does Your Project Use?

```

Sonar Recording
  ↓
Feature (LOFARGRAM)
  ↓
Label
(Ship Present / Ship Absent)
  ↓
Train CNN
  
```

Since you have known labels, it is **Supervised Learning**.

Phase 1 Summary

Concept	Meaning	Example in Your Project
AI	Making machines perform intelligent tasks	Automatic ship detection
ML	Learning patterns from data	Learning from labeled sonar recordings
DL	ML using deep neural networks	CNN classifying LOFARGRAMs
Supervised Learning	Data + labels	Ship / No Ship dataset
Unsupervised Learning	Data only	Grouping similar underwater sounds
Reinforcement Learning	Learning by rewards	Not used for your project

Real-World Analogy

Imagine you're training a new sonar operator.

You give them 10,000 recordings.

For each recording, you also provide the answer:

```

Recording 1 → Ship
Recording 2 → No Ship
Recording 3 → Ship
  
```


Recording 4 → No Ship

After enough practice, they begin recognizing ships on their own.

A machine learning model learns in exactly the same way—except instead of ears and experience, it uses mathematical algorithms and data.

What We'll Learn Next (Still in Phase 1)

The next lesson is **Dataset Fundamentals**, where we'll cover:

- What is a Dataset?
- What is a Sample?
- What is a Feature?
- What is a Label?
- What is a Feature Vector?
- What are Classes?
- Why are these concepts the foundation of every machine learning project?

These are the most frequently used terms in ML papers, tutorials, and implementations, so understanding them well will make everything that follows much easier.

Ok next

Excellent! This is one of the **most important topics in Machine Learning**. If you understand these concepts well, then almost every research paper and ML tutorial will make sense.

Phase 1 (Part 2): Dataset Fundamentals

We'll learn these in order:

1. Dataset
2. Data Point / Sample
3. Feature
4. Label (Target)
5. Feature Vector
6. Class
7. Binary Classification
8. Multi-class Classification

We'll use **your sonar project** as the example throughout.

1. What is a Dataset?

A **dataset** is simply a collection of data used to train or test a machine learning model.

Think of it as a **textbook** from which the model learns.

Example

Suppose you collect 10,000 sonar recordings.

```
Dataset
Recording 1
Recording 2
Recording 3
...
Recording 10000
```

This entire collection is called the **dataset**.

Sonar Example

Recording	Label
audio001.wav	Ship Present
audio002.wav	No Ship
audio003.wav	Ship Present
audio004.wav	No Ship

This table is your dataset.

2. What is a Sample (Data Point)?

A **sample** is **one individual example** from the dataset.

If your dataset has:

```
10,000 sonar recordings
```

Then

```
1 recording = 1 sample
```

Example:

```
audio003.wav
```

This single recording is one sample.

Analogy

Imagine a classroom.

There are **60 students**.

The classroom is the **dataset**.

Each student is one **sample**.

In Your Project

One sample could be:

3-second sonar clip

or

One LOFARGRAM image

Depending on your pipeline.

3. What is a Feature?

This is one of the most important concepts.

A **feature** is a measurable property or characteristic of a sample that helps the model make a prediction.

Think of it as a **clue**.

Human Example

Suppose you want to identify a person.

Useful features include:

- Height
- Weight
- Eye color
- Hair color
- Age

These are all features.

Sonar Example

Suppose your sample is a sonar recording.

Possible features are:

- FFT values
- Spectrogram
- Mel Spectrogram
- MFCC
- LOFARGRAM
- Energy
- Frequency peaks

These features contain information that helps distinguish:

- Ship Present
- Ship Absent

Important Point

The **raw audio** itself is usually **not** the feature.

We process the audio to extract informative features.

For example:

```
Raw Audio
  ↓
STFT
  ↓
LOFARGRAM
```

The **LOFARGRAM** becomes the feature given to the model.

4. What is a Label (Target)?

A **label** is the correct answer that the model should learn to predict.

Example:

Sample	Label
audio001.wav	Ship Present
audio002.wav	No Ship

The label tells the model whether its prediction is correct.

In Your Project

Sample:

LOFARGRAM Image

Label:

Ship Present

or

Ship Absent

Without labels, the model doesn't know the correct answer during supervised training.

5. What is a Feature Vector?

A **feature vector** is the numerical representation of all features for one sample.

Machine learning models work with numbers, not concepts like "ship" or "noise."

Example

Suppose a sample has these extracted features:

Feature	Value
Energy	0.85
Peak Frequency	120 Hz
Spectral Centroid	340 Hz
Bandwidth	180 Hz

The feature vector is:

[0.85, 120, 340, 180]

This list of numbers is what the model actually receives.

In Deep Learning

If you use a **LOFARGRAM**, the feature vector is no longer a simple list.

Instead, it is a **matrix of pixel values**.

Example:

128 × 128

The CNN processes this image directly.

6. What is a Class?

A **class** is one possible category that a sample can belong to.

In Your Project

There are two classes:

Class 0

No Ship

Class 1

Ship Present

Every sample belongs to exactly one class.

7. What is Binary Classification?

Binary means **two**.

A binary classification problem has exactly two classes.

Examples:

Problem	Classes
Spam Detection	Spam / Not Spam
Medical Diagnosis	Disease / Healthy
Face Unlock	Authorized / Unauthorized
Your Project	Ship Present / Ship Absent

Your project is a **binary classification** problem.

8. What is Multi-class Classification?

Here, there are more than two classes.

Example:

Instead of just detecting a ship, suppose you want to identify its type:

Sample	Label
Audio 1	Cargo Ship
Audio 2	Fishing Boat
Audio 3	Submarine
Audio 4	Oil Tanker

Now the model chooses one of several categories.

That is **multi-class classification**.

Putting Everything Together

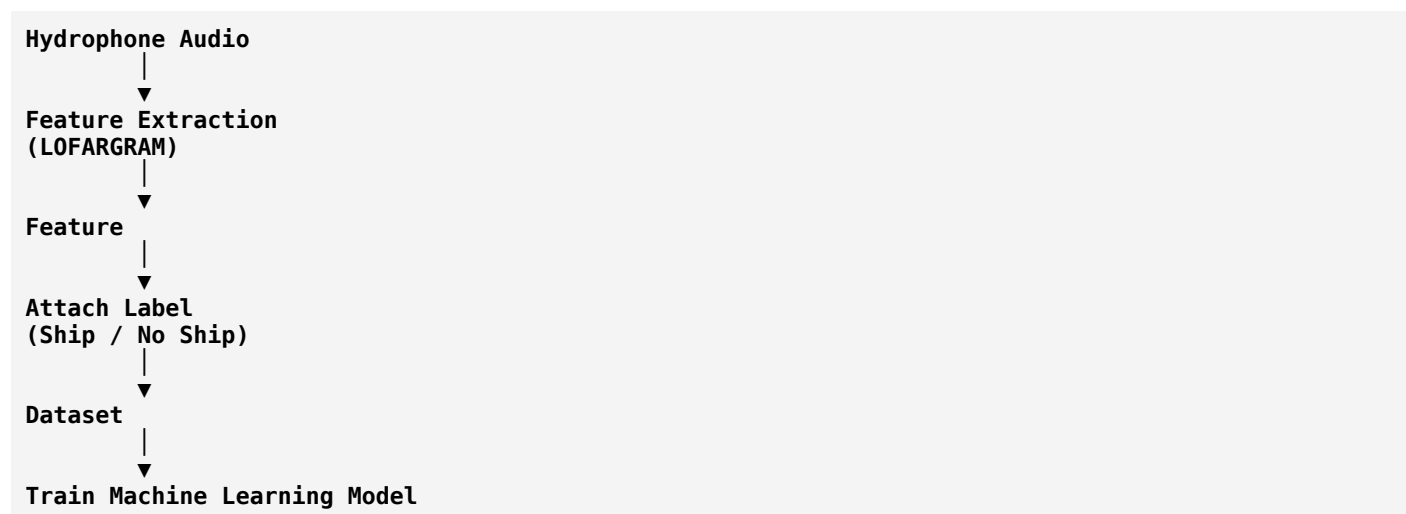
Imagine this dataset:

Sample	Feature	Label
audio001.wav	LOFARGRAM	Ship Present
audio002.wav	LOFARGRAM	No Ship
audio003.wav	LOFARGRAM	Ship Present

Here:

- The whole table is the **dataset**.
- Each row is one **sample**.
- The LOFARGRAM is the **feature**.
- "Ship Present" or "No Ship" is the **label**.
- There are **2 classes**, making it a **binary classification** problem.

Visual Flow



Summary Table

Term	Meaning	Example in Your Project
Dataset	Collection of all data	All sonar recordings
Sample	One data item	One sonar clip
Feature	Information used for learning	LOFARGRAM, Spectrogram, MFCC
Label	Correct answer	Ship Present / Ship Absent
Feature Vector	Numerical representation of features	Pixel values of a LOFARGRAM or extracted numeric features

Term	Meaning	Example in Your Project
Class	Possible output category	Ship, No Ship
Binary Classification	Two output classes	Ship Present vs Ship Absent
Multi-class Classification	More than two output classes	Cargo Ship, Fishing Boat, Submarine, etc.

Quiz (Try Without Looking)

1. In your project, what is the **dataset**?
2. What is **one sample**?
3. What is the **feature** if you use LOFARGRAM?
4. What is the **label**?
5. Why is your project called **binary classification**?

If you can answer these confidently, you've understood one of the most fundamental parts of machine learning.

Next Lesson

Next we'll cover another crucial topic:

Training Set, Validation Set, and Test Set

You'll learn:

- Why we don't train on all the data.
- Why we split the dataset.
- What **data leakage** is.
- How models are evaluated fairly.

This topic is essential because every machine learning project—including yours—depends on splitting data correctly before training.

Next

Excellent! This is one of the **most important concepts in Machine Learning**.

Many beginners think:

"I have 10,000 sonar recordings. I'll train the model on all of them."

This seems logical, but it's actually **a mistake**.

Let's understand why.

Phase 1 (Part 3): Training Set, Validation Set, and Test Set

Why Can't We Train on All the Data?

Imagine you are preparing for an exam.

Your teacher gives you **100 questions**.

You memorize every answer.

During the exam, the teacher asks **the exact same 100 questions**.

You score **100/100**.

Does that mean you truly understand the subject?

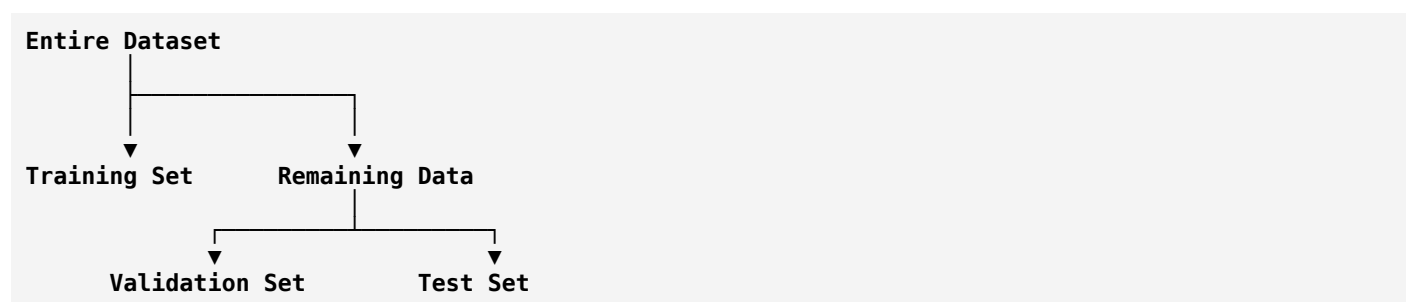
No.

It only means you memorized the questions.

Machine learning models can do the same thing—they can **memorize** the training data instead of learning general patterns.

The Solution

We divide the dataset into three parts:



Each part has a different purpose.

1. Training Set

The **training set** is the data used to **teach the model**.

Example:

Suppose you have

10,000 sonar recordings

You might use

8,000 recordings

for training.

The model repeatedly looks at these samples and learns patterns.

Example:

```

LOFARGRAM
  ↓
CNN
  ↓
Prediction
  ↓
Compare with Label
  ↓
Improve

```

This learning process happens only on the training set.

2. Validation Set

Now suppose the model has learned from the training data.

How do we know if it is improving?

We test it on **new data that it has never seen during training**.

This is the **validation set**.

Example:

```
1,000 sonar recordings
```

The model predicts:

```
Ship Present
```

Actual label:

```
Ship Present
```

Good!

If the predictions improve over time, training is progressing well.

Why Validation?

The validation set helps us answer questions like:

- Should we train for more epochs?
- Should we stop training?
- Which model performs better?
- Which learning rate is best?

It guides decisions **during development**.

3. Test Set

The **test set** is the final exam.

The model has **never seen** these samples during:

- Training
- Validation
- Hyperparameter tuning

Example:

1,000 completely unseen recordings

Now we evaluate the model.

This tells us how well it is likely to perform in the real world.

Visual Example

Suppose your dataset contains **100 sonar recordings**.

Split it like this:

100 Recordings

- 70 → Training
- 15 → Validation
- 15 → Test

Typical real-world splits are:

- **70% / 15% / 15%**
- **80% / 10% / 10%**

The exact percentages are less important than keeping the sets separate.

Real-Life Analogy

Imagine preparing for a driving license.

Training

You practice driving every day.

↓

Training Set

Validation

Your instructor gives you practice tests.



Validation Set

Test

Government driving test.



Test Set

You don't practice during the real exam.

Your Sonar Project Example

Suppose you have

12,000 sonar clips

Split them as:

Training

9,600

Validation

1,200

Test

1,200

Now:

Training



Model learns

Validation



Model tuning

Test



Final performance

What is Data Leakage?

This is a very important concept.

Imagine:

```
Training
audio001.wav
```

and

```
Test
audio001.wav
```

The same recording appears in both sets.

The model already knows the answer!

It will appear to perform extremely well, but this performance is misleading.

This problem is called **data leakage**.

Rule

Never allow information from the training set to leak into the validation or test sets.

Another Example

Suppose you split one long recording into many overlapping clips.

If clips from the same original recording appear in both training and testing, the model may recognize recording-specific characteristics instead of learning true ship signatures.

This is another form of data leakage and is especially important in audio projects.

Why Keep the Test Set Hidden?

Suppose after testing you get:

```
Accuracy
80%
```

You modify the model.

Test again.

Now:

```
Accuracy
82%
```

Modify again.

Eventually, you may unintentionally optimize specifically for the test set.

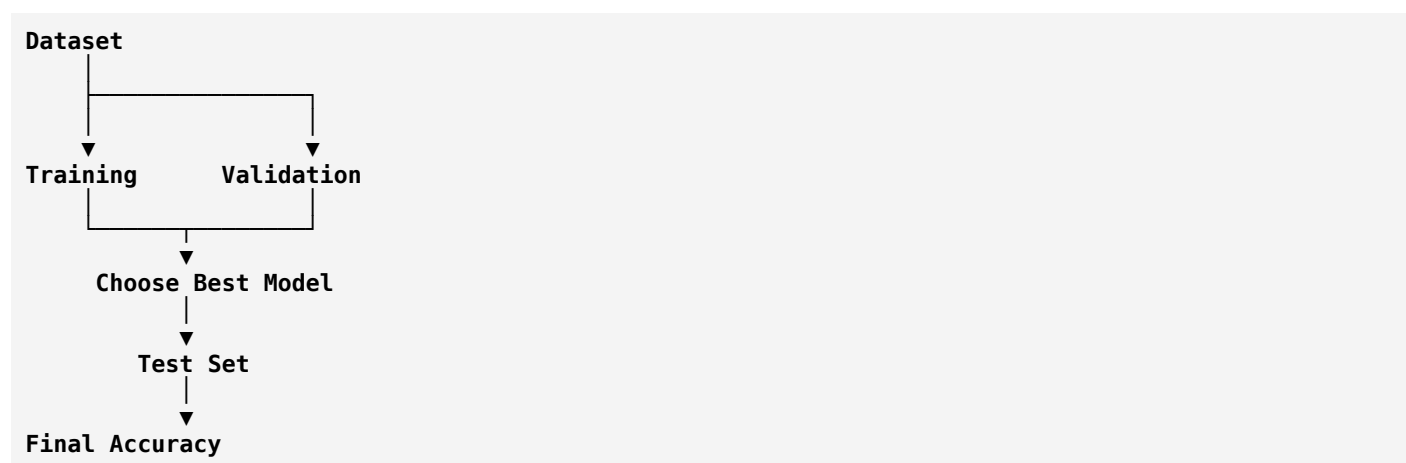
Then the test set is no longer an unbiased evaluation.

That's why many researchers evaluate on the test set **only once**, after all development is complete.

Summary Table

Dataset Split	Purpose	Used for Learning?
Training Set	Teach the model	✓ Yes
Validation Set	Tune the model and monitor training	✗ No
Test Set	Final evaluation	✗ No

Complete Workflow



In Your Passive Sonar Project

Imagine your dataset contains **LOFARGRAM** images:

- **Training Set:** Used to teach the CNN what "Ship Present" and "Ship Absent" look like.
- **Validation Set:** Used during training to decide if the model is improving or starting to overfit.
- **Test Set:** Used only at the end to estimate how well the model will classify completely new sonar recordings.

Key Terms to Remember

- **Training Set** → Used to learn.
- **Validation Set** → Used to tune and monitor.
- **Test Set** → Used for the final evaluation.
- **Data Leakage** → Any information from the test/validation data reaching the training process.

Next Lesson

Next we'll begin **Phase 2: Model Fundamentals**, starting with:

1. **What is a Machine Learning Model?**
2. **What are Parameters?**
3. **What are Hyperparameters?**
4. **What is Prediction (Inference)?**

These concepts explain what a model actually is and how it turns a LOFARGRAM into the prediction **"Ship Present"** or **"Ship Absent."** This is where you'll start understanding how learning actually happens inside the model.

Next

Excellent! Now we reach the heart of Machine Learning.

Up to now you know:

- ☒ What AI is
- ☒ What Machine Learning is
- ☒ What Deep Learning is
- ☒ Dataset
- ☒ Sample
- ☒ Feature
- ☒ Label
- ☒ Training/Validation/Test sets

Now the biggest question is:

What exactly is a Machine Learning Model?

This is the foundation for everything that follows.

Phase 2: Model Fundamentals

Topic 1: What is a Machine Learning Model?

Definition

A **Machine Learning Model** is a mathematical function that learns patterns from data and uses those patterns to make predictions on new data.

Think of it as a **decision-maker**.

Example

Imagine you train a new naval officer.

After listening to thousands of sonar recordings, they can identify whether a ship is present.

Now imagine replacing that officer with a computer.

The trained computer program is called the **model**.

```
Sonar Recording
  ↓
Trained Model
  ↓
Ship Present
```

The model is the part that has "learned."

Before Training

Imagine a student on the first day of class.

You play a sonar recording.

Student:

"I don't know."

Similarly, before training:

```
Input
  ↓
Model
  ↓
Random Guess
```

The model has not learned anything yet.

After Training

Now the student has listened to 10,000 labeled recordings.

You play a new recording.

Student:

"This sounds like a ship."

Similarly:

```
Input
  ↓
Trained Model
  ↓
Ship Present
```


The model has learned patterns from experience.

What Does the Model Actually Learn?

It **does not memorize** recordings.

Instead, it learns relationships like:

- Certain frequency patterns
- Certain tonal signatures
- Energy distributions
- Shapes in the LOFARGRAM

For example:

Horizontal bright lines
↓
Likely Ship

These learned relationships are stored inside the model.

Topic 2: What are Parameters?

This is one of the most important concepts.

Definition

Parameters are the internal values that the model learns automatically during training.

They represent the model's knowledge.

Analogy

Imagine you're adjusting a radio.

You turn the tuning knob until the signal becomes clear.

The knob position is like a **parameter**.

The model continuously adjusts millions of such values while learning.

In a Neural Network

Every connection has a **weight**.

Example:

Input
|
(weight = 0.8)
|
Neuron

Initially:

Weight = 0.23

After learning:

Weight = 1.87

These learned weights are the **parameters**.

Example

Suppose the model sees:

LOFARGRAM

Initially it predicts:

No Ship

Wrong!

The model adjusts its parameters.

Next time it predicts:

Ship Present

Better.

This adjustment happens thousands of times until the parameters represent useful knowledge.

Topic 3: What are Hyperparameters?

This is where many beginners get confused.

Let's compare.

Parameters

- Learned automatically.
- Change during training.

Example:

Weight = 0.83

The model updates this by itself.

Hyperparameters

Hyperparameters are **chosen by you before training**.

The model does **not** learn them.

Examples:

- Learning Rate
- Batch Size
- Number of Epochs
- Optimizer
- Number of Layers

These are settings that control how the model learns.

Analogy

Imagine cooking rice.

Parameters:

- The actual taste of the rice after cooking.

Hyperparameters:

- Cooking time
- Water quantity
- Flame level

You choose these before cooking.

Parameter vs Hyperparameter

Parameter	Hyperparameter
Learned automatically	Set by the developer
Changes during training	Usually fixed before training
Example: Weights, Biases	Example: Learning Rate, Batch Size

Topic 4: What is Prediction?

Once a model has been trained, we can use it to make predictions.

Example:

```
New Sonar Recording
↓
Model
↓
Ship Present
```

The output is called a **prediction**.

Topic 5: What is Inference?

Inference is simply the process of using a trained model to make predictions.

Training:

```
Model learns
```

Inference:

```
Model uses what it learned
```

Example

During training:

```
10,000 recordings
```

↓

Model learns.

Later, the navy receives a new sonar recording.

```
New Recording
↓
Model
↓
Prediction
```

This prediction process is called **inference**.

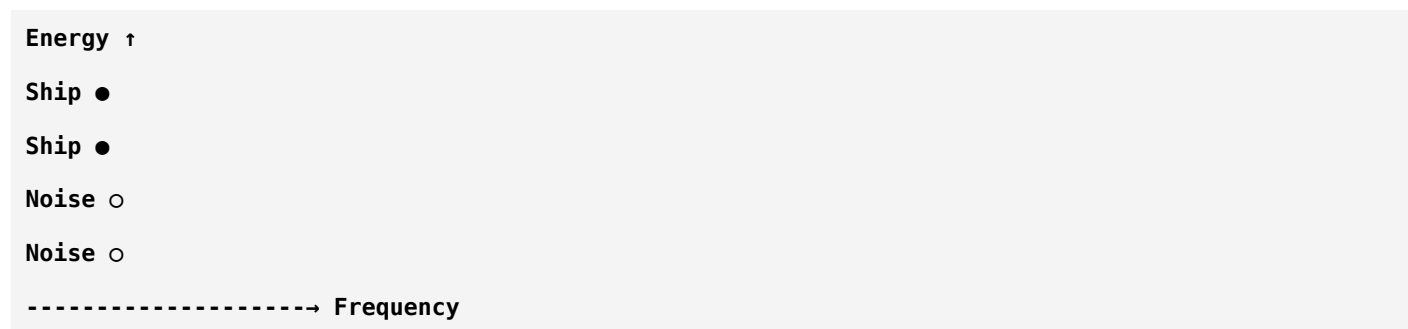
Topic 6: Feature Space (Basic Idea)

Imagine each sonar recording has two features:

- Engine Frequency

- Signal Energy

We can plot each recording as a point.

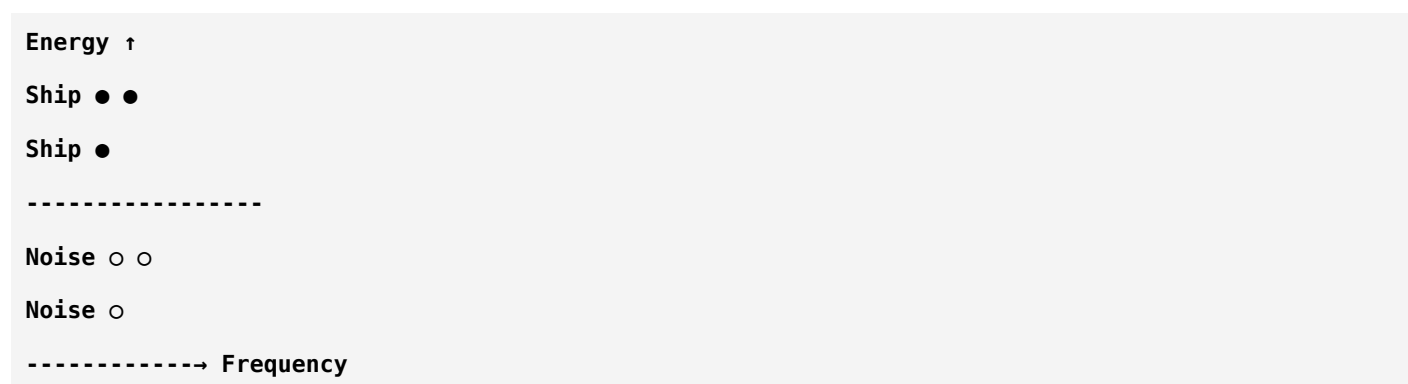


This graph is called the **feature space**.

The model tries to separate the two groups.

Topic 7: Decision Boundary

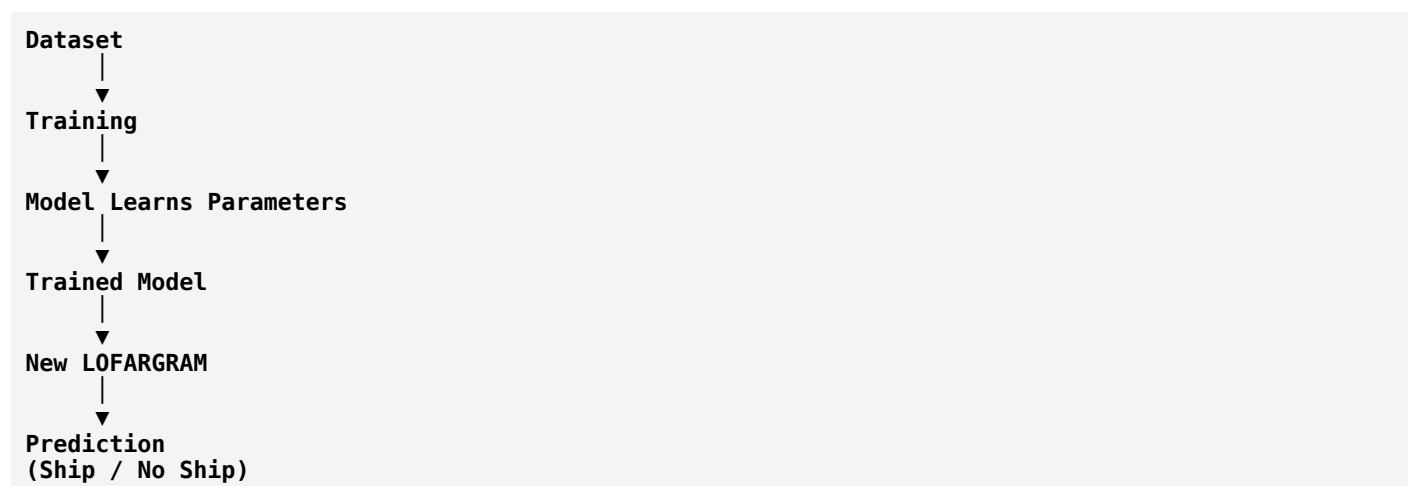
Imagine drawing a line that separates ships from noise.



That separating line is called the **decision boundary**.

A good model learns a decision boundary that correctly separates the classes.

Complete Flow



Summary

Concept	Meaning	Example in Your Project
Model	Learns patterns from data	CNN trained on LOFARGRAMs
Parameter	Values learned during training	Weights and biases
Hyperparameter	Settings chosen before training	Learning rate, batch size
Prediction	Model's output	Ship Present
Inference	Using the trained model	Classifying a new sonar clip
Feature Space	Space where samples are represented	LOFARGRAM features
Decision Boundary	Separation between classes	Ship vs No Ship

One Important Clarification

Many people think:

Model = Algorithm

They are related but **not the same**.

- **Algorithm** → The learning procedure (e.g., how a CNN updates its weights during training).
- **Model** → The trained result after learning.

For example:

- Before training: an untrained CNN architecture.
- After training: a CNN model that has learned to detect ships.

Next Topic (Very Important)

Next we'll learn **how a model actually learns**, step by step.

We'll cover:

1. **Epoch**
2. **Batch**
3. **Batch Size**
4. **Iteration**
5. **Forward Propagation**
6. **Loss Function**

These concepts describe the complete training process and are essential before moving on to gradient descent, backpropagation, and neural networks. This is where machine learning starts becoming truly intuitive.

Next

Excellent! This is the **most important phase** in Machine Learning.

After this lesson, you'll understand **how a model actually learns**.

This is the concept that every ML engineer, researcher, and deep learning practitioner must understand.

Phase 3: How Does a Model Learn?

Imagine you're teaching a student to identify ships from sonar recordings.

The learning process is:

```

See Example
  ↓
Guess
  ↓
Check Answer
  ↓
Correct Mistake
  ↓
Learn
  ↓
Repeat Thousands of Times
  
```

A machine learning model learns in exactly the same way.

Complete Training Cycle

```

Dataset
  ↓
Take One Batch
  ↓
Model Makes Prediction
  ↓
Calculate Error (Loss)
  ↓
Improve Parameters
  ↓
Next Batch
  ↓
Repeat
  
```

Every concept we'll learn fits into this cycle.

1. What is an Epoch?

Definition

An **Epoch** is **one complete pass through the entire training dataset**.

Example

Suppose your training dataset contains:

1000 sonar recordings

If the model has seen all 1000 recordings once,
that is:

1 Epoch

Example

Training Data

Recording 1
Recording 2
...
Recording 1000

After seeing all of them

↓

Epoch 1 Complete ✓

If it sees them again

↓

Epoch 2 Complete ✓

Again

↓

Epoch 3 Complete ✓

Why Multiple Epochs?

A student doesn't learn after reading a book once.

They revise.

Similarly,

the model needs multiple passes.

Example

Epoch 1
Accuracy = 55%

Epoch 5
Accuracy = 80%

Epoch 20
Accuracy = 95%

The model gradually improves.

2. What is a Batch?

Imagine you have

10,000 sonar recordings

Should the computer process all 10,000 at once?

Usually **no**, because it would require too much memory.

Instead, we divide the dataset into smaller groups.

Each group is called a **batch**.

Example

Training Data

1000 Samples

Divide into

100
100
100
100
100
100
100
100
100
100
100
100

Each group of 100 samples is a **batch**.

3. What is Batch Size?

The number of samples in one batch is called the **Batch Size**.

Example

Batch Size = 32

means

the model learns from **32 samples at a time**.

Common Batch Sizes

16
32
64
128
256

There is no universally "best" batch size. It depends on the dataset, model, and available GPU memory.

Example

Suppose

Training Samples
1000

Batch Size

100

Then

$1000 / 100 = 10$ batches

The model processes

Batch 1

↓

Batch 2

↓

Batch 3

↓

...

↓

Batch 10

Then

Epoch 1 finishes.

4. What is an Iteration?

Many beginners confuse **Epoch** and **Iteration**.

Let's clarify.

Suppose

Training Samples

1000

Batch Size

100

Therefore

10 batches

Each time the model processes **one batch**, it completes **one iteration**.

So

10 Iterations
=
1 Epoch

Formula

Iterations
=
Training Samples
÷
Batch Size

Example

Training Samples = 2000
Batch Size = 100
Iterations = 20

If you train for

30 Epochs

Total iterations

20 × 30
=
600

600

Visual Example

Training Data

□□□□□□□□

Batch 1

■

Iteration 1

↓

Batch 2

■

Iteration 2

↓

...

↓

Batch 10

■

Iteration 10

↓

Epoch Finished

Sonar Project Example

Suppose

Training Images

8000 LOFARGRAMS

Batch Size

32

Iterations

8000 / 32

=

250

Therefore

250 Iterations

=

1 Epoch

Train for

20 Epochs

↓

Total iterations

5000

Summary Table

Term	Meaning
Epoch	One complete pass through the training dataset
Batch	A small group of samples processed together
Batch Size	Number of samples in one batch
Iteration	One update of the model after processing a batch

Real-Life Analogy

Imagine a teacher has **300 answer sheets**.

Instead of checking all 300 at once,
they check **30 papers at a time**.

- **30 papers** → Batch
- **Batch Size = 30**
- After checking one batch → **1 Iteration**
- After checking all 300 papers → **1 Epoch**

If the teacher reviews all papers **5 times**, that's **5 Epochs**.

Complete Learning Flow

8000 LOFARGRAM Images

↓
Split into Batches

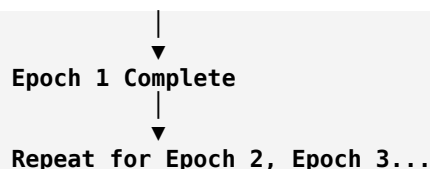
↓
Batch 1 → Iteration 1

Batch 2 → Iteration 2

Batch 3 → Iteration 3

...

Batch 250 → Iteration 250



Quiz (Answer Without Looking)

1. If you have **1,000 training samples** and a **batch size of 100**, how many **iterations** are in one epoch?
2. What is the difference between a **batch** and a **batch size**?
3. If you train for **10 epochs**, how many times has the model seen each training sample?

If you answer:

1. **10 iterations**
2. **A batch is the group; batch size is the number of samples in that group**
3. **10 times**

then you've understood this topic correctly.

Next Lesson (The Heart of Deep Learning)

Now that you know **how data is fed into the model**, the next lesson explains **what happens inside the model for each batch**:

1. **Forward Propagation**
2. **Prediction**
3. **Loss Function**
4. **Cost Function**
5. **Why the model makes mistakes**
6. **How those mistakes are measured**

This is where you'll finally understand how a neural network begins learning from its errors.

Next

Excellent! ★

This is the **heart of Machine Learning**.

If you understand this lesson, you'll understand **how every deep learning model (CNN, ResNet, EfficientNet, etc.) learns**.

Everything that happens during training can be summarized like this:

Input
↓

```

Prediction
↓
Compare with Correct Answer
↓
Calculate Error
↓
Improve Model
↓
Repeat

```

Let's understand each step.

Phase 3 (Part 2): Forward Propagation, Prediction & Loss Function

Step 1: Forward Propagation

Definition

Forward Propagation (Forward Pass) is the process where the input data passes through the model to produce an output (prediction).

Nothing is learned in this step.

The model is simply making a guess using its current parameters.

Your Sonar Project

Input:

LOFARGRAM Image

↓

CNN

↓

Output:

Ship Present = 0.82
Ship Absent = 0.18

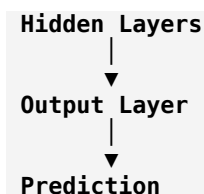
This entire process is called **Forward Propagation**.

Visual Flow

```

LOFARGRAM
↓
Convolution Layers
↓

```



Think of forward propagation as:

The model looks at the input and gives its current best answer.

Step 2: Prediction

The output produced during forward propagation is called the **prediction**.

Example:

Actual Label:

Ship Present

Model predicts:

Ship Present

Correct!

Another example:

Actual Label:

Ship Present

Prediction:

Ship Absent

Wrong!

The model now needs to learn from this mistake.

Step 3: Probability

Modern classifiers don't usually say only:

Ship Present

Instead they output probabilities.

Example:

Ship Present = 0.95

Ship Absent = 0.05

Meaning:

"I'm 95% confident that a ship is present."

Another example:

Ship Present = 0.52

Ship Absent = 0.48

The model is uncertain.

Step 4: What is Loss?

Suppose the correct answer is

Ship Present

Model predicts

Ship Absent

Clearly,

the model made a mistake.

But...

How big is the mistake?

We need a way to measure it.

That measurement is called the **Loss**.

Definition

A **Loss Function** measures how wrong the model's prediction is for **one sample or one batch**.

Small Loss

↓

Good Prediction

Large Loss

↓

Poor Prediction

Example

Actual:

Ship Present

Prediction:

Ship Present = 0.99

Loss

↓

Very Small

Another example

Actual:

Ship Present

Prediction:

Ship Present = 0.20

Loss

↓

Very Large

The larger the loss, the more the model needs to improve.

Student Analogy

Imagine a math test.

Question:

5 + 5 = ?

Student writes

10

Error

↓

0

Excellent.

Another student writes

17

Large error.

Similarly,

loss measures **how far the prediction is from the correct answer**.

Step 5: What is a Loss Function?

A **Loss Function** is simply the mathematical formula used to calculate the loss.

Different problems use different loss functions.

For your binary classification project, the most common choice is:

Binary Cross-Entropy (BCE) Loss.

For now, you only need to remember:

Loss Function = Formula that measures prediction error.

We'll study Binary Cross-Entropy in detail later.

Step 6: Loss vs Accuracy

Many beginners think these are the same.

They are not.

Example:

100 test samples

Model correctly predicts 90.

Accuracy:

90%

Now look at the wrong predictions.

Case 1:

The model was almost correct.

Ship = 0.49

No Ship = 0.51

Case 2:

The model was extremely confident but wrong.

Ship = 0.01

No Ship = 0.99

Both count as **one wrong prediction** for accuracy.

But the **loss** is much larger in Case 2 because the model was confidently incorrect.

That's why training uses **loss**, not accuracy, to improve the model.

Step 7: What is Cost Function?

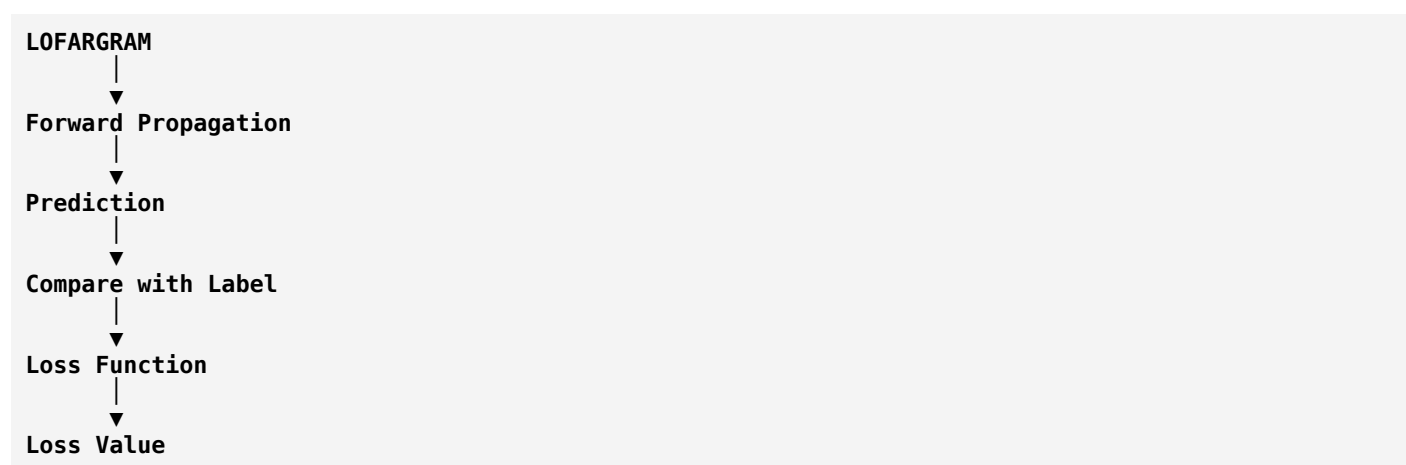
You may hear both **Loss Function** and **Cost Function**.

The distinction is:

- **Loss** → Error for one sample (or sometimes one batch, depending on context).
- **Cost** → Average loss over many samples.

In modern deep learning, people often use the words interchangeably, so don't be surprised if a paper calls the training objective "loss."

Complete Training Flow



At this point,

the model knows:

"I made a mistake."

But it still doesn't know **how to fix it**.

That comes next.

Sonar Example

Input:

LOFARGRAM

Actual Label:

Ship Present

Prediction:

Ship Absent

Loss:

High

The model realizes:

"My prediction is poor. I need to adjust my internal parameters."

How does it adjust them?

Using:

- **Gradient**
- **Gradient Descent**
- **Backpropagation**

These are the next topics.

Summary

Concept	Meaning	Example
Forward Propagation	Input moves through the model to produce an output	LOFARGRAM → CNN → Prediction
Prediction	The model's output	Ship Present (0.92)
Probability	Model's confidence	92% Ship Present
Loss	Measures prediction error	High if the prediction is wrong
Loss Function	Formula to calculate loss	Binary Cross-Entropy
Cost Function	Average loss over many samples	Used to evaluate training progress

Real-Life Analogy

Imagine you're learning archery.

```

Shoot Arrow
  ↓
See Where It Lands
  ↓
Measure Distance from Bullseye
  ↓
Adjust Your Aim
  ↓
Shoot Again

```

- **Arrow** → Prediction

- **Bullseye** → Correct Label
- **Distance from Bullseye** → Loss
- **Adjusting Your Aim** → Learning

The model learns in exactly the same way.

Next Lesson (The Core of Learning)

Now comes the most important lesson in deep learning:

1. **Gradient**
2. **Gradient Descent**
3. **Learning Rate**
4. **Backpropagation**
5. **Optimizer (SGD, Adam)**

These concepts explain **how the model changes its parameters to reduce the loss**. Once you understand them, you'll know exactly how a CNN learns to distinguish **Ship Present** from **Ship Absent**.

Ok next

Excellent! 🌟

This is the **most important topic in all of Machine Learning**.

After this lesson, you'll understand **how a model actually learns from its mistakes**.

Every neural network (CNN, ResNet, EfficientNet, Transformer, etc.) learns using these concepts.

Phase 3 (Part 3): Gradient, Gradient Descent, Learning Rate & Backpropagation

Let's first ask a simple question.

The Problem

Suppose the model predicts:

Ship Absent

But the correct answer is:

Ship Present

The model calculates a **high loss**.

Now the question is:

How does the model know which parameters (weights) to change, and by how much?

The answer involves **Gradient** and **Backpropagation**.

Step 1: What is a Gradient?

Definition

A **gradient** tells the model:

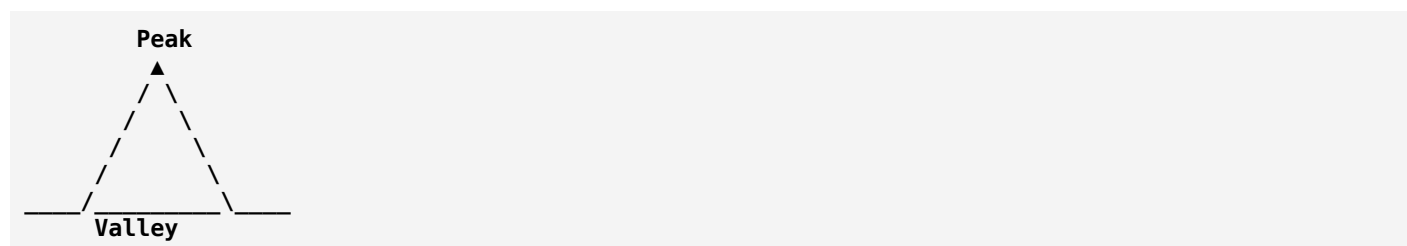
- Which direction to change the parameters.
- How much changing a parameter will increase or decrease the loss.

Think of it as a **guide**.

Mountain Analogy

Imagine you're standing on a mountain.

Your goal is to reach the **lowest point** (the valley).



You don't know the path.

But you can look at the slope.

The slope tells you:

"Walk downhill."

In machine learning:

- Mountain height = **Loss**
- Valley = **Minimum Loss**
- Slope = **Gradient**

The gradient tells the model which way reduces the error.

Step 2: What is Gradient Descent?

Definition

Gradient Descent is the optimization algorithm that updates the model's parameters to reduce the loss.

Simple idea:

```

High Loss
  ↓
Compute Gradient
  ↓
Update Parameters
  ↓
Lower Loss
  
```

Repeat this process many times until the model performs well.

Sonar Example

Initially:

```

Prediction
Ship Absent ❌
  
```

Loss:

```

0.95
  
```

Gradient says:

"Increase some weights, decrease others."

The model updates its parameters.

Next prediction:

```

Ship Present ✅
  
```

Loss:

```

0.20
  
```

The model has improved.

Step 3: What is a Learning Rate?

This is one of the most important **hyperparameters**.

Definition

The **learning rate** controls **how big a step the model takes** when updating its parameters.

Mountain Analogy

Imagine you're walking downhill.

Very Small Steps

```
.
.
.
.
.
Valley
```

You'll eventually reach the valley, but it takes a long time.

Very Large Steps

```
Valley
← Jump →
← Jump →
← Jump →
```

You may keep jumping past the valley and never settle there.

Medium Steps

```
Step
Step
Step
Valley ✓
```

This is usually what we want.

Learning Rate Values

Examples:

```
0.1
0.01
0.001
0.0001
```

Common starting values (depending on the optimizer) are:

- **0.001** (often used with Adam)
- **0.01** (common with SGD, but not universal)

The best value depends on the model and dataset.

Too Small vs Too Large

Learning Rate	Result
Too Small	Training is very slow
Too Large	Training may become unstable or fail to converge
Well Chosen	Faster and stable learning

Step 4: What is Backpropagation?

This is one of the most famous terms in deep learning.

Definition

Backpropagation is the algorithm that computes the gradients for all parameters by propagating the error backward through the network.

It tells every weight:

"Here's how much you contributed to the error."

Why "Back" Propagation?

Remember:

Forward Propagation

```

Input
↓
Hidden Layers
↓
Prediction

```

Now we know the prediction is wrong.

The error travels in the opposite direction.

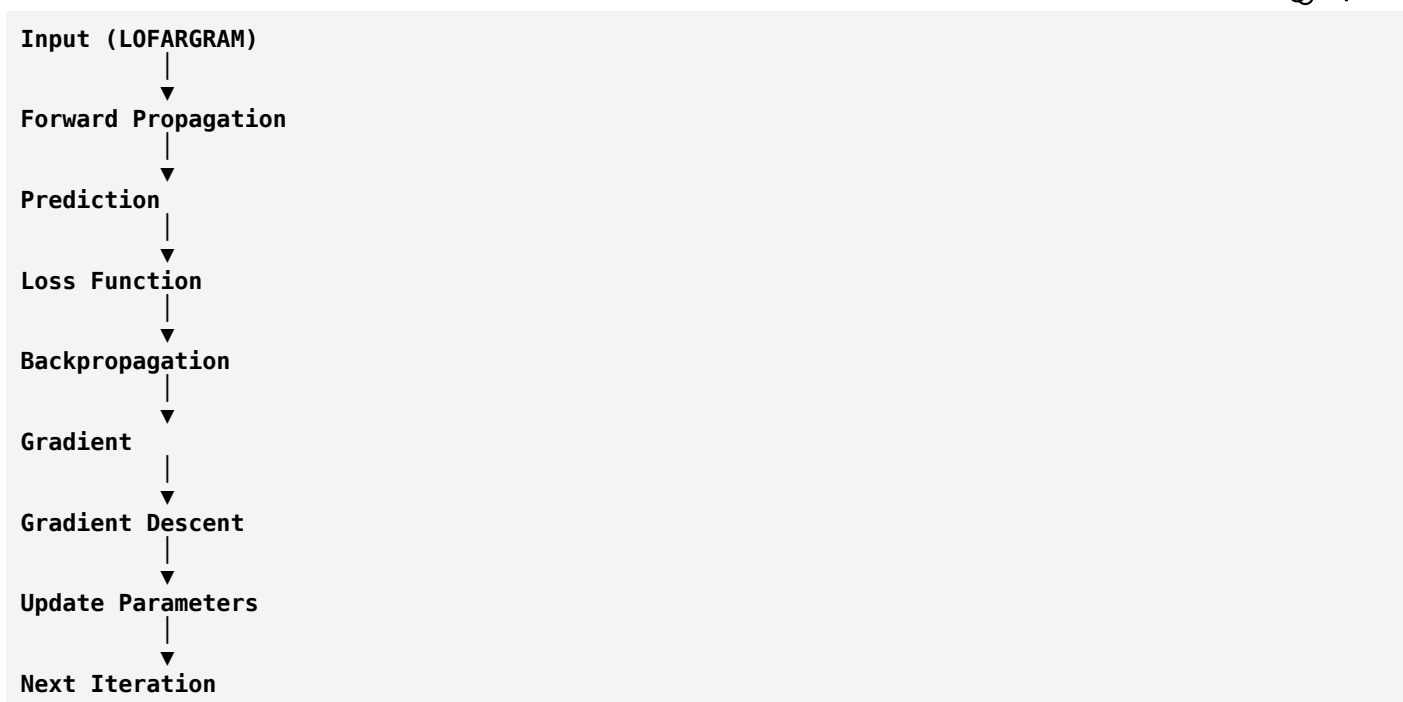
```

Loss
↑
Hidden Layers
↑
Input

```

This backward flow computes gradients for every parameter.

Complete Learning Cycle



This cycle repeats thousands of times during training.

Step 5: What is an Optimizer?

An **optimizer** is the algorithm that uses the gradients to update the model's parameters.

You can think of it as the **driver** that moves the model toward lower loss.

Common optimizers include:

- **SGD (Stochastic Gradient Descent)** – Simple and effective.
- **Adam (Adaptive Moment Estimation)** – One of the most popular optimizers because it adapts the learning process automatically.
- **RMSProp** – Often useful in some sequence-learning tasks.

For many deep learning projects, **Adam** is a common starting choice.

Sonar Project Example

Suppose the model sees:

LOFARGRAM

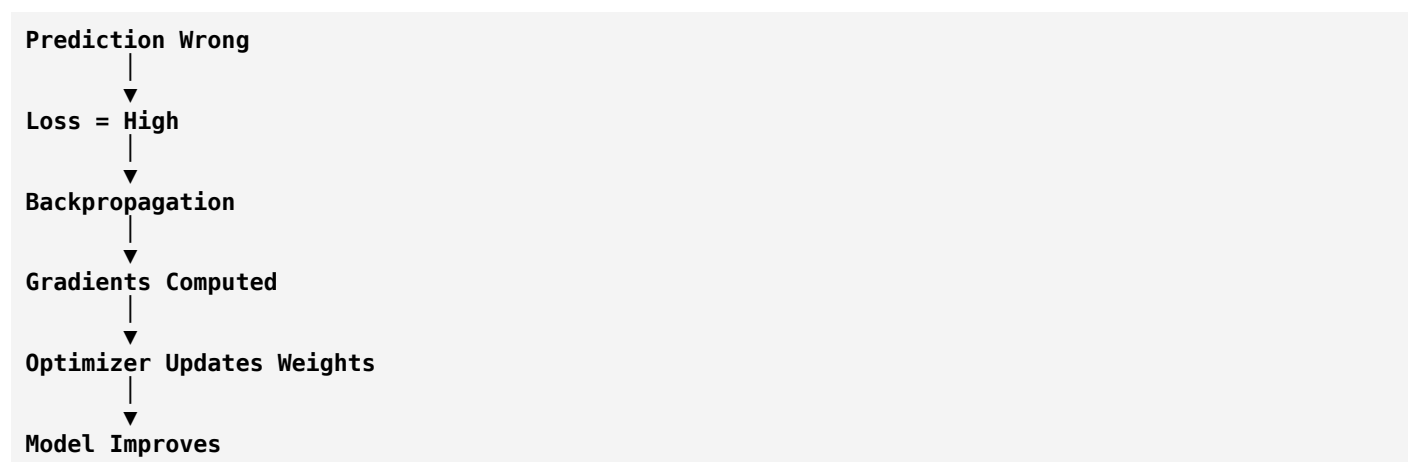
Correct Label:

Ship Present

Prediction:

Ship Absent

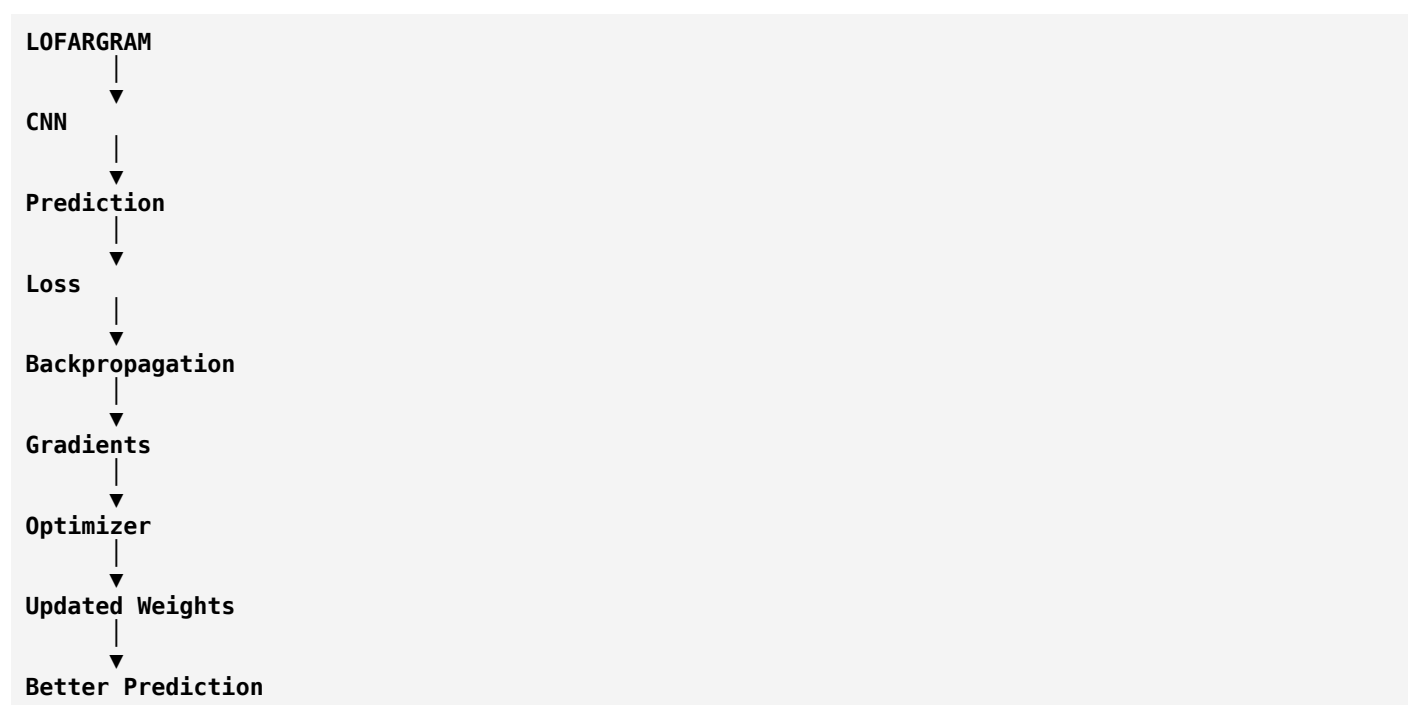
The training process is:



Repeat this for thousands of sonar samples.

Eventually the model learns to recognize ship signatures.

Visual Summary



Real-Life Analogy

Imagine you're learning to throw a ball into a basket.

Attempt 1:

✗ Missed to the left.

You realize:

"I need to throw slightly to the right."

Attempt 2:

✗ Too far.

You adjust again.

Attempt 3:

✓ Basket!

In this analogy:

- **Prediction** → Your throw.
- **Loss** → Distance from the basket.
- **Gradient** → Which direction to adjust.
- **Learning Rate** → How much to adjust.
- **Backpropagation** → Figuring out what caused the miss.
- **Optimizer** → Applying the adjustment.

Summary Table

Concept	Meaning	Simple Analogy
Gradient	Direction and rate of change of the loss	Slope of a mountain
Gradient Descent	Method to reduce loss	Walking downhill
Learning Rate	Step size for parameter updates	Length of each step
Backpropagation	Computes gradients for all parameters	Tracing mistakes back to their causes
Optimizer	Updates parameters using gradients	Driver following the downhill path

One Important Clarification

People often say:

"Backpropagation trains the model."

More precisely:

- **Backpropagation** computes the gradients.
- **The optimizer** uses those gradients to update the parameters.
- **Gradient Descent** is the underlying optimization idea that many optimizers are based on.

They work together as parts of one learning process.

What Comes Next?

You now understand **how the model learns**.

The next phase is **Neural Network Fundamentals**, where we'll build up from a single artificial neuron to a complete neural network:

1. Artificial Neuron
2. Perceptron
3. Weights & Biases (in depth)
4. Activation Functions (ReLU, Sigmoid, Tanh)
5. Input, Hidden, and Output Layers
6. How multiple neurons form a neural network

Once you understand these, CNNs will become much easier to learn because a CNN is built from these same fundamental building blocks.

Ok next

Excellent! 🌟

You now understand **how a model learns**.

The next step is understanding **what the model is made of**.

Think of it this way:

- Previous lesson → **How the brain learns**
- This lesson → **How the brain is built**

This is the foundation of every deep learning model, including **CNN**, **ResNet**, and **EfficientNet**.

Phase 4: Neural Network Fundamentals

We'll cover:

1. Artificial Neuron
2. Perceptron
3. Weights
4. Bias
5. Activation Function
6. Input Layer
7. Hidden Layer
8. Output Layer

1. What is a Neuron?

Let's start with the human brain.

Your brain contains billions of **neurons**.

A neuron:

- Receives information.
- Processes it.
- Sends information to other neurons.

Example:

```

Eyes
↓
Brain Neuron
↓
Decision

```

Artificial Intelligence was inspired by this idea.

2. What is an Artificial Neuron?

An **Artificial Neuron** is the basic building block of a neural network.

It does three things:

1. Receives inputs.
2. Performs mathematical calculations.
3. Produces an output.

Think of it as a **small decision-making unit**.

Sonar Example

Inputs:

```

Feature 1 → Engine Frequency
Feature 2 → Signal Energy
Feature 3 → Spectral Power

```

↓

Artificial Neuron

↓

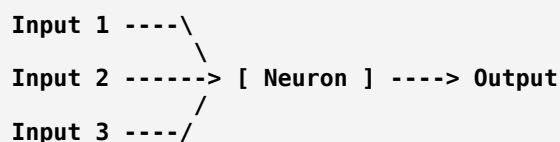
Output:

```

Ship Probability

```

Structure of an Artificial Neuron



Every neuron works like this.

3. What are Weights?

Every input has an associated **weight**.

A weight tells the neuron:

"How important is this input?"

Example

Suppose a neuron receives:

Engine Frequency
Signal Energy
Background Noise

Maybe the model learns:

Feature	Weight
Engine Frequency	0.9
Signal Energy	0.8
Background Noise	0.2

This means:

- Engine frequency is very important.
- Signal energy is important.
- Background noise is less useful.

These weights are **learned during training**.

4. What is Bias?

Bias is another value added before the neuron produces its output.

It allows the neuron to shift its decision threshold.

Think of it like adjusting the starting point of a weighing scale before you begin measuring.

Without a bias, the neuron would be much less flexible.

Simple Formula

A neuron computes something like:

```
Output = Activation((Input × Weight) + Bias)
```

Don't worry about the math right now.

Just remember:

- **Weights** control the importance of each input.
- **Bias** shifts the neuron's decision.

5. What is an Activation Function?

Imagine the neuron calculates:

```
Value = 2.8
```

Should this mean:

- Ship?
- No Ship?

The neuron needs a rule to decide.

That rule is called the **Activation Function**.

It determines the neuron's output.

Why Do We Need Activation Functions?

Without activation functions,

a neural network would behave like a simple linear calculator.

It wouldn't be able to learn complex patterns such as:

- Ship engine harmonics.
- Different tonal signatures.
- Complex LOFARGRAM textures.

Activation functions allow neural networks to learn these nonlinear relationships.

Common Activation Functions

1. ReLU (Rectified Linear Unit)

Rule:

```
If  $x > 0$ 
Output =  $x$ 
Else
Output = 0
```

Example:

Input	Output
-4	0
-1	0
3	3
7	7

ReLU is the most commonly used activation function inside modern CNNs because it is simple and effective.

2. Sigmoid

The output is always between:

```
0
and
1
```

Example:

```
0.95
```

means

95% probability.

For binary classification, the final output layer often uses a sigmoid activation.

Example:

```
0.92
```

↓

Ship Present

```
0.08
```

↓

Ship Absent

3. Tanh

Output range:

-1
to
+1

It is used in some neural network architectures, especially certain recurrent networks, but is less common than ReLU in CNN hidden layers.

6. Input Layer

This is where the data enters the network.

In your project:

LOFARGRAM Image

↓

Input Layer

The input layer simply passes the data into the network.

7. Hidden Layers

These are the layers where learning happens.

Example:

Input

↓

Hidden Layer 1

↓

Hidden Layer 2

↓

Hidden Layer 3

↓

Output

Each hidden layer learns increasingly complex patterns.

For a LOFARGRAM:

- Early layers may detect edges or simple lines.
- Middle layers may detect tonal bands and harmonics.
- Deeper layers may recognize ship-specific patterns.

8. Output Layer

This produces the final prediction.

For your binary classification project:

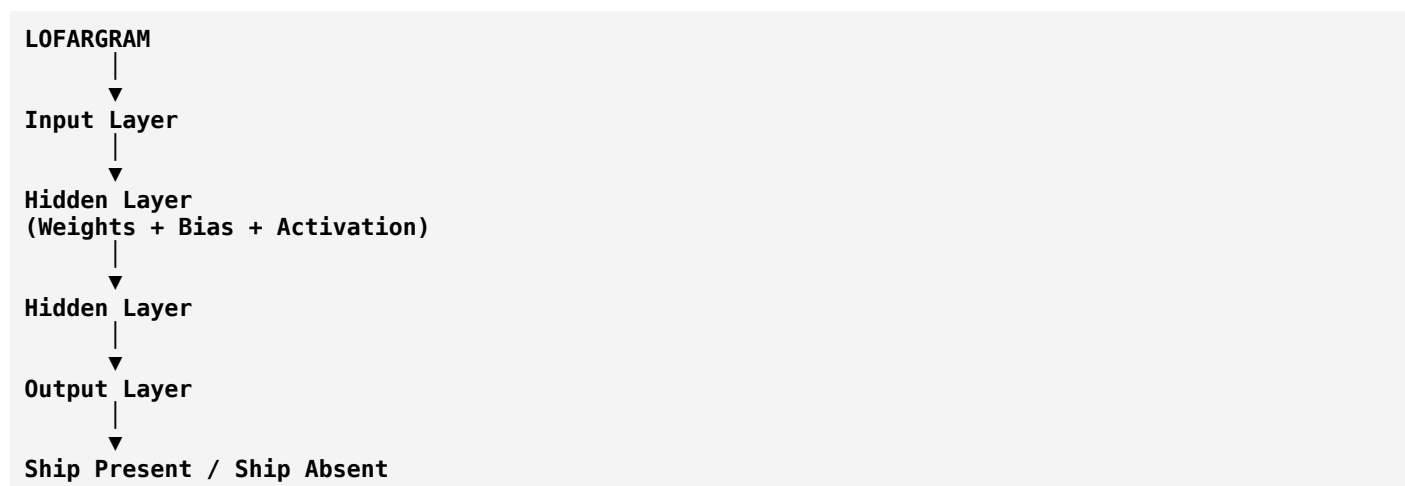
Ship Present = 0.94

or

Ship Absent = 0.06

The output layer gives the model's final decision.

Complete Neural Network



Real-Life Analogy

Imagine hiring people to inspect sonar recordings.

- **Input Layer** → Receives the recording.
- **Hidden Layers** → Teams of experts analyze different clues.
- **Weights** → Importance each expert gives to a clue.
- **Bias** → Personal adjustment to decision-making.
- **Activation Function** → Decides whether the evidence is strong enough.
- **Output Layer** → Final verdict: "Ship Present" or "Ship Absent."

Summary Table

Concept	Meaning	Example in Your Project
Artificial Neuron	Basic computing unit	Processes sonar features
Weight	Importance of an input	Engine frequency may get a higher weight
Bias	Adjustable offset	Helps shift the neuron's decision threshold
Activation Function	Decides the neuron's output	ReLU in hidden layers, Sigmoid in output
Input Layer	Receives data	LOFARGRAM image
Hidden Layer	Learns patterns	Detects tonal signatures and harmonics
Output Layer	Produces prediction	Ship Present / Ship Absent

One Important Clarification

Many beginners think:

One neuron = One complete AI model.

That's not true.

A single neuron can solve only very simple problems.

Real deep learning models contain **thousands to millions of neurons** working together. For example, CNNs like **ResNet** and **EfficientNet** have many layers with millions of trainable parameters.

What Comes Next?

Now that you understand the basic building blocks, we'll move to the topic that is **directly related to your project**:

Convolutional Neural Networks (CNNs)

We'll learn:

1. Why ordinary neural networks are not ideal for images.
2. What a convolution is.
3. What kernels (filters) are.
4. Feature maps.
5. Padding.
6. Stride.
7. Pooling.
8. Flatten.
9. Fully Connected Layers.

10. Why CNNs are perfect for **LOFARGRAMs and spectrograms**.

This is the most important deep learning topic for your passive sonar classification project.

Ok next

Excellent! ★

This is **the most important topic for your project**.

Your model will most likely take a **LOFARGRAM or Spectrogram** as input, and the model that processes these images is usually a **Convolutional Neural Network (CNN)**.

If you understand CNNs well, you'll understand almost every research paper related to sonar image classification.

Phase 5: Convolutional Neural Networks (CNN)

Before learning CNNs, let's answer one question.

Why Can't We Use a Normal Neural Network?

Suppose your LOFARGRAM image is:

128 × 128 pixels

Total pixels:

128 × 128 = 16,384

A normal neural network would connect **every pixel to every neuron**.

Example:

16,384 Inputs
↓
1000 Neurons

That means **millions of connections (weights)**.

Problems:

- Very high memory usage.
- Slow training.
- More prone to overfitting.
- Ignores the fact that nearby pixels are related.

CNNs solve these problems.

What is a CNN?

A **Convolutional Neural Network (CNN)** is a neural network designed specifically for image-like data. Instead of looking at the whole image at once, it looks at **small regions** and gradually learns patterns. Think of how a human examines a picture:

- First, you notice simple edges.
- Then shapes.
- Then complete objects.

A CNN works in a similar hierarchical way.

Your Project

Input:

LOFARGRAM Image

↓

CNN

↓

Prediction:

Ship Present

or

Ship Absent

What is Convolution?

This is the core operation of a CNN.

Instead of processing the whole image, a small matrix called a **kernel (or filter)** slides over the image. At every position, it performs a mathematical operation and produces a new value.

This process helps detect patterns.

Analogy

Imagine reading a large book.

You don't look at all pages simultaneously.

You read:

- Line 1
- Then Line 2
- Then Line 3

Similarly, a CNN examines an image **piece by piece**.

What is a Kernel (Filter)?

A **kernel** is a small matrix, such as:

3×3

or

5×5

It moves across the image searching for specific patterns.

Different kernels learn different features.

Examples:

- Horizontal lines
- Vertical lines
- Edges
- Corners
- Curves
- Textures

During training, the CNN learns the best kernel values automatically.

Sonar Example

Suppose a ship creates strong horizontal tonal lines in a LOFARGRAM.

A learned kernel may become very good at detecting those horizontal structures.

Another kernel may learn to detect:

- Engine harmonics
- Broadband noise
- Repeating frequency patterns

Each kernel specializes in a different feature.

What is a Feature Map?

After a kernel scans the image, it produces a new image.

This new image is called a **Feature Map** (also called an activation map).

Original Image



Kernel scans



Feature Map

The feature map highlights where the kernel found its pattern.

Example

Input:

LOFARGRAM

Kernel:

"Detect horizontal lines"



Feature Map:

Bright regions where horizontal tonal lines are present.

This makes it easier for later layers to recognize ships.

Multiple Kernels

A CNN doesn't use just one kernel.

It may learn dozens or even hundreds.

For example:

Kernel 1 → Horizontal lines

Kernel 2 → Vertical edges

Kernel 3 → Harmonics

Kernel 4 → Curved patterns

Each kernel creates its own feature map.

Together, they provide a rich description of the input image.

What is Stride?

Stride is the number of pixels the kernel moves each time.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Example:

Stride = 1

The kernel moves one pixel at a time.

Stride = 2

The kernel jumps two pixels at a time.

Effect

- Smaller stride → More detailed analysis, but slower computation.
- Larger stride → Faster computation, but some detail may be skipped.

What is Padding?

Suppose the kernel reaches the edge of the image.

Without padding, the output image becomes smaller after each convolution.

Padding solves this by adding extra pixels (usually zeros) around the border before applying the kernel.

Benefits:

- Preserves edge information.
- Helps maintain image size when desired.

What is Pooling?

After convolution, CNNs often perform **pooling**.

Pooling reduces the size of the feature map while keeping the most important information.

This makes computation faster and reduces overfitting.

Max Pooling

This is the most common type.

Suppose we have:

```
2 5
3 1
```

Max Pooling keeps:

```
5
```

It selects the maximum value from a small region.

Think of it as asking:

"What is the strongest feature in this area?"

Why is Pooling Useful for Sonar?

Imagine a ship's tonal line shifts slightly because of noise or movement.

Even if its position changes a little, pooling helps preserve the important feature.

This makes the CNN more robust to small variations.

What is Flatten?

After many convolution and pooling layers, the feature maps are still arranged as images.

Eventually, we convert them into a single long vector.

This process is called **Flattening**.

Example:

```
Feature Maps
  ↓
Flatten
  ↓
[0.2, 0.8, 0.4, ...]
```

This vector is then passed to the final classifier.

Fully Connected Layer

The flattened vector goes into one or more **Fully Connected (Dense) Layers**.

These layers combine all the extracted features to make the final decision.

Example:

```
Extracted Features
  ↓
Dense Layer
  ↓
Output
```

Final Output Layer

For your binary classification project, the output layer usually contains **one neuron** with a **Sigmoid** activation.

Example:

```
0.97
```

Interpretation:

97% probability that a ship is present.

Complete CNN Pipeline for Your Project



How a CNN "Sees" a LOFARGRAM

Instead of seeing:

"This is a ship."

It learns gradually:

- Layer 1 → Simple edges and lines.
- Layer 2 → Tonal bands and harmonics.
- Layer 3 → Complex frequency patterns.
- Layer 4 → Ship-specific acoustic signatures.
- Final Layer → Ship Present or Ship Absent.

This hierarchical learning is what makes CNNs so powerful.

Summary Table

Concept	Meaning	Role in Your Project
CNN	Neural network for image data	Processes LOFARGRAMs
Convolution	Scans the image with kernels	Detects local patterns
Kernel (Filter)	Small learnable matrix	Learns ship-related features
Feature Map	Output of convolution	Highlights detected patterns

Concept	Meaning	Role in Your Project
Stride	Kernel movement step	Controls scanning speed/detail
Padding	Adds border pixels	Preserves edge information
Pooling	Reduces feature map size	Keeps important features, reduces computation
Flatten	Converts feature maps into a vector	Prepares features for classification
Fully Connected Layer	Final decision-making layer	Combines learned features
Sigmoid	Produces probability (0–1)	Outputs Ship Present / Ship Absent

Why CNNs Are Ideal for Passive Sonar

Your LOFARGRAM is essentially an **image**, where:

- **X-axis** = Time
- **Y-axis** = Frequency
- **Pixel intensity** = Signal strength

Ship engines produce characteristic visual patterns—such as horizontal tonal lines and harmonic structures—that CNNs are very good at learning automatically. This is why CNN-based approaches are widely used in passive sonar image classification.

Next Topic

Now that you understand CNNs, we'll move to **Transfer Learning**, where you'll learn:

1. What is a pre-trained model?
2. Why use **ResNet-18** or **EfficientNet** instead of building a CNN from scratch?
3. Feature extraction vs fine-tuning.
4. Frozen vs trainable layers.
5. Why transfer learning is especially useful when your sonar dataset is relatively small.

This is the next key concept because many modern passive sonar classification systems are built by adapting pre-trained CNN models rather than training an entirely new network from scratch.

Ok next

Excellent! ★

Now you've reached one of the **most practical topics** in Deep Learning.

If you read recent research papers on passive sonar, you'll often see models like:

- **ResNet-18**
- **EfficientNet**

- **MobileNet**
- **DenseNet**
- **VGG16**

You may wonder:

"Why don't researchers build their own CNN from scratch?"

The answer is **Transfer Learning**.

Phase 6: Transfer Learning

First, What is the Problem?

Suppose you want to build a CNN from scratch.

Your dataset contains:

2,000 LOFARGRAM images

Is this enough?

Usually **no**.

Modern CNNs have **millions of parameters**. Training them well often requires **tens or hundreds of thousands of images** (or more, depending on the model and task).

With only a small dataset, the model may **overfit**—it memorizes the training data instead of learning patterns that generalize.

The Solution: Transfer Learning

Definition

Transfer Learning means taking a model that has already learned useful visual features from a large dataset and adapting it to solve your new task.

Instead of starting from zero, you start with an experienced model.

Human Analogy

Imagine two people.

Person A

Never learned to drive.

Teaching them takes months.

Person B

Already knows how to drive a car.

Now teach them to drive a truck.

They learn much faster because they already understand:

- Steering
- Braking
- Traffic rules

This is **transfer learning**.

Existing knowledge is transferred to a new task.

Deep Learning Example

Suppose a CNN has already learned from millions of images.

It already understands basic visual concepts like:

- Edges
- Lines
- Curves
- Shapes
- Textures

Instead of training a new CNN,
you reuse this knowledge.

Then teach it:

Ship Present

Ship Absent

Only the final part of the network needs to adapt to your specific problem.

Pre-trained Model

A **pre-trained model** is a model that has already been trained on a large dataset.

It has already learned useful features.

Examples include:

- ResNet-18
- EfficientNet

- MobileNet
- DenseNet

These models are commonly pre-trained on a large image dataset called **ImageNet**, which contains millions of labeled images.

Even though ImageNet contains everyday images (animals, vehicles, objects), the early layers learn general features like edges and textures that are useful for many vision tasks—including spectrograms and LOFARGRAMs.

Why Can It Help with Sonar?

You might ask:

"Ships don't look like cats or cars. Why use an ImageNet model?"

Great question.

The first layers of a CNN don't recognize ships or cats.

They learn **basic visual patterns** such as:

- Edges
- Horizontal lines
- Vertical lines
- Curves
- Textures

A LOFARGRAM also contains these kinds of visual structures.

So those learned low-level features can still be valuable.

The deeper layers are then adapted to recognize sonar-specific patterns.

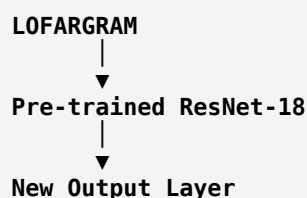
Feature Extraction

One way to use transfer learning is **Feature Extraction**.

Here:

- Keep almost all of the pre-trained model unchanged.
- Replace the final classification layer with a new one for your task.

Example:



↓
Ship / No Ship

Only the new output layer is trained.

Fine-Tuning

Another approach is **Fine-Tuning**.

Here:

- Start with a pre-trained model.
- Continue training some (or all) of its layers using your sonar dataset.

The model gradually adjusts its existing knowledge to your specific domain.

Feature Extraction vs Fine-Tuning

Feature Extraction	Fine-Tuning
Most pre-trained layers are kept fixed	Some or all layers continue learning
Faster training	Slower training
Needs less data	Usually benefits from more data
Lower risk of overfitting	More flexible, but higher overfitting risk if data is limited

Frozen Layers

A **frozen layer** is a layer whose parameters are **not updated** during training.

It keeps the knowledge learned from pre-training.

Example:

Layer 1

Frozen

↓

Layer 2

Frozen

↓

Layer 3

Frozen



Output Layer

Trainable

Trainable Layers

These layers continue learning.

Their parameters change during training.

Example:

Last Few Layers

Trainable

They gradually adapt to sonar data.

Which Strategy is Better?

It depends on your dataset.

Small Dataset

Use:

- Feature Extraction
- Freeze most layers

This reduces the risk of overfitting.

Larger Dataset

Use:

- Fine-Tuning

The model can learn more sonar-specific features.

Sonar Example

Suppose you have:

4,000 LOFARGRAM images

A common approach is:

```

ResNet-18
|
Freeze Early Layers
|
Fine-Tune Later Layers
|
Train New Classifier
|
Ship Present / Ship Absent

```

The early layers keep their general visual knowledge.

The later layers specialize in detecting sonar signatures.

Why Researchers Use ResNet-18

ResNet-18 is popular because it:

- Is relatively lightweight.
- Trains faster than deeper ResNet variants.
- Often performs well on smaller datasets.
- Uses **skip (residual) connections**, which help very deep networks learn more effectively.

We'll discuss residual connections when we study model architectures.

Why Researchers Use EfficientNet

EfficientNet is designed to achieve strong accuracy while using fewer parameters and less computation.

It balances:

- Network depth
- Width
- Input image resolution

This often makes it a good choice when computational resources are limited.

Complete Transfer Learning Pipeline

```

LOFARGRAM
|
▼
Pre-trained CNN
(ResNet-18 / EfficientNet)
|
▼
Feature Extraction
|
▼
Fine-Tuning (Optional)
|
▼
New Classifier
|

```

Summary Table

Concept	Meaning	Role in Your Project
Transfer Learning	Reusing knowledge from a pre-trained model	Faster and better training
Pre-trained Model	Model already trained on a large dataset	ResNet-18, EfficientNet
Feature Extraction	Keep most layers fixed and train a new classifier	Good for small datasets
Fine-Tuning	Continue training some or all layers	Adapts the model to sonar data
Frozen Layers	Parameters do not change	Preserve pre-trained knowledge
Trainable Layers	Parameters continue updating	Learn sonar-specific patterns

Real-Life Analogy

Imagine hiring an experienced music teacher to teach a student to recognize ship engine sounds.

The teacher already understands:

- Sound patterns
- Rhythm
- Frequency

Now they only need to learn:

- What a ship engine sounds like.

That's much easier than training someone with no experience at all.

Transfer learning works the same way.

Congratulations! 🎉

At this point, you've built a solid conceptual foundation in:

- ✅ AI, ML, and Deep Learning
- ✅ Datasets, Features, and Labels
- ✅ Training, Validation, and Test Sets
- ✅ Models, Parameters, and Hyperparameters
- ✅ Epochs, Batches, and Iterations
- ✅ Forward Propagation and Loss
- ✅ Gradient Descent and Backpropagation
- ✅ Neural Networks

-  CNNs
-  Transfer Learning

Next Phase

The next topic is **Model Evaluation**, where you'll learn how to answer:

"Is my ship classifier actually good?"

We'll cover:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score
- Sensitivity
- Specificity
- ROC Curve
- AUC
- Precision-Recall Curve

These metrics are essential because **accuracy alone can be misleading**, especially if your sonar dataset is imbalanced (for example, if "No Ship" recordings greatly outnumber "Ship Present" recordings). Understanding these metrics will help you correctly evaluate and compare different models.

Ok next

Excellent! 

This is one of the **most important phases** of Machine Learning.

Many beginners think:

"My model has 98% accuracy, so it's excellent."

Not necessarily.

A model can have **98% accuracy and still be almost useless**.

Let's understand why.

Phase 7: Model Evaluation Metrics

Suppose your trained model predicts:

LOFARGRAM
↓
CNN
↓
Ship Present

How do we know if the prediction is good?

We use **evaluation metrics**.

The most important one is the **Confusion Matrix**.

1. What is a Confusion Matrix?

A **Confusion Matrix** is a table that compares:

- What the model predicted
- What the correct answer actually was

For a binary classification problem like yours:

Actual \ Predicted	Ship Present	Ship Absent
Ship Present	TP	FN
Ship Absent	FP	TN

This table is the foundation of almost every evaluation metric.

2. Understanding TP, TN, FP, FN

These four terms are extremely important.

True Positive (TP)

A ship is actually present.

The model predicts:

Ship Present

Correct!

Example:

Actual	Prediction
Ship	Ship

This is **True Positive**.

True Negative (TN)

No ship is actually present.

The model predicts:

Ship Absent

Correct.

Actual	Prediction
No Ship	No Ship

This is **True Negative**.

False Positive (FP)

There is **no ship**.

The model predicts:

Ship Present

Wrong.

This is also called a **False Alarm**.

In sonar systems, false alarms can waste time and resources because operators investigate something that isn't a ship.

False Negative (FN)

A ship **is present**.

The model predicts:

Ship Absent

Wrong.

This means the system **missed a real ship**.

For many defense applications, this is often considered the most serious type of error.

Visual Summary

Actual	Prediction	Result
Ship	Ship	True Positive (TP) 

Actual	Prediction	Result
No Ship	No Ship	True Negative (TN) ✓
No Ship	Ship	False Positive (FP) ✗
Ship	No Ship	False Negative (FN) ✗

Example

Suppose you test the model on **100 sonar recordings**.

Results:

TP = 40

TN = 50

FP = 5

FN = 5

Now let's calculate the metrics.

3. Accuracy

Definition

Accuracy tells us:

How many predictions were correct overall?

Formula:

Accuracy

=

(TP + TN)

/

(TP + TN + FP + FN)

Example:

(40 + 50)

/

100

=

90%

The model is correct **90% of the time**.

Problem with Accuracy

Imagine this dataset:

990
No Ship
10
Ship

Now imagine a lazy model that always predicts:

No Ship

Correct predictions:

990

Wrong predictions:

10

Accuracy:

99%

Looks amazing!

But the model **never detects a ship**.

So accuracy alone can be very misleading, especially with **imbalanced datasets**.

4. Precision

Definition

Precision answers:

When the model predicts "Ship Present," how often is it correct?

Formula:

Precision
=
TP
/
(TP + FP)

Example:

TP = 40

FP = 5

Precision:

40 / 45

=

88.9%

High precision means **few false alarms**.

Sonar Meaning

High Precision

↓

If the system says

Ship Present

it is usually correct.

Operators waste less time checking false detections.

5. Recall (Sensitivity)

Definition

Recall answers:

Of all the real ships, how many did the model successfully detect?

Formula:

Recall

=

TP

/

(TP + FN)

Example:

TP = 40

FN = 5

Recall:

40 / 45

=

88.9%

Sonar Meaning

High Recall

↓

The system misses very few ships.

This is often extremely important in surveillance applications.

Precision vs Recall

Imagine a radar operator.

High Precision

Whenever they report a ship,
they are almost always correct.

But they may miss some ships.

High Recall

They detect almost every ship.

But sometimes they also report false alarms.

There is often a trade-off between precision and recall.

6. F1 Score

Sometimes we want one number that balances both precision and recall.

That is the **F1 Score**.

It becomes high only when **both precision and recall are high**.

Example:

Precision:

90%

Recall:

90%

↓

High F1 Score.

If one is very low, the F1 Score also drops.

7. Specificity

Specificity answers:

Of all the recordings with no ship, how many did the model correctly identify as "No Ship"?

Formula:

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

High specificity means the model produces fewer false alarms.

8. ROC Curve

A classifier often outputs probabilities, such as:

Ship Present = 0.82

You choose a threshold (for example, 0.5) to decide whether to classify it as "Ship Present."

By changing this threshold, the balance between true positives and false positives changes.

A **ROC (Receiver Operating Characteristic) Curve** shows this trade-off across many thresholds.

9. AUC

AUC (Area Under the ROC Curve) summarizes the ROC curve into a single number.

General interpretation:

- **1.0** → Excellent separation between classes.
- **0.5** → About as good as random guessing.

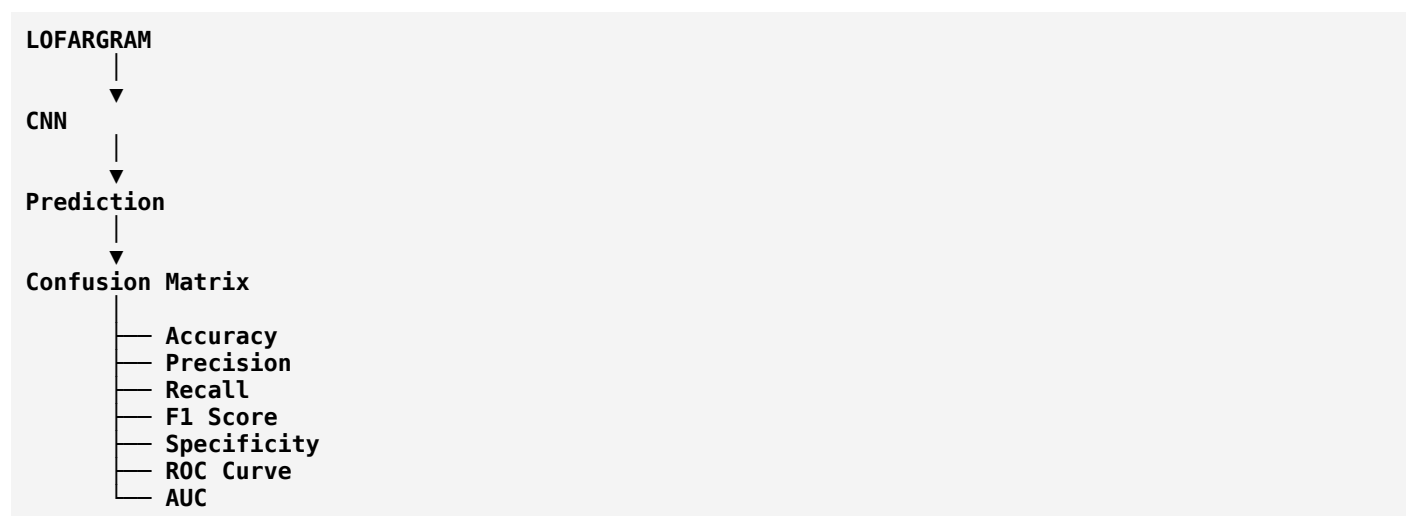
Higher AUC generally indicates better discrimination between "Ship Present" and "Ship Absent."

10. Precision-Recall Curve

When your dataset is **imbalanced** (for example, far more "No Ship" than "Ship Present" samples), the **Precision-Recall (PR) Curve** is often more informative than the ROC curve.

Since many passive sonar datasets can be imbalanced, researchers frequently report both ROC-AUC and PR-AUC.

Complete Evaluation Flow



Summary Table

Metric	Answers the Question	Why It Matters
Confusion Matrix	What was predicted vs what was correct?	Foundation of all metrics
Accuracy	How many predictions were correct overall?	Good for balanced datasets
Precision	When the model predicts "Ship," how often is it right?	Reduces false alarms
Recall (Sensitivity)	How many real ships were detected?	Reduces missed ships
F1 Score	Is there a good balance between precision and recall?	Useful when both are important
Specificity	How well are "No Ship" recordings identified?	Measures rejection of false alarms
ROC Curve	How performance changes with the threshold	Evaluates classifiers across thresholds
AUC	Overall ability to separate classes	Higher is generally better
PR Curve	Precision vs Recall across thresholds	Especially useful for imbalanced datasets

Which Metrics Matter Most for Your Project?

For a **Passive Sonar Ship Detection** system, these are especially important:

1. **Recall (Sensitivity)** – Missing a real ship (False Negative) can be very costly.
2. **Precision** – Too many false alarms reduce operator trust and waste resources.

3. **F1 Score** – Good overall balance when both missed detections and false alarms matter.

4. **ROC-AUC / PR-AUC** – Useful for comparing different models and thresholds.

Accuracy is still useful, but it **should not be the only metric** you rely on.

Congratulations! 🎉

You now understand the complete conceptual pipeline from:

- Digital Audio Processing (FFT, STFT, Spectrogram, Mel Spectrogram, MFCC, LOFARGRAM)
- Machine Learning fundamentals
- Neural Networks
- CNNs
- Transfer Learning
- Model Evaluation

You now have the background needed to start reading passive sonar research papers with much greater confidence. The next natural step is to study **popular CNN architectures** (ResNet, EfficientNet, MobileNet, DenseNet, VGG) and understand **why researchers choose one architecture over another** for sonar classification tasks. That will make it much easier to understand the models used in your project.

What about deep learning basics? have we learned everything that we have to learn or to know?

That's a very good question. The answer is:

No. We have built a strong foundation, but we haven't learned everything yet.

What we've covered is roughly **50–60% of the deep learning concepts** you'll need for your project. We now understand *how* models learn, but not yet *why different architectures work* or *how they're designed*.

Here's where you stand:

✅ Completed (Strong Foundation)

1. Digital Audio Processing

- ✅ Fourier Transform
- ✅ FFT
- ✅ STFT
- ✅ Spectrogram
- ✅ Mel Spectrogram
- ✅ MFCC

- ☒ LOFARGRAM
-

2. Machine Learning Basics

- ☒ AI vs ML vs DL
 - ☒ Supervised Learning
 - ☒ Dataset
 - ☒ Sample
 - ☒ Feature
 - ☒ Label
 - ☒ Binary Classification
 - ☒ Training / Validation / Test Sets
-

3. Model Fundamentals

- ☒ Model
 - ☒ Parameters
 - ☒ Hyperparameters
 - ☒ Prediction
 - ☒ Inference
-

4. Training Process

- ☒ Epoch
 - ☒ Batch
 - ☒ Batch Size
 - ☒ Iteration
 - ☒ Forward Propagation
 - ☒ Loss Function
 - ☒ Gradient
 - ☒ Gradient Descent
 - ☒ Learning Rate
 - ☒ Backpropagation
 - ☒ Optimizer
-

5. Neural Networks

- ☒ Artificial Neuron
 - ☒ Perceptron
 - ☒ Weights
 - ☒ Bias
 - ☒ Activation Functions
 - ☒ Hidden Layers
 - ☒ Output Layer
-

6. CNN Basics

- ☒ Convolution
 - ☒ Kernel
 - ☒ Feature Map
 - ☒ Stride
 - ☒ Padding
 - ☒ Pooling
 - ☒ Flatten
 - ☒ Fully Connected Layer
-

7. Transfer Learning

- ☒ Pre-trained Models
 - ☒ Feature Extraction
 - ☒ Fine-Tuning
 - ☒ Frozen Layers
 - ☒ Trainable Layers
-

8. Evaluation

- ☒ Confusion Matrix
- ☒ Accuracy
- ☒ Precision
- ☒ Recall

-  F1 Score
-  Specificity
-  ROC
-  AUC

Still Remaining (Very Important)

These topics are commonly used in research papers and practical deep learning.

Phase A: Deep Learning Architectures ★★★★★

- ResNet
- EfficientNet
- VGG
- MobileNet
- DenseNet
- Inception
- AlexNet

These explain **why researchers choose one model over another**.

Phase B: Regularization ★★★★★

These help prevent overfitting.

- Overfitting
- Underfitting
- Bias vs Variance
- Dropout
- Weight Decay (L2 Regularization)
- Early Stopping
- Data Augmentation

Phase C: Optimization ★★★★★

Go a bit deeper into:

- SGD

- Adam
 - RMSProp
 - AdamW
 - Momentum
 - Learning Rate Scheduler
 - Warm-up
-

Phase D: Loss Functions ★★★★★

Understand when to use:

- Binary Cross-Entropy
 - Categorical Cross-Entropy
 - Focal Loss
 - Dice Loss (good to know, common in segmentation)
-

Phase E: Practical Deep Learning ★★★★★

These are essential when implementing models.

- Tensor
 - Tensor Shape
 - Channels
 - Batch Dimension
 - GPU
 - CUDA
 - TensorFlow vs PyTorch
 - Model Checkpoint
 - Saving & Loading Models
-

Phase F: Reading Research Papers ★★★★★

Learn terms like:

- Baseline
- Proposed Model
- State-of-the-Art (SOTA)

- Ablation Study
- Benchmark Dataset
- Generalization

Phase G: Sonar-Specific Deep Learning ★★★★★

This is directly related to your project.

- Why CNN works on spectrograms
- Why ResNet-18 is popular in sonar
- Why EfficientNet performs well
- Attention Mechanisms (basic idea)
- LSTM / Bi-LSTM for temporal information
- CNN + LSTM hybrid models
- Vision Transformers (ViT) (basic understanding)

What We DON'T Need (For Your Current Project)

You don't need to master everything in deep learning right now, such as:

- Generative Adversarial Networks (GANs)
- Diffusion Models
- Large Language Models (LLMs)
- Reinforcement Learning
- Graph Neural Networks (GNNs)
- Object Detection models (YOLO, Faster R-CNN)
- Image Segmentation models (U-Net, Mask R-CNN)

These are valuable topics, but they aren't necessary for a passive sonar binary classification project.

My Recommended Learning Order

This is the path I'd suggest from here:

- Phase 1  Digital Audio Processing
- Phase 2  Machine Learning Basics
- Phase 3  Training Process
- Phase 4  Neural Networks
- Phase 5  CNN

Phase 6  Transfer Learning

Phase 7  Evaluation Metrics

NOW CONTINUE WITH

Phase 8 → Overfitting & Underfitting

↓

Phase 9 → Regularization Techniques

↓

Phase 10 → Deep CNN Architectures
(ResNet, EfficientNet, etc.)

↓

Phase 11 → Practical Deep Learning Concepts

↓

Phase 12 → Sonar-Specific Deep Learning

↓

Phase 13 → Reading Research Papers

↓

Phase 14 → Implementation (PyTorch/TensorFlow)

My recommendation

Before jumping into **ResNet** and **EfficientNet**, I recommend learning:

1. **Overfitting & Underfitting**
2. **Bias & Variance**
3. **Dropout**
4. **Early Stopping**
5. **Data Augmentation**

The reason is that these concepts explain **why models succeed or fail**. Once you understand them, the design choices in architectures like ResNet and EfficientNet will make much more sense, and you'll be able to understand research papers much more easily.

After that, we'll study **ResNet-18** and **EfficientNet** in depth, since those are the architectures most relevant to your passive sonar classification project.

Ok start next

Perfect. From this point onward, we'll focus on the remaining deep learning concepts that are **directly useful for your passive sonar project**.

Phase 8: Overfitting, Underfitting & Bias-Variance Tradeoff

This is one of the **most important concepts** in deep learning.

Many students build a CNN, get **99% training accuracy**, and think:

"My model is amazing!"

But when they test it on new data, the accuracy drops to **60%**.

Why?

The answer is **Overfitting**.

Imagine This Scenario

Suppose you have:

- **Training Dataset:** 8,000 LOFARGRAM images
- **Test Dataset:** 2,000 LOFARGRAM images

After training:

```
Training Accuracy = 99%
Test Accuracy = 58%
```

This looks strange.

If the model is so good, why is the test accuracy so low?

Because it **memorized** the training data instead of learning general patterns.

This is called **Overfitting**.

1. What is Underfitting?

Definition

Underfitting occurs when the model is **too simple** to learn the patterns in the data.

It performs poorly on both the training and test datasets.

Example

```
Training Accuracy = 60%
Test Accuracy = 58%
```

The model cannot even learn the training data well.

Student Analogy

A student studies only the chapter titles before an exam.

Result:

- Performs poorly in practice tests.
- Performs poorly in the final exam.

The student didn't learn enough.

That's **underfitting**.

2. What is Overfitting?

Definition

Overfitting occurs when the model memorizes the training data instead of learning general patterns. It performs very well on the training set but poorly on unseen data.

Example

Training Accuracy = 99%
Test Accuracy = 62%

The model remembers the exact training examples but struggles with new recordings.

Student Analogy

Imagine a student memorizes the answers to last year's question paper.

If the same questions appear:

✓ 100%

If new questions appear:

✗ Poor performance.

The student memorized instead of understanding.

This is exactly what an overfit model does.

3. What is Good Fitting?

This is what we want.

Example:

Training Accuracy = 95%
Test Accuracy = 93%

The model has learned the underlying patterns rather than memorizing the data.

Visual Comparison

Underfitting

Training = 60%
Testing = 58%

Model hasn't learned enough.

Good Fit

Training = 95%
Testing = 93%

Model generalizes well.

Overfitting

Training = 99%
Testing = 60%

Model memorized the training data.

Why Does Overfitting Happen?

Common reasons include:

- Dataset is too small.
- Model is too complex.
- Training for too many epochs.
- Too many parameters.
- Too much noise in the data.

For sonar data, overfitting is common because collecting large, labeled underwater datasets is difficult.

Why Is Overfitting Dangerous?

Imagine your sonar system is deployed at sea.

If it only recognizes ships that look like the training examples, it may fail to detect ships in different ocean conditions or with different noise patterns.

A good model must work well on **new** data, not just the data it has already seen.

4. What is Bias?

In machine learning (not the neuron bias we discussed earlier), **bias** refers to error caused by an overly simple model.

High bias means:

"The model cannot capture the true relationship in the data."

Characteristics:

- Low training accuracy.
- Low test accuracy.
- Underfitting.

5. What is Variance?

Variance refers to how much a model changes when trained on different datasets.

High variance means:

"The model is too sensitive to the training data."

Characteristics:

- Very high training accuracy.
- Poor test accuracy.
- Overfitting.

Bias vs Variance

High Bias	High Variance
Model is too simple	Model is too complex
Underfitting	Overfitting
Misses important patterns	Memorizes training data

The goal is to find a balance between the two.

How Do We Detect Overfitting?

Compare the training and validation performance during training.

Example:

Epoch	Training Accuracy	Validation Accuracy
1	65%	64%
5	82%	81%
10	92%	91%
20	99%	85%
30	100%	78%

Notice:

- Training accuracy keeps increasing.
- Validation accuracy starts decreasing.

This is a classic sign of **overfitting**.

Sonar Example

Suppose your CNN learns:

- Exact background noise from one recording location.
- Exact ocean conditions from your training set.

When tested in a different sea region, performance drops.

The model learned **location-specific noise** instead of **ship characteristics**.

That's overfitting.

How Can We Reduce Overfitting?

We'll study these in detail next:

- ☒ More training data
- ☒ Data Augmentation
- ☒ Dropout
- ☒ Early Stopping
- ☒ Weight Decay (L2 Regularization)
- ☒ Transfer Learning
- ☒ Batch Normalization (good to know)

Summary Table

Concept	Meaning	Result
Underfitting	Model is too simple	Poor training and test performance
Good Fit	Learns general patterns	High training and test performance
Overfitting	Memorizes training data	High training, poor test performance
High Bias	Can't learn enough	Underfitting
High Variance	Learns too specifically	Overfitting

Key Takeaways

- **Training accuracy alone is not enough.**
- **Validation performance tells you how well the model generalizes.**
- The ultimate goal is **good performance on unseen sonar recordings**, not perfect performance on the training set.

Next Lesson

Now that you know what overfitting is, the next lesson is:

Regularization Techniques

We'll cover:

1. **Dropout** (one of the most important techniques in deep learning)
2. **Early Stopping**
3. **Data Augmentation**
4. **Weight Decay (L2 Regularization)**
5. **Batch Normalization**

These techniques are used in almost every modern CNN—including ResNet and EfficientNet—to improve generalization and reduce overfitting.

Ok next

Excellent! ★

This is one of the **most practical topics** in Deep Learning.

Almost every research paper you read will mention one or more of these techniques.

These methods don't make the model "smarter"—they make it **generalize better**, meaning it performs well on **new sonar recordings**, not just the training data.

Phase 9: Regularization Techniques

What is Regularization?

Definition

Regularization refers to techniques used to **reduce overfitting** and improve a model's ability to generalize to unseen data.

Think of it as preventing a student from memorizing answers and instead encouraging them to understand the subject.

1. Dropout

What is Dropout?

Imagine a classroom where the same few students always answer every question.

The others never learn.

A teacher decides that in each class, some of the active students must stay silent so others are forced to participate.

This helps the whole class learn better.

That's the idea behind **Dropout**.

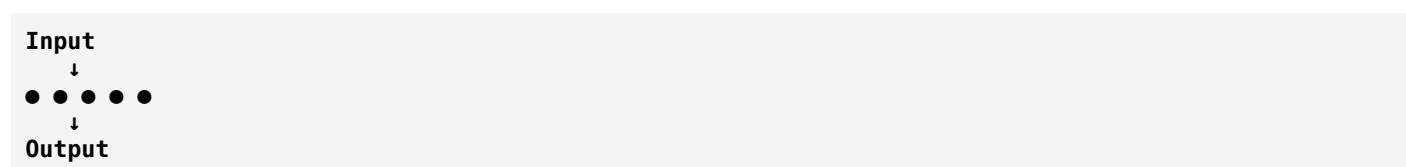
How Dropout Works

During training:

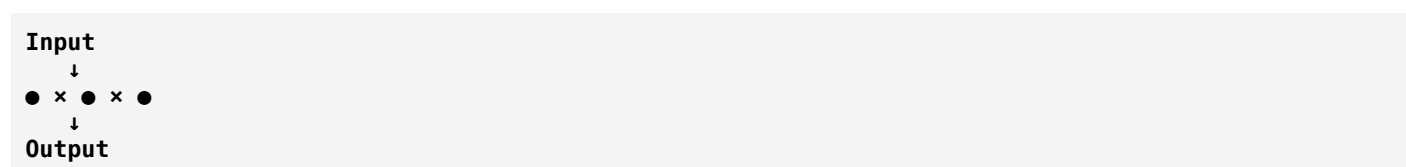
Some neurons are **temporarily turned off (dropped)**.

Example:

Without Dropout



With Dropout



- • = Active neuron
- × = Dropped neuron

The dropped neurons do **not** participate in that training step.

In the next batch, a different set of neurons may be dropped.

Why Use Dropout?

Without dropout:

- The network may rely too much on a few neurons.
- It can memorize the training data.

With dropout:

- The network learns more robust features.
 - It becomes less dependent on any single neuron.
 - Overfitting is reduced.
-

Dropout Rate

The **dropout rate** is the fraction of neurons dropped during training.

Examples:

- 0.2 → Drop 20% of neurons.
- 0.5 → Drop 50% of neurons.

Common values are **0.2 to 0.5**, but the best choice depends on the model and dataset.

During Testing

An important point:

Dropout is used only during training.

During validation and testing:

- All neurons are active.
- The full network is used to make predictions.

2. Early Stopping

The Problem

Suppose you train for 100 epochs.

Training accuracy keeps improving.

But validation accuracy behaves like this:

Epoch	Validation Accuracy
5	80%
10	88%
15	92%
20	93% 
25	92%
30	90%
40	86%

After epoch 20, the model starts to overfit.

Solution

Stop training when validation performance stops improving.

This is called **Early Stopping**.

Instead of training for 100 epochs, you might stop at epoch 20 and keep the best-performing model.

Analogy

A student studies:

- Day 1 → Learns.
- Day 2 → Learns more.
- Day 3 → Learns more.
- Day 20 → Fully prepared.

After that, they keep rereading the same notes without gaining new understanding.

Stopping at the right time is more effective.

3. Data Augmentation

What is Data Augmentation?

Instead of collecting more real data, we create **new training samples** by applying transformations to existing ones.

For ordinary images, examples include:

- Rotation
- Cropping
- Brightness adjustment
- Horizontal flipping (when appropriate)

This helps the model see more variation.

Sonar-Specific Note

For **LOFARGRAMs** and **spectrograms**, augmentation must be chosen carefully.

Some common image augmentations (like arbitrary rotations or horizontal flips) may change the meaning of the data because the axes represent **time** and **frequency**.

Researchers often use augmentations that preserve the physical meaning of the signal, such as:

- Small time shifts

- Controlled noise addition
- Time masking
- Frequency masking (similar to SpecAugment)
- Slight amplitude scaling

The right augmentation depends on the application and should reflect realistic sonar conditions.

Why It Helps

More variation means:

- Less memorization.
 - Better generalization.
 - Better performance on unseen recordings.
-

4. Weight Decay (L2 Regularization)

The Idea

During training, some weights may become unnecessarily large.

Very large weights can make the model fit the training data too closely.

Weight Decay adds a penalty for large weights.

This encourages the model to keep weights smaller unless they are truly needed.

Analogy

Imagine a company gives bonuses based on performance.

But if someone asks for an extremely large bonus, there's a penalty.

Employees are encouraged to make reasonable choices.

Similarly, weight decay discourages excessively large parameter values.

5. Batch Normalization

The Problem

As data flows through many layers, its distribution changes.

This can make training slower and less stable.

Solution

Batch Normalization normalizes the activations within a mini-batch before passing them to the next layer.

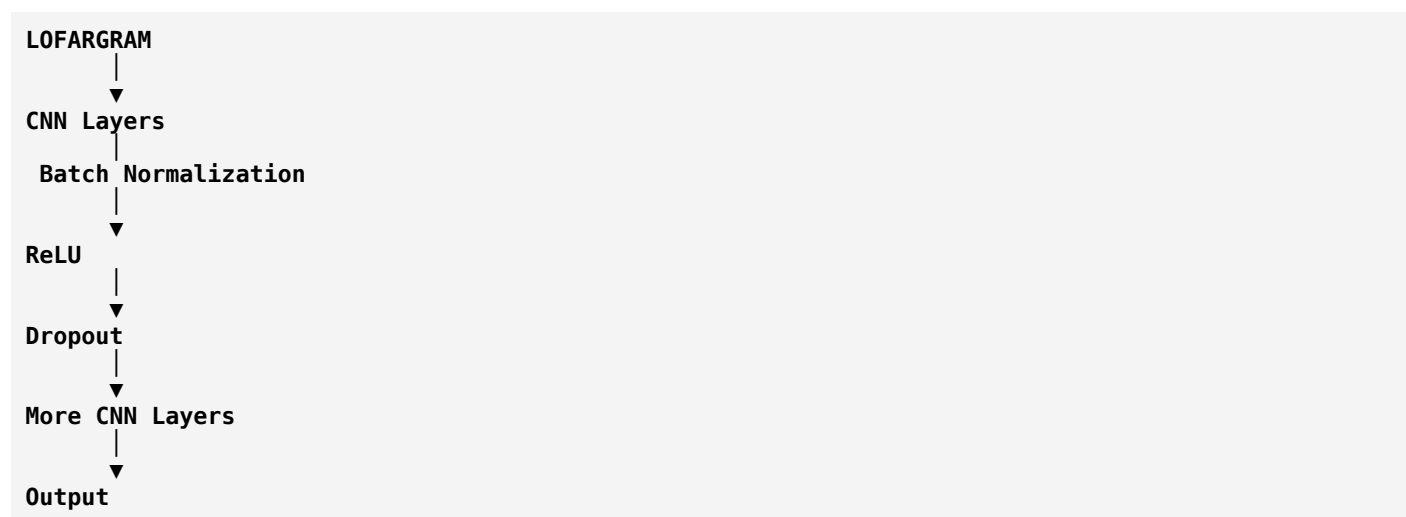
Benefits include:

- Faster training.
- More stable optimization.
- Often allows higher learning rates.
- Can provide a small regularization effect.

Batch Normalization does **not** replace dropout—they solve different problems and are often used together.

Putting It All Together

Imagine training a CNN for your sonar project.



During training:

- Batch Normalization stabilizes learning.
- Dropout reduces overfitting.
- Weight Decay keeps weights under control.
- Early Stopping prevents training too long.
- Data Augmentation provides more diverse training examples.

Together, these techniques help produce a model that performs well on **new sonar recordings**.

Summary Table

Technique	Purpose	Main Benefit
Dropout	Randomly disable neurons during training	Reduces overfitting
Early Stopping	Stop when validation stops improving	Prevents over-training
Data Augmentation	Create realistic variations of training data	Improves generalization
Weight Decay (L2)	Penalize very large weights	Encourages simpler models
Batch Normalization	Normalize activations during training	Faster, more stable training

Real-Life Analogy

Preparing for an exam:

- **Dropout** → Practice without always relying on your strongest friend.
- **Early Stopping** → Stop revising once you're well prepared instead of studying endlessly.
- **Data Augmentation** → Solve many different types of practice questions.
- **Weight Decay** → Avoid overcomplicating your answers.
- **Batch Normalization** → Keep your learning process organized and consistent.

Final Takeaway

A good deep learning model is **not** the one with the highest training accuracy.

A good model is the one that performs reliably on **new, unseen data**.

Regularization techniques help achieve that goal.

Next Phase (One of the Most Important)

Now we'll move to **Deep CNN Architectures**, where you'll learn:

1. **AlexNet** – The CNN that sparked the modern deep learning revolution.
2. **VGG16** – Simplicity through deeper networks.
3. **GoogLeNet (Inception)** – Efficient multi-scale feature extraction.
4. **ResNet** – Residual (skip) connections and why they solved the vanishing gradient problem.
5. **DenseNet** – Dense connectivity between layers.
6. **MobileNet** – Lightweight CNNs for mobile and embedded devices.
7. **EfficientNet** – Balancing depth, width, and resolution for excellent accuracy and efficiency.

These architectures are especially important because research papers on passive sonar classification frequently compare or build upon them, particularly **ResNet** and **EfficientNet**.

Ok let's go for these deep CNN architecture but this time don't tell about every architecture at once tell about the each one by one and give me clear much more clarity I have to learn indepth working of each it's upto you how you will make me to get the clarity

Perfect. I actually **prefer** teaching it this way.

If I explain all architectures (AlexNet, VGG, ResNet, DenseNet, MobileNet, EfficientNet) in one lesson, you'll only memorize names.

Instead, we'll study them **like a university course**, where each architecture builds on the limitations of the previous one.

This way, when you read a research paper that says:

"We used ResNet-18 because of residual learning..."

you'll immediately understand **why**, not just **what**.

Our New Roadmap (Deep CNN Architectures)

We'll spend several lessons on each architecture.

```
Part 1 → History of CNN Evolution
Part 2 → AlexNet (3-4 lessons)
Part 3 → VGGNet (2-3 lessons)
Part 4 → GoogLeNet/Inception (3-4 lessons)
Part 5 → ResNet (5-6 lessons)
Part 6 → DenseNet (3 lessons)
Part 7 → MobileNet (3 lessons)
Part 8 → EfficientNet (4-5 lessons)
Part 9 → Comparison of all architectures
Part 10 → Which architecture is best for your sonar project?
```

We'll understand:

- Why it was invented.
- What problem it solved.
- Its architecture.
- How data flows inside it.
- Advantages.
- Limitations.
- Why researchers still use it (or don't).
- Why it matters for your passive sonar project.

Lesson 1: The Evolution of CNNs (Before AlexNet)

Before studying AlexNet, you need to understand **why AlexNet changed the world**.

Every architecture exists because the previous one had a limitation.

Deep learning evolved by solving one problem after another.

Before 2012: Computer Vision Was Very Different

Imagine you want to detect a ship from a LOFARGRAM today.

You would probably do:

```
LOFARGRAM
  ↓
CNN
  ↓
Prediction
```

Simple.

But before 2012, researchers couldn't do this.

Instead, they followed a long pipeline.

Traditional Machine Learning Pipeline

```
Raw Signal
  ↓
FFT
  ↓
Spectrogram
  ↓
Handcrafted Features
(MFCC, HOG, SIFT, etc.)
  ↓
Classifier
(SVM, Random Forest)
  ↓
Prediction
```

Notice something important.

The computer **did not learn the features**.

Humans manually designed them.

What are Handcrafted Features?

Suppose you want to detect ships.

An engineer might say:

"Ship engines usually have strong harmonic frequencies."

So they manually calculate:

- Energy
- MFCC
- Spectral Centroid

- Zero Crossing Rate
- Band Energy

Then they feed these numbers into an SVM.

The success of the system depends heavily on whether the engineer chose good features.

The Biggest Problem

Imagine you're a doctor.

Every patient is different.

Would you want to manually write rules for every disease?

Probably not.

You'd rather have a system that learns patterns automatically.

The same applies to images and sonar.

Researchers wanted a model that could say:

"Don't tell me which features to use. I'll learn them myself."

That idea led to deep learning.

Why Was It Difficult?

People already knew about neural networks in the 1980s and 1990s.

So why didn't they use them?

Because of three major problems.

Problem 1: Small Datasets

Neural networks need a lot of data.

Back then, image datasets were tiny.

Example:

```
500 images
1000 images
2000 images
```

That's nowhere near enough to train a deep network.

Problem 2: Slow Computers

Training a deep network requires billions of calculations.

CPUs were simply too slow.

Training one large model could take weeks or months.

Then researchers realized something important:

Graphics cards (GPUs) are extremely good at performing many calculations in parallel.

This changed everything.

Problem 3: Better Training Techniques Were Needed

Researchers gradually developed:

- Better activation functions (like ReLU)
- Better optimization algorithms
- Better initialization methods
- Better regularization techniques (Dropout)
- Larger labeled datasets

These advances made deep networks practical.

Then Came ImageNet

One of the biggest breakthroughs was the creation of **ImageNet**.

ImageNet contains **millions of labeled images** across thousands of object categories.

For the first time, researchers had enough data to train deep CNNs effectively.

This dataset became the benchmark for computer vision research.

The ImageNet Challenge (ILSVRC)

Every year, researchers competed to build the best image classification model.

Before 2012, progress was gradual.

Then something incredible happened.

The Revolution

In **2012**, a CNN called **AlexNet** dramatically outperformed traditional computer vision methods in the ImageNet competition.

Its improvement was so significant that it convinced much of the research community that deep learning was the future of computer vision.

This event is widely regarded as the beginning of the modern deep learning era.

Why AlexNet Won

AlexNet wasn't just "a bigger CNN."

It combined several key ideas:

- A deeper architecture than many earlier successful models.
- **ReLU** activation instead of slower activation functions like sigmoid.
- **Dropout** to reduce overfitting.
- GPU-based training.
- Training on the large ImageNet dataset.

Each of these contributed to its success.

We'll study each one in detail.

Why Should You Care?

Your passive sonar project also follows the same philosophy.

Old approach:

```
Signal
↓
MFCC
↓
SVM
```

Modern approach:

```
Signal
↓
LOFARGRAM
↓
CNN
```

Instead of manually deciding which features are important, the CNN learns useful features directly from the LOFARGRAM.

That is exactly the revolution AlexNet helped popularize.

Summary

Before AlexNet	After AlexNet
Handcrafted features	Automatic feature learning
Traditional ML (SVM, etc.)	Deep CNNs
CPU-based training	GPU-accelerated training
Shallow models	Deep neural networks
Feature engineering by humans	Feature learning by the model

What We'll Learn Next (AlexNet Part 1)

The next lesson is entirely about **AlexNet**.

We'll answer questions like:

- What exactly is AlexNet?
- Who invented it?
- Why did it outperform other methods?
- What is its complete architecture?
- Why does it have 8 layers?
- How does an image travel from input to output?
- What is happening inside each layer?

We'll go layer by layer so that by the end, you'll be able to explain AlexNet from memory and understand why it became one of the most influential CNN architectures in history.

From this point onward, we're entering the architecture level of deep learning—the same concepts you'll encounter in research papers on passive sonar classification.

Excellent. You're now entering the level where you'll understand **why there are so many ResNet versions** and why almost every sonar paper mentions **ResNet-18** or **ResNet-50**.

This lesson is one of the most important for your project.

Deep CNN Architectures

Chapter 3: ResNet (Part 3)

Understanding ResNet-18, ResNet-34, ResNet-50, ResNet-101 & ResNet-152

First Question

Open any deep learning research paper and you'll see names like:

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

Students often think:

"These must be completely different networks."

They are not.

They all belong to the **ResNet family**.

The main difference is **how many layers (or residual blocks) they contain**.

Think of them like different models of the same car.

Real-Life Analogy

Imagine buying a laptop.

You have:

Laptop Series

↓

i3

↓

i5

↓

i7

↓

i9

All are laptops.

The difference is mainly in **power and complexity**.

Similarly:

ResNet Family

↓

ResNet-18

↓

ResNet-34

↓

ResNet-50

↓

ResNet-101

↓

ResNet-152

All use the **same core idea: Residual Learning**.

What Does the Number Mean?

The number roughly represents the **number of learnable layers** in the network.

Examples:

Model	Learnable Layers
ResNet-18	18
ResNet-34	34
ResNet-50	50
ResNet-101	101
ResNet-152	152

Notice something.

The architecture becomes deeper.

Why Make the Network Deeper?

Imagine identifying ships.

With a small network,

the model may only recognize:

- Engine harmonics
- Tonal lines

A deeper network can also learn:

- Complex engine modulation
- Cavitation patterns
- Higher-level acoustic signatures

More layers generally mean **greater representational power**.

But deeper isn't always better—you need enough data and computation to benefit from the extra depth.

How is ResNet Built?

Instead of thinking in terms of individual layers,

think in terms of **Residual Blocks**.

A Residual Block is the building block of the network.

Imagine LEGO.

One LEGO brick:

[Residual Block]

Now build a wall.

[Block]

↓

[Block]

↓

[Block]

↓

[Block]

The more blocks,
the deeper the network.

ResNet-18

ResNet-18 contains fewer residual blocks.

It is:

- Relatively lightweight
- Fast
- Easy to train
- Less computationally expensive

That's why it's extremely popular in research with limited datasets.

ResNet-152

Now imagine:

Many more residual blocks.

The network becomes much deeper.

Advantages:

- Can learn more complex features.

Disadvantages:

- Requires much more computation.
- Needs more memory.
- Usually benefits from larger datasets.

Think Like a Student

Suppose two students prepare for an exam.

Student A studies:

10 chapters.

Student B studies:

100 chapters.

Who performs better?

You might say:

Student B.

But what if:

- The exam only covers 10 chapters?

Or

- Student B only had one day to study?

Then studying more wasn't necessarily helpful.

The same idea applies to deep networks.

The model depth should match the problem and the available data.

Why ResNet-18 is Popular in Sonar Research

Now let's connect this to your project.

Imagine your dataset.

You are **not** working with ImageNet.

You are working with sonar data, where datasets are often much smaller.

Using ResNet-152 might:

- Overfit more easily.
- Require much longer training.
- Offer little or no improvement.

ResNet-18 often provides an excellent balance between:

- Accuracy.
- Speed.
- Generalization.

That's one reason why it appears so often in passive sonar classification papers.

Two Types of Residual Blocks

This is something many beginners don't know.

Not every ResNet uses the same kind of block.

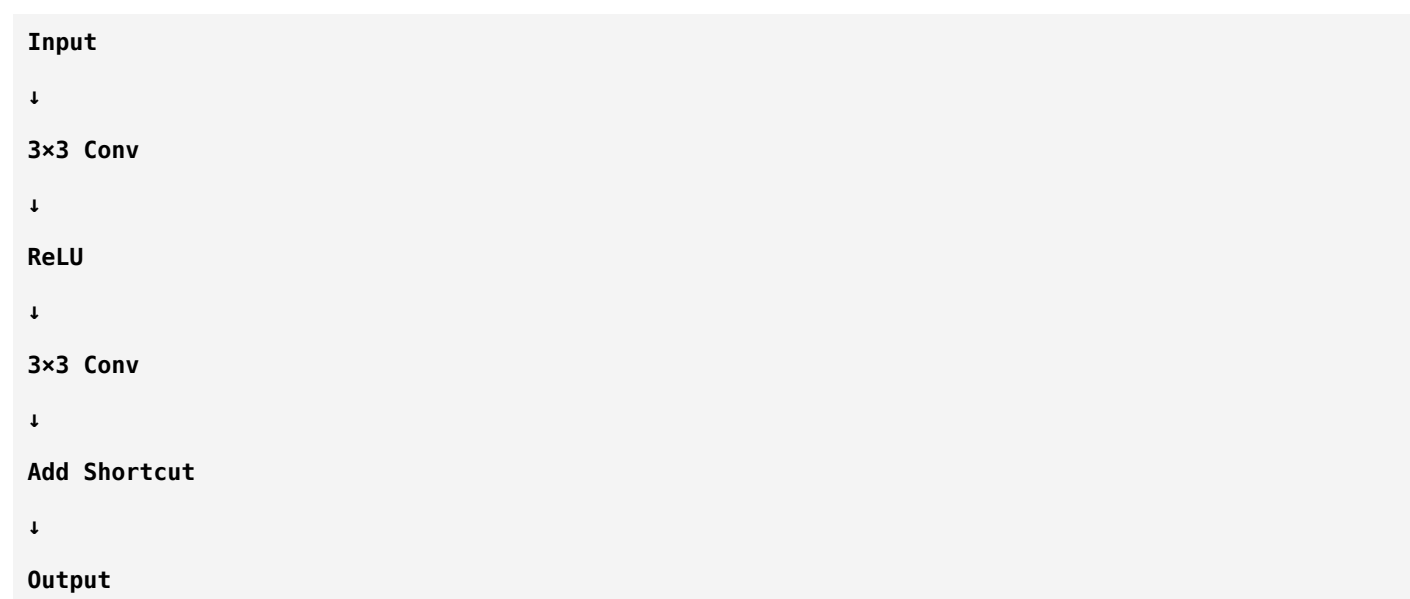
There are **two main block types**.

Type 1: Basic Block

Used in:

- ResNet-18
- ResNet-34

Structure:



Simple.

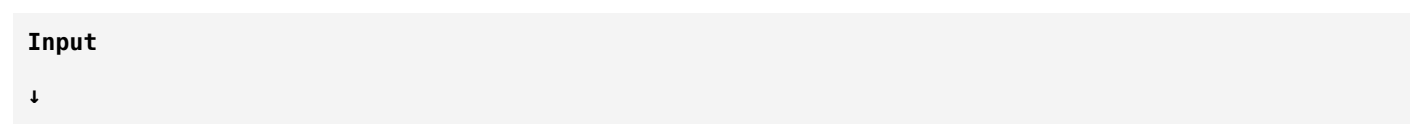
Easy to understand.

Type 2: Bottleneck Block

Used in:

- ResNet-50
- ResNet-101
- ResNet-152

Structure:



```

1×1 Conv
↓
3×3 Conv
↓
1×1 Conv
↓
Add Shortcut
↓
Output

```

This looks more complicated.

Why?

Because researchers wanted to make very deep networks more computationally efficient.

We'll study bottleneck blocks in detail in the next lesson.

Why Introduce 1×1 Convolutions?

At first glance, a **1×1 convolution** seems useless.

Students ask:

"How can a filter that only looks at one pixel help?"

Excellent question.

The answer is:

It doesn't look across neighboring pixels.

Instead, it learns to **mix and transform information across the channel dimension**.

It can also reduce or increase the number of channels, making later computations much cheaper.

This is one of the clever ideas behind deeper ResNets.

We'll cover it thoroughly next.

Comparing the Models

Model	Block Type	Advantages	Trade-offs
ResNet-18	Basic Block	Fast, lightweight	Lower capacity
ResNet-34	Basic Block	More expressive than 18	More computation
ResNet-50	Bottleneck	Strong accuracy, efficient for its depth	More complex
ResNet-101	Bottleneck	Very powerful	Heavy computation
ResNet-152	Bottleneck	Extremely deep	High memory and compute requirements

Which One Should You Use?

Suppose you're classifying:

- Dogs.
- Cats.
- Cars.

With millions of images.

A deeper model like ResNet-50 or ResNet-101 might make sense.

Now consider your passive sonar project.

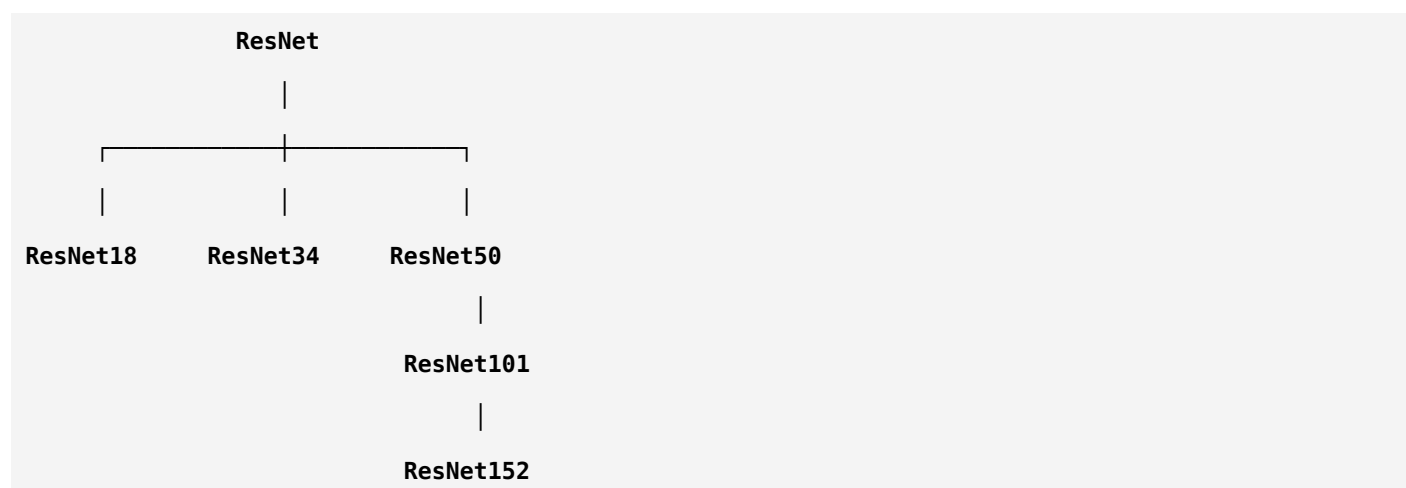
Typical datasets are much smaller.

In many cases:

ResNet-18 or ResNet-34 is a very sensible starting point because they provide strong performance without requiring enormous amounts of data or computation.

Of course, the best choice depends on your dataset size, hardware, and performance goals.

Visualizing the Family



The concept is the same.

Only the depth and block design change.

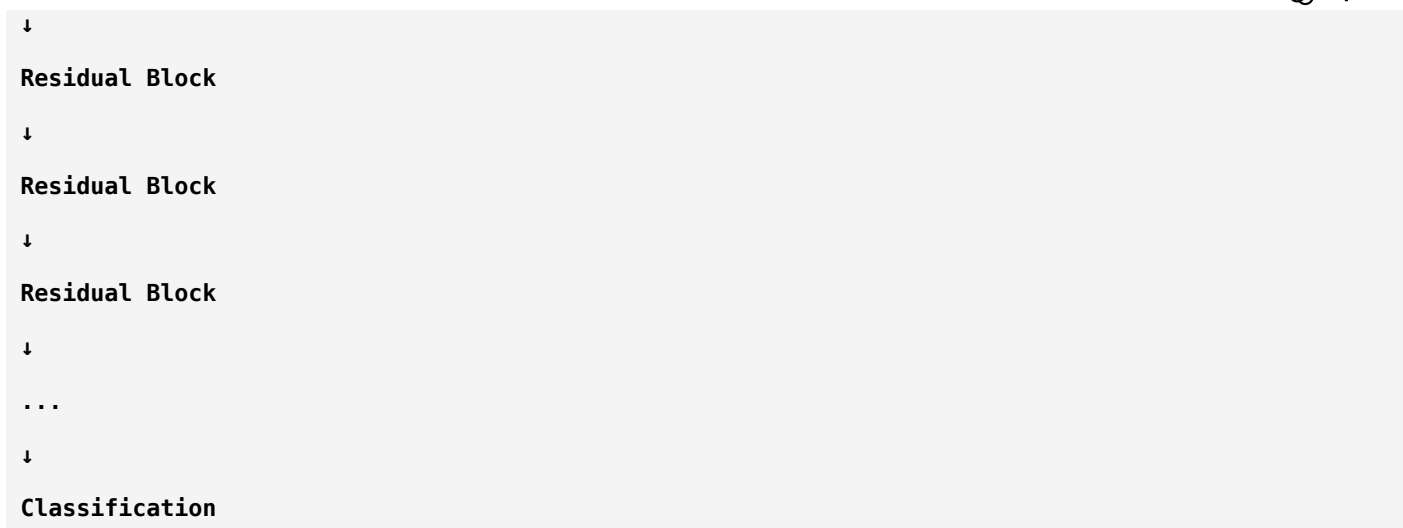
How Does a LOFARGRAM Flow Through ResNet?

Imagine your input.

LOFARGRAM

↓

Conv Layer



Each block gradually improves the learned representation.

Instead of relearning everything,

each block asks:

"What useful information should I add to what I already know?"

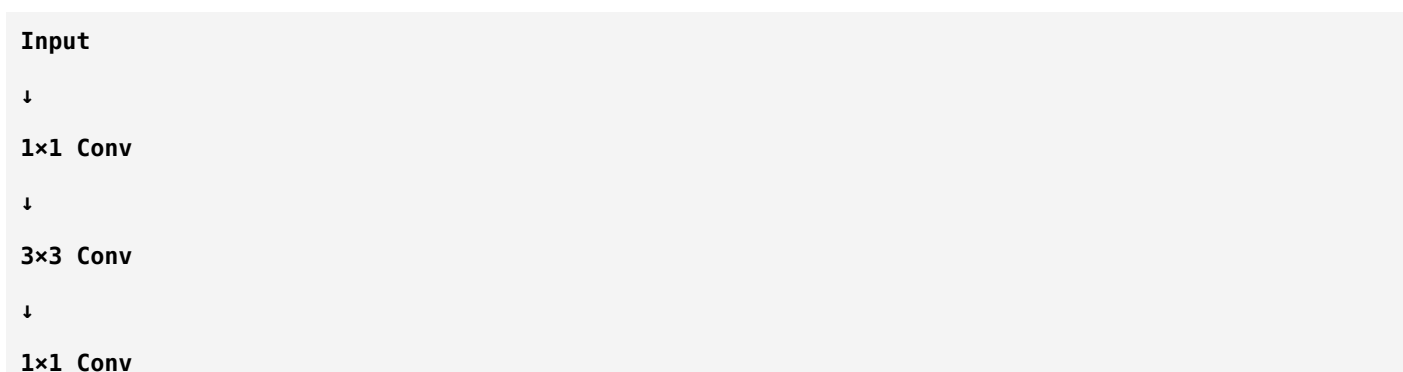
That's the heart of residual learning.

Summary

Concept	Key Idea
ResNet Family	Same core architecture, different depths
ResNet-18	Lightweight, Basic Blocks
ResNet-34	Deeper Basic Blocks
ResNet-50+	Uses Bottleneck Blocks
Deeper Networks	More capacity but higher computational cost
Sonar Applications	ResNet-18 is a common and practical choice for many datasets

One Thought Before the Next Lesson

Today we saw this block:



↓

Output

It probably looks strange.

You may be wondering:

"Why would anyone use a 1×1 convolution?"

That question led to another important breakthrough in CNN design.

Next Lesson (ResNet Part 4)

In the next lesson, we'll go deep into the **Bottleneck Block**, where you'll learn:

- What exactly is a **1×1 convolution**?
- How can it reduce computation without throwing away useful information?
- Why does ResNet-50 use bottleneck blocks instead of basic blocks?
- How does the bottleneck design make very deep networks practical?
- Why do modern architectures like **DenseNet**, **EfficientNet**, and many others also use **1×1 convolutions**?

By the end of that lesson, you'll understand one of the smartest design ideas in modern CNNs.

Next

Excellent! This lesson is one of the **deepest concepts in CNN architecture**.

When students first see a **1×1 convolution**, almost everyone says:

"This makes no sense."

Because they think:

"A convolution should look at neighboring pixels."

A 1×1 filter doesn't even look at neighbors.

So...

Why does it exist?

Today we'll answer that.

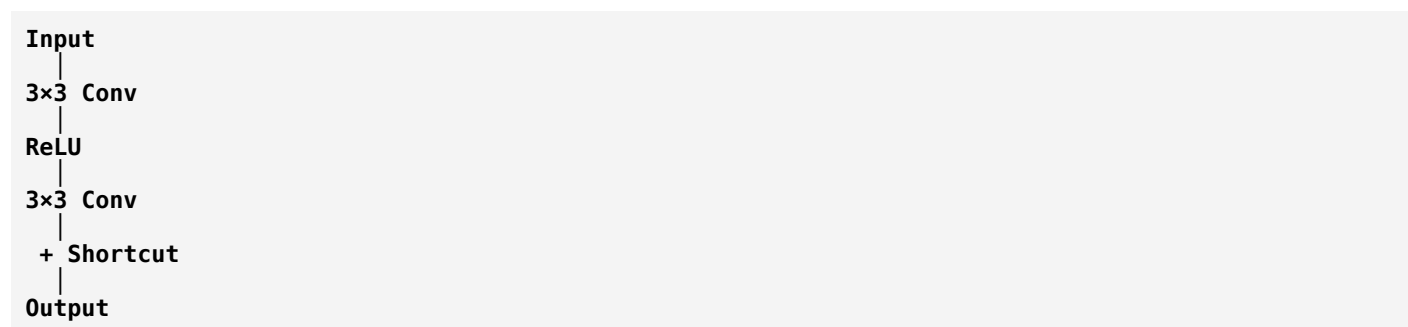
Deep CNN Architectures

Chapter 3: ResNet (Part 4)

The Bottleneck Block & the Mystery of 1×1 Convolutions

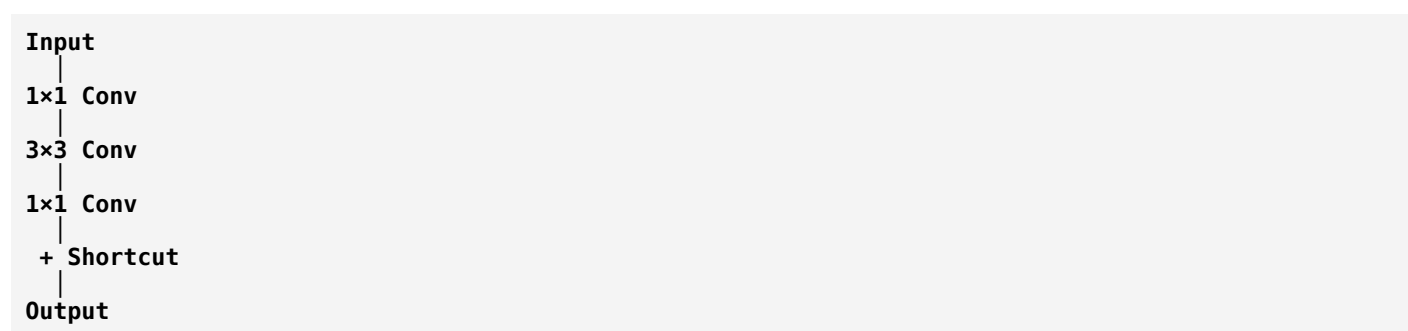
First, Let's Recall the Basic Block

In ResNet-18, one block looks like:



Simple.

Now look at ResNet-50.



Students immediately ask:

"Why add two extra layers?"

Good question.

Imagine a Big Library

Suppose you enter a library with **1000 books**.

You only need books about:

- Ships
- Ocean Acoustics
- Signal Processing

Would you carry all 1000 books everywhere?

No.

You would first select only the useful books.

Then study them.

Then organize your final notes.

That is exactly what a Bottleneck Block does.

The Secret of Channels

Remember from previous lessons:

After several convolution layers, your feature map may look like:

56 × 56 × 256

Let's understand this carefully.

- Height = 56
- Width = 56
- Channels = 256

Those **256 channels** represent **256 different learned features**.

Think of them as 256 experts.

Expert 1:

Looks for harmonic lines.

Expert 2:

Looks for broadband noise.

Expert 3:

Looks for engine modulation.

...

Expert 256:

Looks for another acoustic pattern.

The Problem

Now suppose we apply a **3×3 convolution** directly.

Every filter has to process **all 256 channels**.

That is computationally expensive.

Imagine asking one person to interview **256 experts** before making a decision.

Slow.

The Brilliant Idea

Instead of using all 256 channels,

what if we first reduce them?

Example:

56 × 56 × 256

↓

1×1 Conv

↓

56 × 56 × 64

Notice something.

Height:

Still 56.

Width:

Still 56.

Only the channels changed:

256

↓

64

The network has compressed the information.

Wait...

How Can 1×1 Convolution Reduce Channels?

Excellent question.

This is where most tutorials stop.

Let's understand it.

Think About Mixing Colors

Imagine you have:

- Red
- Green
- Blue

You mix them.

The result is a new color.

Did you look at neighboring pixels?

No.

You only mixed information at the **same pixel location**.

A **1×1 convolution** does the same thing.

It combines information **across channels**, not across space.

Example

Suppose one pixel has values:

Channel 1 = 10

Channel 2 = 7

Channel 3 = 15

Channel 4 = 3

A 1×1 filter learns weights like:

0.2

0.5

-0.3

0.8

It combines those channel values into a new feature.

The filter is learning:

"How important is each channel?"

This is why people often describe 1×1 convolutions as **channel mixers** or **channel projections**.

Three Stages of a Bottleneck Block

Now let's understand the complete design.

Stage 1: Reduce

Input:

56 × 56 × 256

↓

1×1 Conv

↓

56 × 56 × 64

Purpose:

Reduce the number of channels.

This saves computation.

Stage 2: Learn

Now apply:

3×3 Conv

But notice:

Instead of processing **256 channels**,

it only processes **64 channels**.

Much cheaper.

This is where the main spatial feature learning happens.

Stage 3: Expand

Finally,

another:

1×1 Conv

restores the channel dimension.

Example:

56 × 56 × 64

↓

56 × 56 × 256

Now the output has the expected shape again, so it can be combined with the shortcut path.

Real-Life Analogy

Imagine writing a report.

Step 1:

Remove unnecessary details.

↓

Step 2:

Analyze the important information.

↓

Step 3:

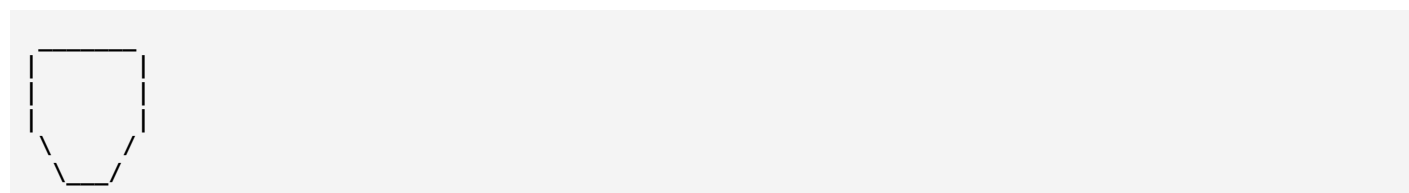
Write the complete report again.

You don't analyze everything.

You analyze the essential parts.

Why Is This Called a Bottleneck?

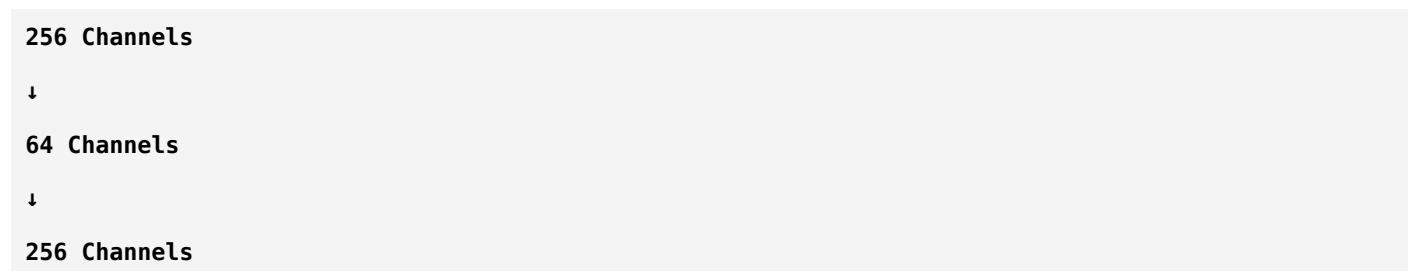
Think of a water bottle.



The neck is narrow.

Everything passes through that narrow region.

Similarly:



The information is squeezed into a smaller representation before expanding again.

That's why it's called a **Bottleneck Block**.

Why Doesn't Information Get Lost?

Students often worry:

"If we reduce 256 channels to 64, aren't we throwing away information?"

A reasonable concern.

The answer is:

The network **learns** which channel combinations are most useful.

If reducing too aggressively hurts performance, training will adjust the weights to preserve important information.

So while compression can discard some redundancy, the goal is to keep the information that matters most for the task.

Sonar Example

Imagine your LOFARGRAM produces channels like:

- Engine harmonics
- Propeller tones
- Broadband noise
- Cavitation
- Ambient ocean noise
- Machinery modulation
- Other learned patterns

Not every channel is equally important for every decision.

The 1×1 convolution learns how to combine and emphasize the most informative channel patterns before the more expensive 3×3 convolution.

Why Is Bottleneck So Powerful?

Without a bottleneck:

3×3 Conv

↓

Huge Computation

With a bottleneck:

Reduce Channels

↓

3×3 Conv

↓

Expand Channels

The expensive spatial convolution operates on fewer channels, making the network much more computationally efficient.

This is one reason ResNet-50 can be much deeper than ResNet-18 while remaining practical.

Basic Block vs Bottleneck Block

Basic Block	Bottleneck Block
$3 \times 3 \rightarrow 3 \times 3$	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
Used in ResNet-18 & 34	Used in ResNet-50, 101 & 152
Simpler	More computationally efficient for deep networks
Good for shallower models	Better suited for very deep models

Why This Matters Beyond ResNet

The idea of **1×1 convolutions** didn't stop with ResNet.

You'll encounter them in many important architectures, including:

- **GoogLeNet (Inception)**
- **DenseNet**
- **MobileNet**
- **EfficientNet**

So understanding this concept now will make those architectures much easier to learn.

Key Takeaways

1. **1×1 convolutions do not look across neighboring pixels.**
2. **They learn relationships between channels.**
3. **They can reduce or increase the number of channels.**
4. **Bottleneck blocks reduce computation while preserving strong performance.**
5. **That's why ResNet-50 and deeper models use bottleneck blocks instead of basic blocks.**

Think Like an Engineer

Suppose you're building a passive sonar classifier.

Your feature maps have **512 channels**.

Would you:

- Run expensive 3×3 convolutions on all 512 channels?

Or

- First compress the channels to a smaller, informative representation, perform the expensive operation there, and then expand them again?

Modern CNN designers usually choose the second approach because it provides a much better balance between computational cost and representational power.

Next Lesson (ResNet Part 5 – Final)

In the final ResNet lesson, we'll connect everything you've learned and answer:

- **How does a complete LOFARGRAM travel through ResNet-18?**
- **What happens at every stage of the network?**
- **Why is ResNet-18 one of the most popular choices for passive sonar target classification?**
- **When should you choose ResNet-18 vs ResNet-50?**
- **How is ResNet used with transfer learning in real research papers?**

That lesson will tie the entire ResNet chapter directly to **your passive sonar project** and prepare you for understanding modern sonar classification research.

Ok next

Excellent. This is the **final chapter on ResNet**, where we'll connect everything directly to **your passive sonar target classification project**.

After this lesson, you won't just know ResNet—you'll understand **how researchers actually use it** in real-world sonar systems.

Deep CNN Architectures

Chapter 3: ResNet (Part 5 – Final)

ResNet in Passive Sonar Target Classification

The Big Picture

Let's assume your goal is:

Binary Classification

Input: Passive sonar recording

Output: Ship Present  or Ship Absent 

At first glance, this seems impossible.

How can a neural network understand an audio signal?

The answer is:

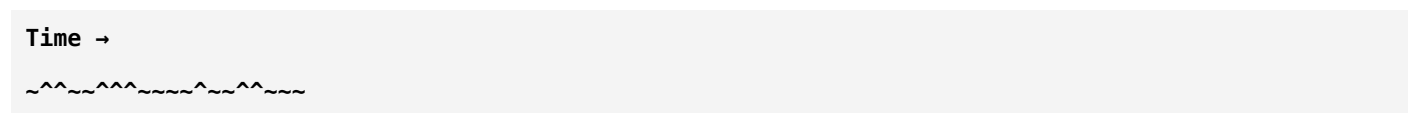
It doesn't work directly on raw audio in most traditional CNN pipelines.

Instead, we first convert the audio into a visual representation.

Step 1: Passive Sonar Signal

Imagine a hydrophone records sound.

The raw waveform looks like this:

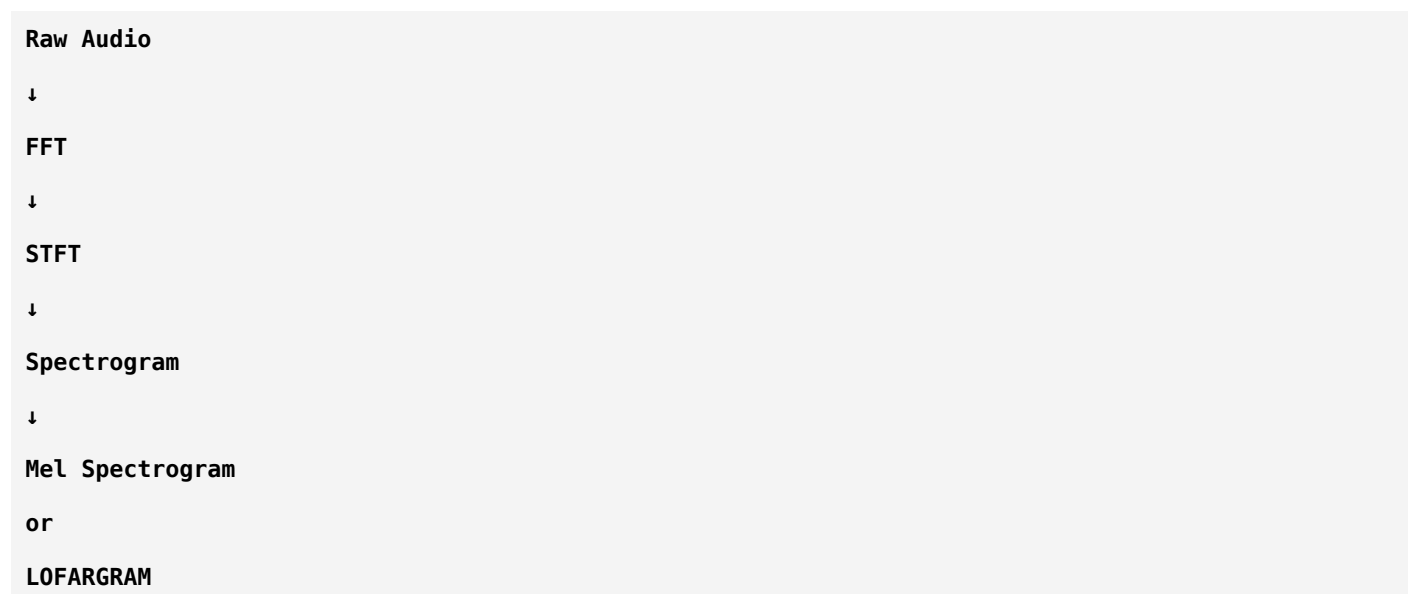


This waveform is difficult for a CNN to process directly.

So we transform it.

Step 2: Signal Processing

You already learned these techniques:

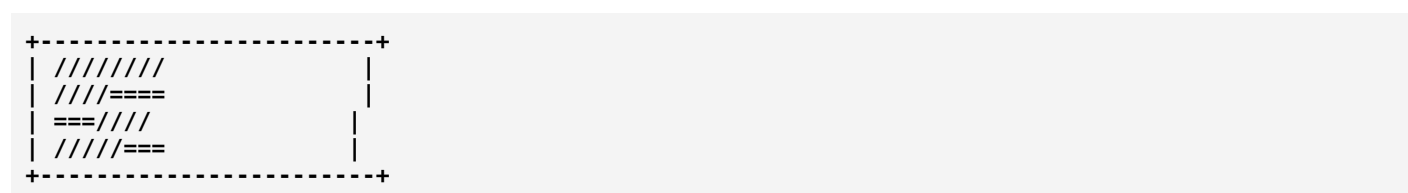


For passive sonar, **LOFARGRAM** is one of the most common choices because it highlights frequency components over time that are useful for detecting ship signatures.

Now the signal becomes an image-like representation.

Step 3: Feed the LOFARGRAM to ResNet

Imagine your LOFARGRAM:



To a human,
this looks like a heatmap.

To ResNet,
it's simply an image.

What Does the First Layer Learn?

The first convolution sees only small regions.

It learns simple patterns like:

- Horizontal lines
- Vertical lines
- Bright frequency bands
- Small edges

In sonar, these may correspond to:

- Narrow tonal lines
- Local changes in energy
- Short-duration events

These are the building blocks.

Middle Layers

The next residual blocks combine those simple patterns.

Instead of individual frequency lines,
they begin recognizing:

- Harmonic groups
- Repeating tonal structures
- Frequency trajectories

These are much more informative than isolated pixels.

Deep Layers

Now ResNet starts recognizing high-level acoustic patterns.

For example:

- Engine acoustic signatures
- Propeller-related harmonics
- Stable frequency structures associated with ships

At this point, the network isn't "seeing lines" anymore.
It's learning abstract representations that help distinguish:

Ship Present vs **Ship Absent**.

Final Classification

After many residual blocks:

```

LOFARGRAM
↓
Residual Blocks
↓
Global Average Pooling
↓
Fully Connected Layer
↓
Softmax / Sigmoid
↓
Prediction

```

For your binary problem, the final output might be:

```

Ship Present = 0.97
Ship Absent = 0.03

```

The network predicts **Ship Present** because it has the higher probability.

Why Global Average Pooling?

Earlier architectures like AlexNet and VGG used very large **fully connected layers**, which introduced millions of parameters.

ResNet instead uses **Global Average Pooling (GAP)** before the final classifier.

Imagine the last feature map:

```
7 × 7 × 512
```

Instead of flattening everything,

GAP computes the **average value of each channel**.

So:

```
7×7×512
```

↓

512 numbers

Each number summarizes one feature channel.

Advantages:

- Far fewer parameters.
- Lower risk of overfitting.
- Better generalization.
- Less memory usage.

This is one reason ResNet is much more efficient than VGG.

Why is ResNet-18 So Popular?

Suppose you have:

- 2,000 sonar recordings.

Would ResNet-152 be a good choice?

Probably not.

It has very high capacity and may overfit if the dataset is too small.

ResNet-18 often works well because:

- It's easier to train.
- It's computationally efficient.
- It generalizes well on many medium-sized datasets.
- It has significantly fewer parameters than deeper ResNets.

This is why you'll frequently see **ResNet-18** in passive sonar research.

Transfer Learning

This is one of the most important ideas for your project.

Imagine you train ResNet from scratch.

The network must learn:

- Basic edges.
- Frequency patterns.
- Intermediate structures.
- High-level representations.

That requires a lot of data.

Instead, researchers often start with a **ResNet pretrained on ImageNet**.

You might wonder:

"ImageNet contains natural images, not sonar. Why would this help?"

Because the **early layers** learn very general features like edges, gradients, and simple textures.

Those low-level feature extraction capabilities can still be useful when fine-tuned on sonar images.

Then researchers replace the final classification layer and train the network on their sonar dataset.

This process is called **Transfer Learning**.

Your Complete Pipeline

Now you can finally understand an end-to-end passive sonar classifier.



This is very close to the architecture used in many research papers.

Why ResNet Fits Your Project

Your project involves recognizing subtle acoustic patterns.

ResNet helps because:

- Residual connections improve optimization.
- It learns hierarchical features automatically.
- It is robust enough for complex acoustic images.
- It works well with transfer learning.
- ResNet-18 offers an excellent balance of accuracy and efficiency.

Comparing AlexNet, VGG, and ResNet

Feature	AlexNet	VGG	ResNet
Main Idea	First successful deep CNN	Showed the power of depth	Solved the degradation problem with residual learning
Learnable Layers	8	16 / 19	18-152+
Residual Connections	✗	✗	✓
Global Average Pooling	✗	✗	✓
Large Fully Connected Layers	✓	✓	Minimal
Common in Modern Sonar Research	Rare	Sometimes	Very common

A Researcher's Thought Process

When reading a sonar paper that says:

"We used ResNet-18."

You should now ask:

- Why ResNet-18 instead of VGG?
- How large was their dataset?
- Did they use transfer learning?
- What input representation did they use? (LOFARGRAM, spectrogram, Mel spectrogram?)
- Why was ResNet-18 sufficient?

These are the kinds of questions that distinguish someone who understands the field from someone who only knows the model names.

What You've Learned So Far

You now understand:

Digital Audio Processing

- FFT
- STFT
- Spectrogram
- Mel Spectrogram
- MFCC
- LOFARGRAM

Machine Learning

- Features
- Labels
- Training
- Validation
- Testing
- Loss
- Gradient Descent
- Backpropagation
- Overfitting
- Regularization

Deep Learning

- Neurons
- Activation Functions
- CNNs
- Convolution
- Pooling
- Feature Maps
- ReLU
- Dropout
- Batch Normalization

CNN Architectures

- AlexNet
- VGG
- ResNet

You've built a solid foundation.

Where Do We Go Next?

Now we move from **classic CNNs** to **modern architectures**.

The order I recommend is:

1. **DenseNet** (understand dense connections and feature reuse)
2. **Inception / GoogLeNet** (multi-scale feature extraction)

3. **EfficientNet** (compound scaling; very relevant to your project)
 4. **MobileNet** (lightweight models)
 5. **Vision Transformers (ViT)** (modern alternative to CNNs)
-

★ My recommendation

Since your project is **Passive Sonar Based Target Ship Classification**, the next architecture should be **DenseNet**.

Why?

Because many sonar classification papers compare:

- ResNet
- DenseNet
- EfficientNet

DenseNet introduces a different idea from ResNet:

Instead of connecting only the input to the output of a block, what if every layer could directly access the outputs of all previous layers?

That simple idea led to another major breakthrough in deep learning.

From here onward, you'll start recognizing the design philosophy behind almost every modern CNN architecture.